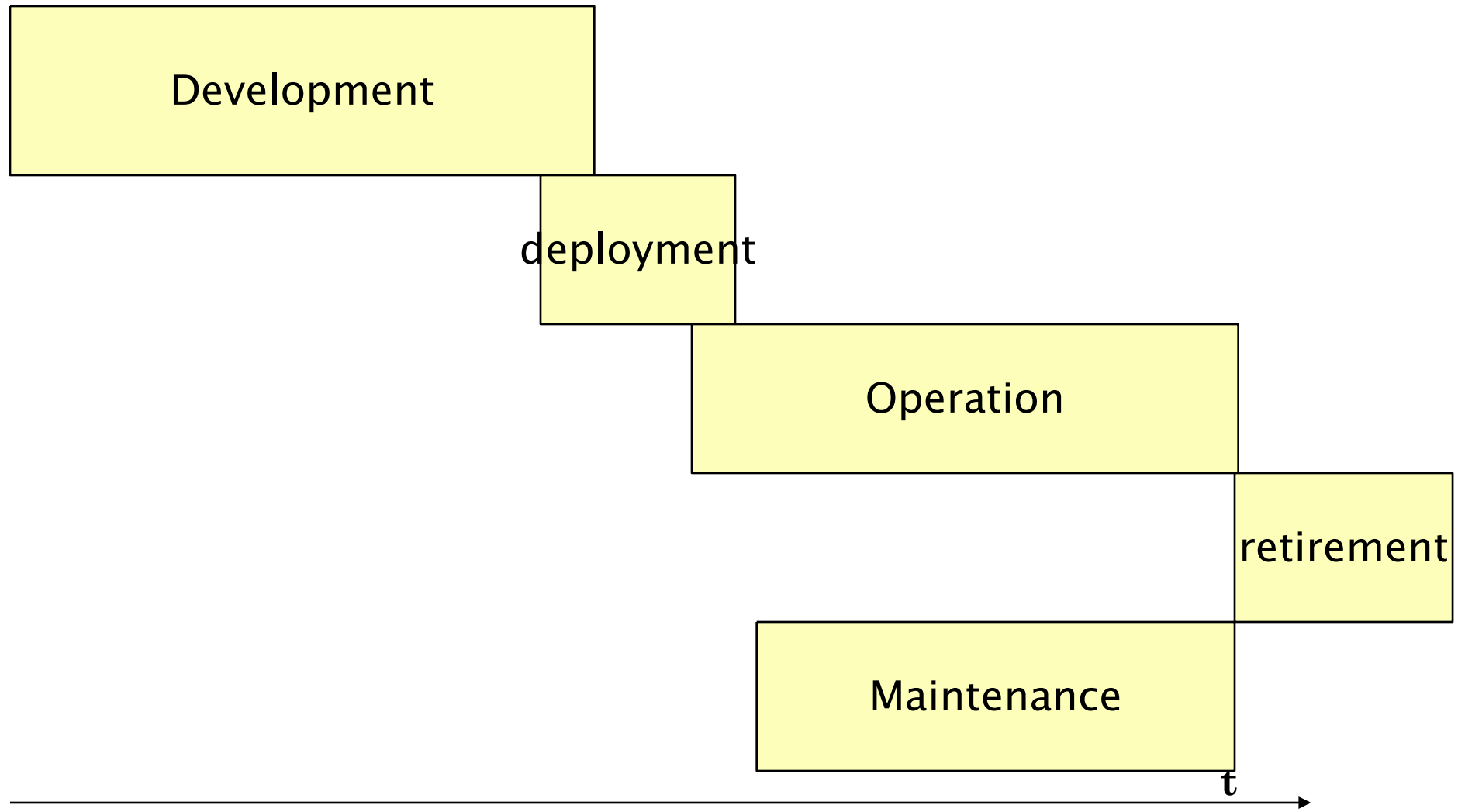
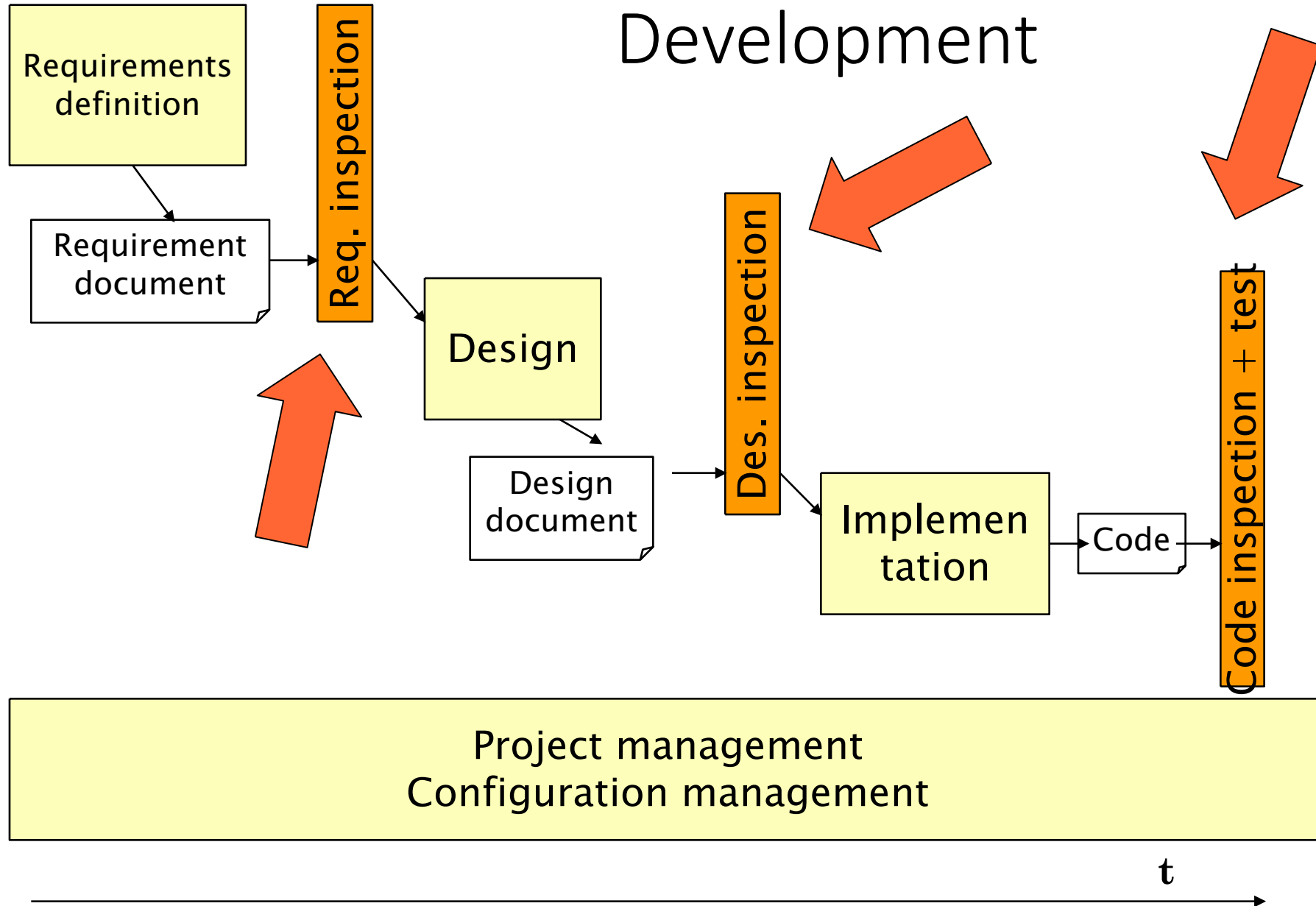


V&V

Main Phases



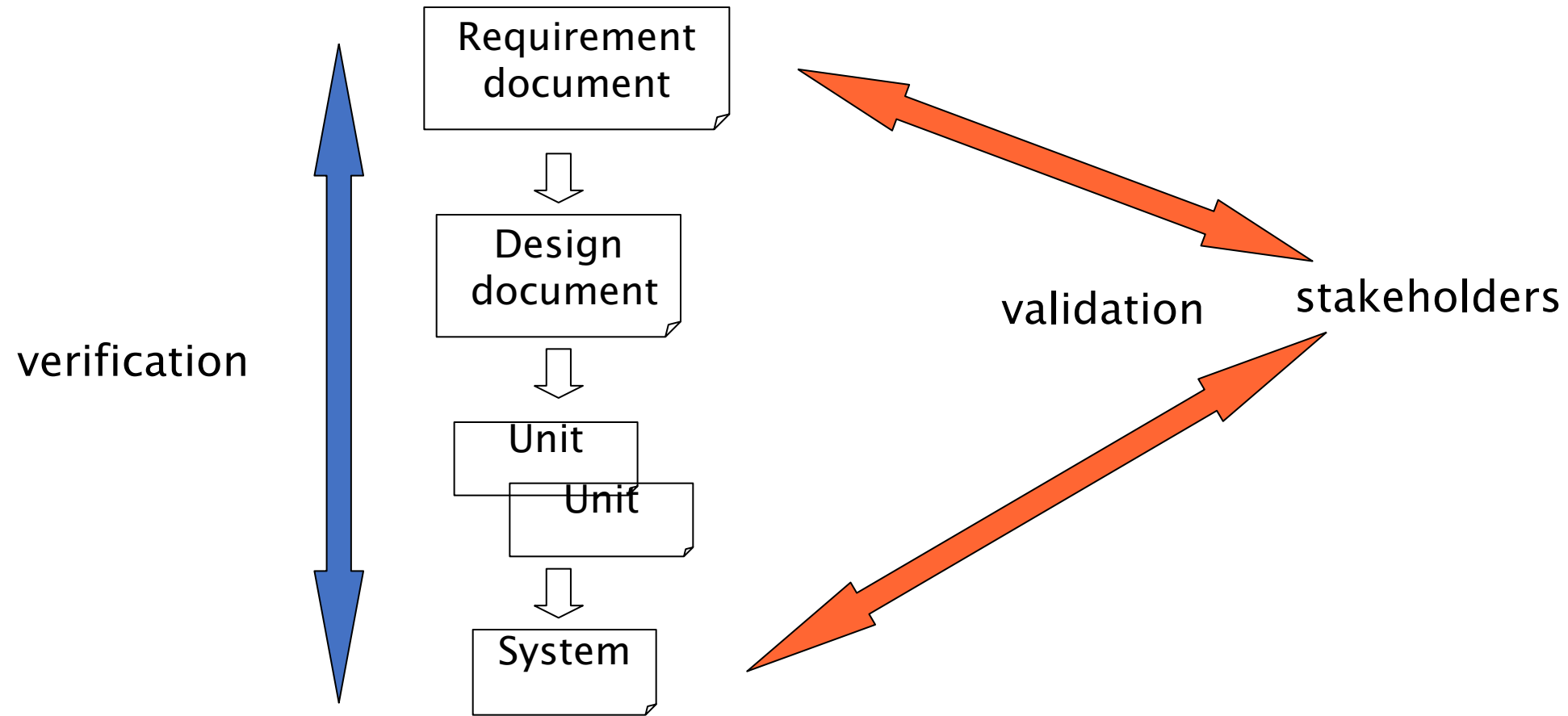
Development



V&V

- Validation
 - is it the right software system?
 - effectiveness
 - external (vs user)
 - reliability
- Verification
 - is the software system right?
 - efficiency
 - internal (correctness of vertical transformations)
 - correctness

V & V



Scenario1 in a dev process

- Stakeholder
 - Real need: big car /6 seats
- Developers
 - R1: compact car (4 seats)
 - Result : compact car (4 seats)
 - Verification: passed
 - Validation : not passed

Scenario2 in a dev process

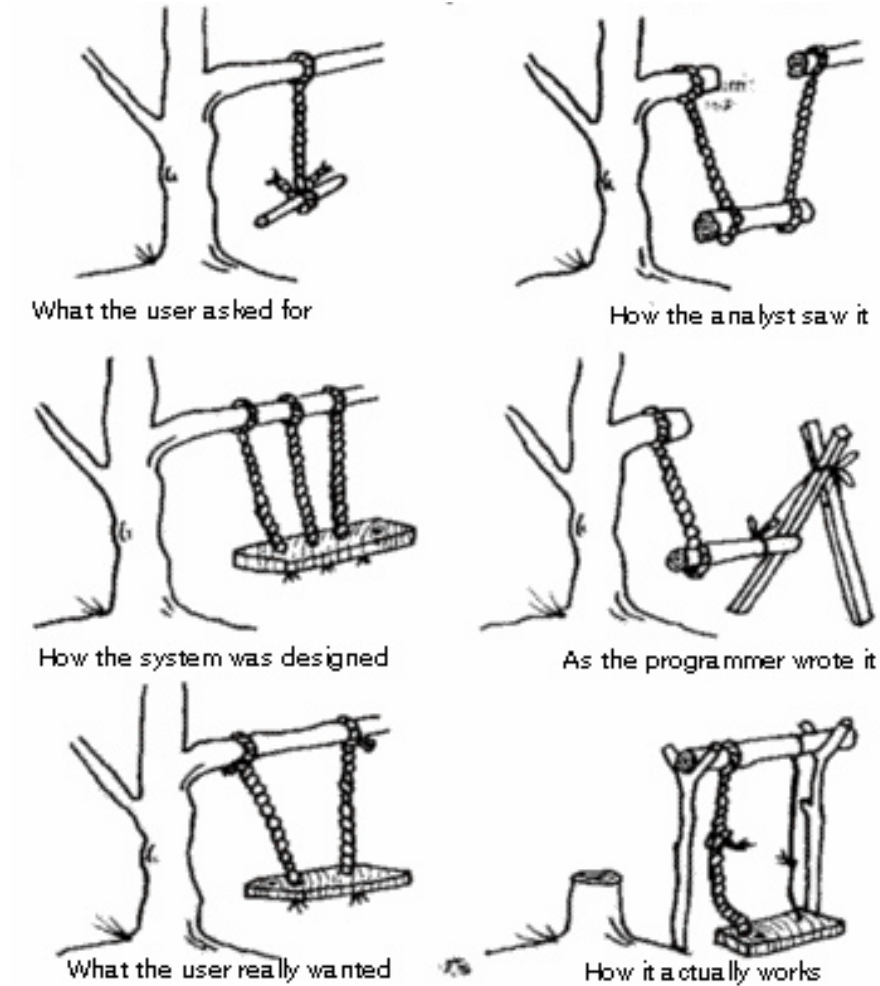
- Stakeholder
 - Real need: big car /6 seats
- Developers
 - R1: big car (6 seats)
 - Result : big car (6 seats)
 - Verification: passed
 - Validation : passed

Scenario3 in a dev process

- Stakeholder
 - Real need: big car /6 seats
- Developers
 - R1: big car (6 seats)
 - Result : compact car (4 seats)
 - Verification: not passed
 - Validation : passed on req doc, not passed on delivered product

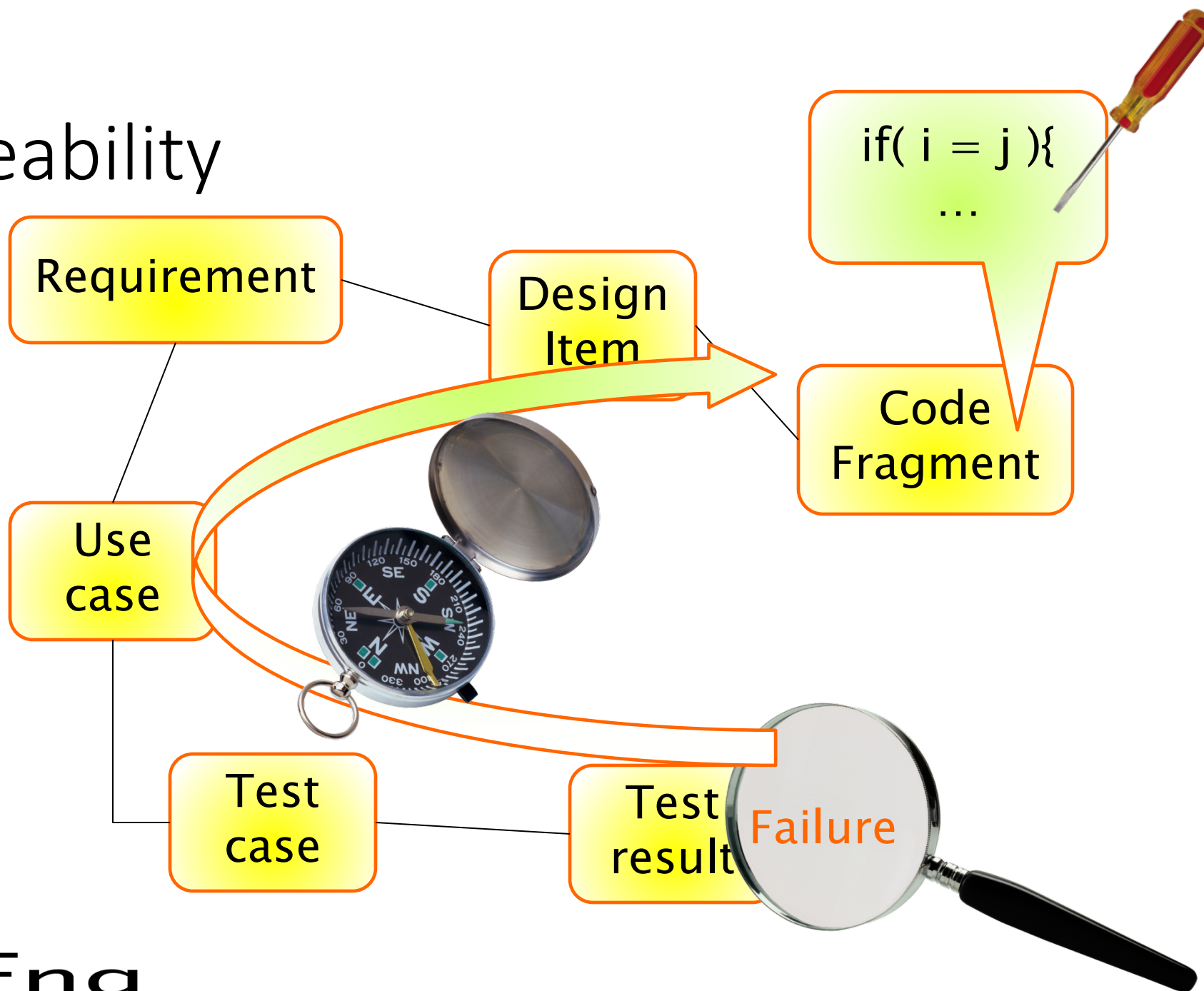
Requirements

- The root...

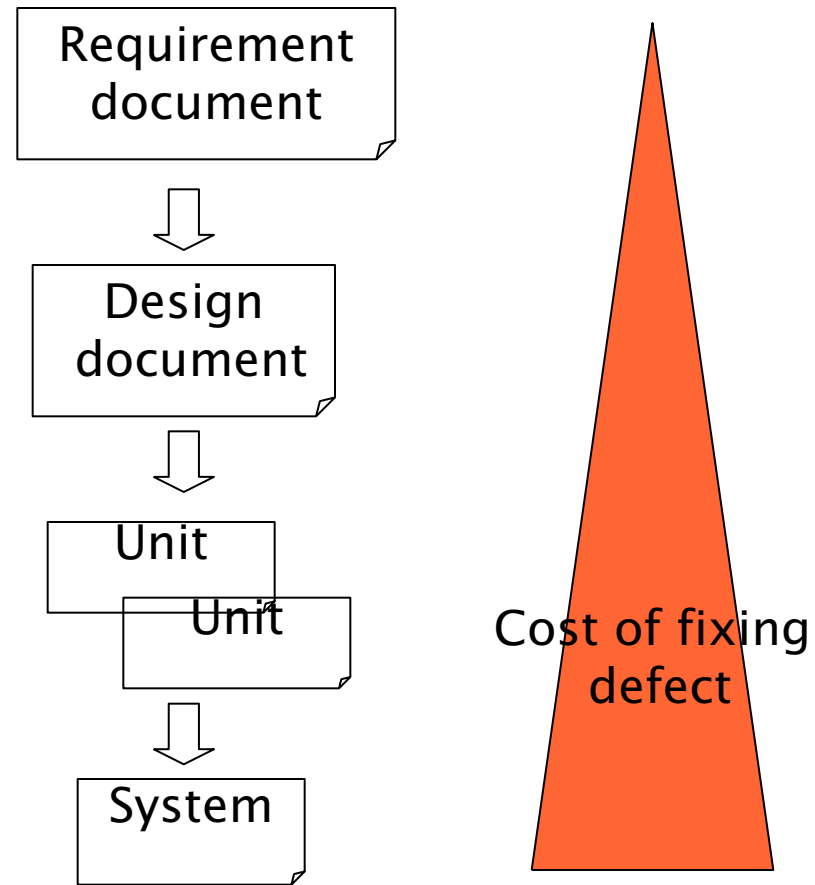


...of all evils

Traceability



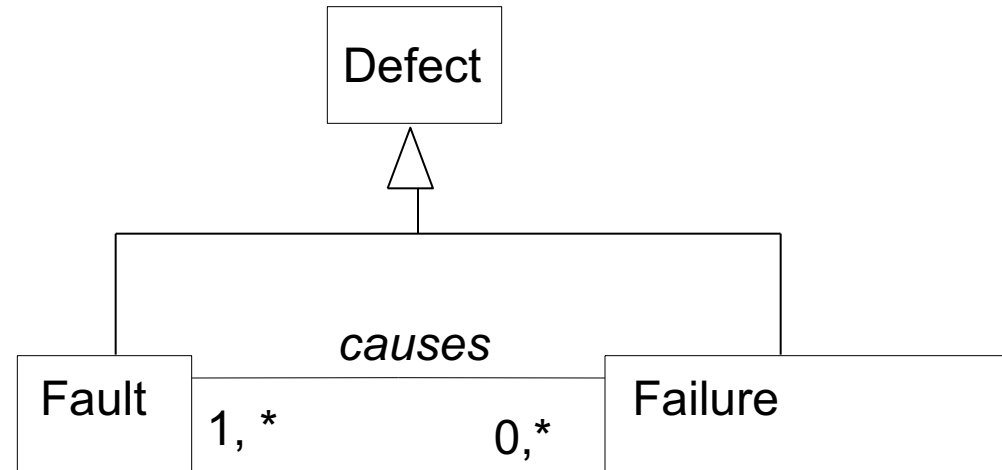
V & V vs. cost of fixing defect



Failure, fault, defect

- Failure
 - An execution event where the software behaves in an unexpected way
- Fault
 - The feature of software that causes a failure
 - May be due to:
 - An error in code
 - Incomplete/incorrect requirements
- Defect
 - Failure or fault

Failure fault defect



Bicycle

- **Failure**

- User falls down
- R1 not satisfied

Requirements

R1 transport person

- Why?

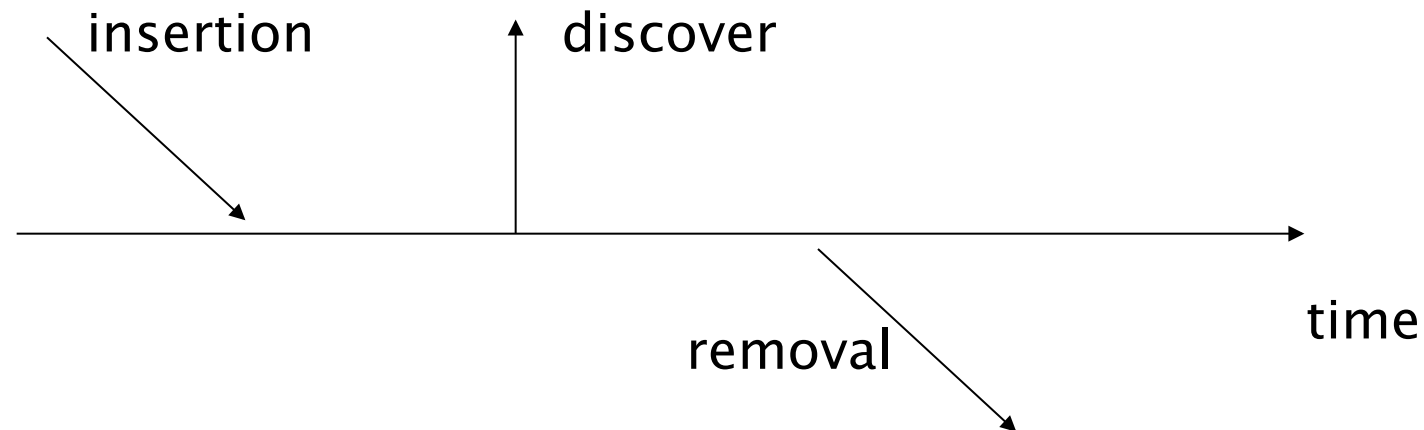
Bicycle

- **Fault**

- where the failure comes from?
 - Handlebar ill mounted? (implementation problem)
 - Saddle too high? (configuration problem)
 - Wheels too far apart? (design problem)
- User cannot bike? (not a bicycle failure)
- Pothole in the road? (not a bicycle failure)

Insertion / removal

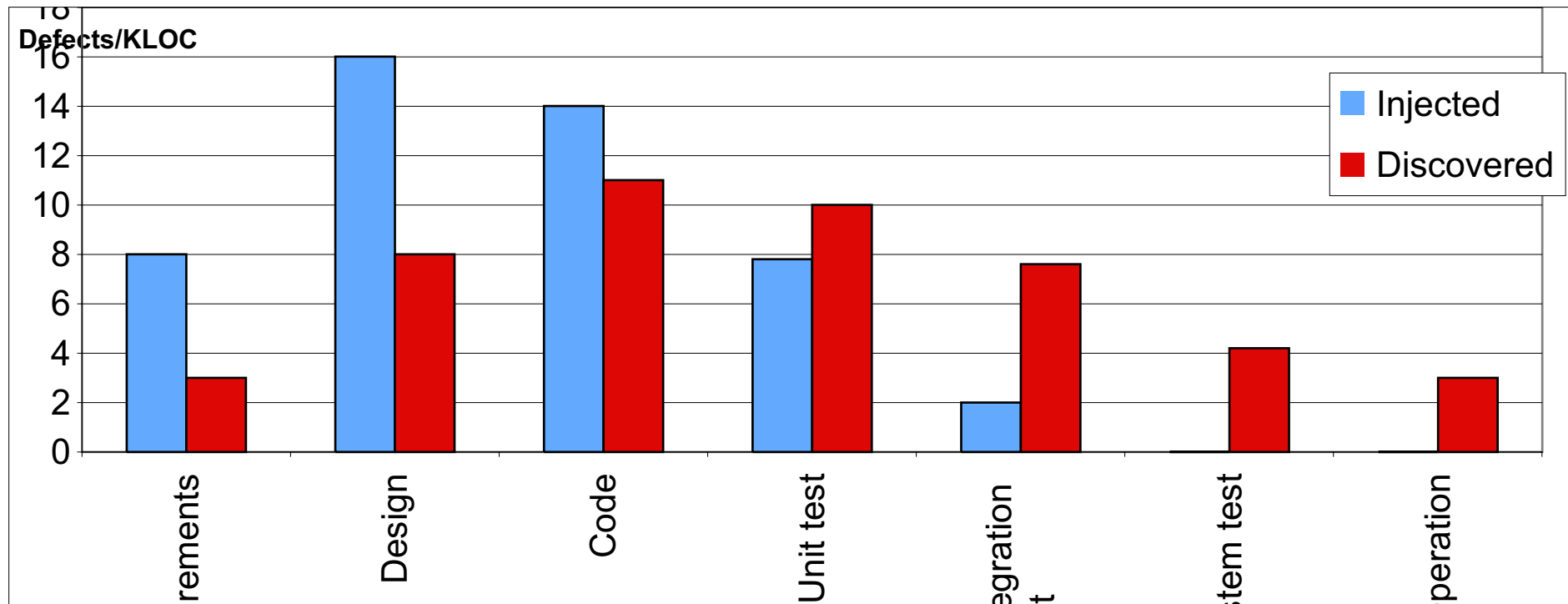
- Defect is characterized by
 - Insertion activity (phase)
 - Removal activity (phase)



Basic goal of VV

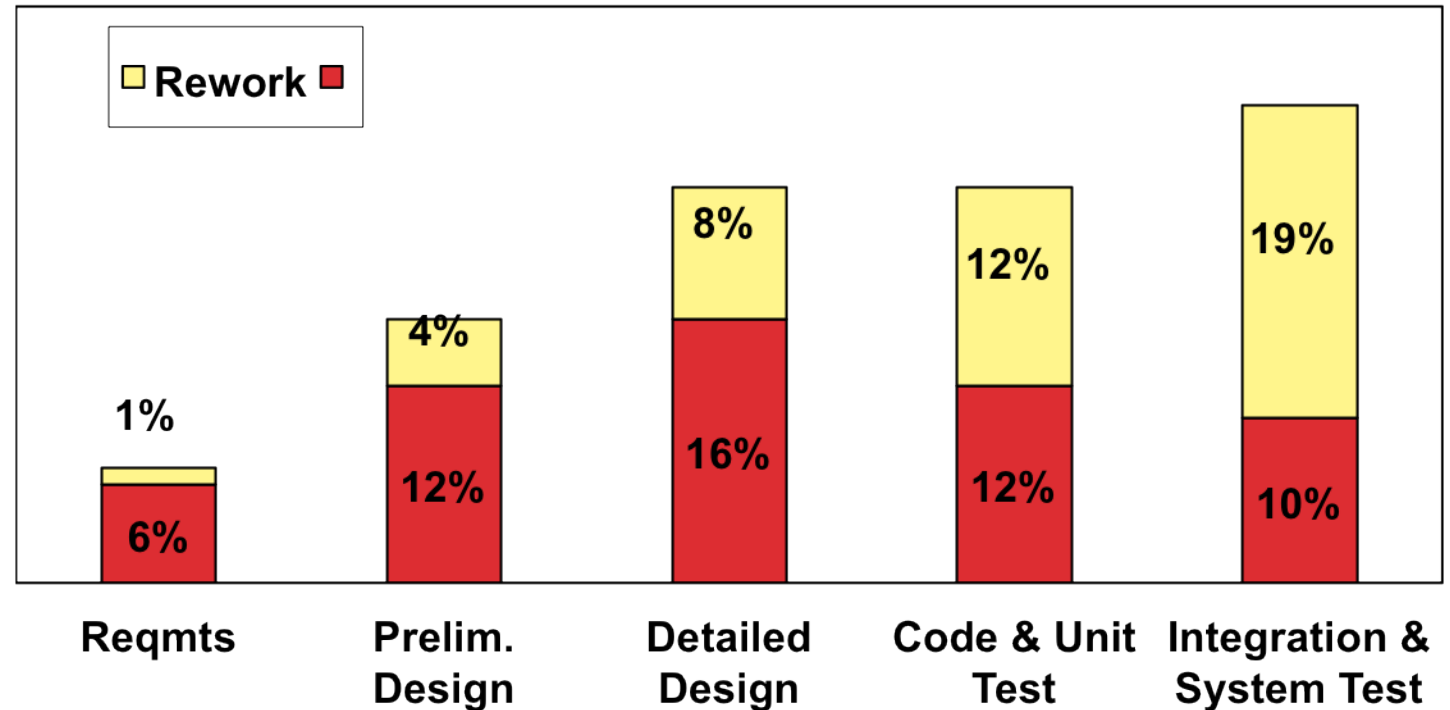
- Minimize number of defects inserted
 - Cannot be zero due to inherent complexity of software
- Maximize number of defects discovered and removed
- Minimize time span between insertion and discover and removal

Insertion/removal by phase – typical scenario

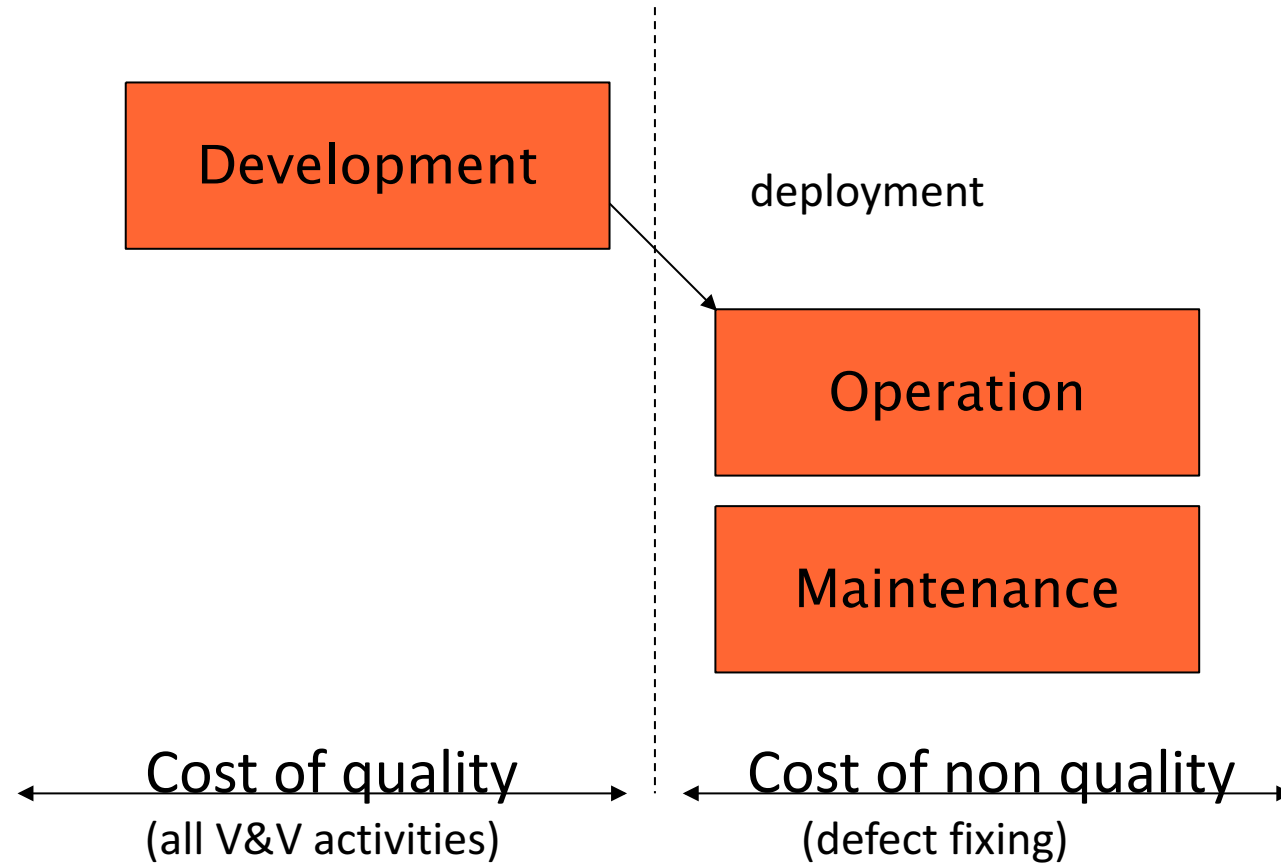


Rework Problem

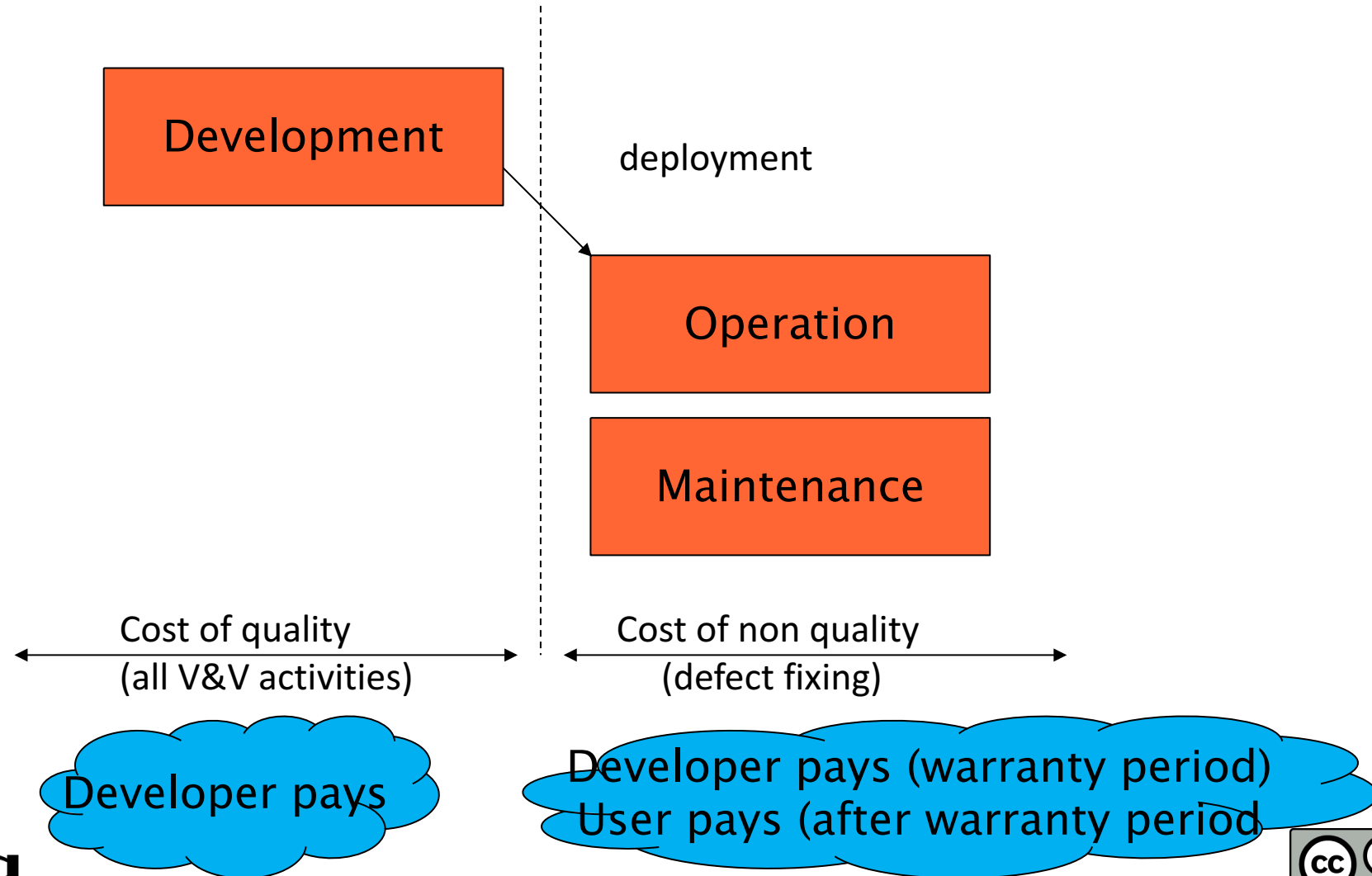
- The longer the delay insert-remove, the higher the cost of removing defect
- Avoidable rework accounts for 40-50% of development [Boehm, 1987; Boehm&Basili, 2001]
 - More recent data available at www.cebase.org



Cost of quality vs cost of non quality



Who pays?



Who pays

- Non quality can be a profitable bargain for developer if the user is the one who pays for it
 - See also lock in:
 - Cost to switch from one vendor (developer) to another is always high for the user, so user is forced to keep the existing developer
- Deciding who (user or developer) is responsible for a defect is often difficult, typically because of unclear requirements

Technical debt

- Concept to define the trade off quality vs non quality
- Debt: money borrowed today, returned in the future, plus interest
- *Technical debt*: effort not spent today, to be spent tomorrow, plus interest

Technical debt

- Ex:

- Variables with non meaningful names

- Int a, int aa, int aaa

- Fast to write today, but what do they actually represent ? Understanding the program will be much harder later

- Code clones

- Faster to do cut and paste today instead of defining a function. Will require more effort in maintenance later (ex fix defect 3 times, if code was cloned 3 times)

V&V techniques - types

- Static
 - inspections
 - source code analysis
- Dynamic
 - testing

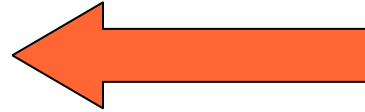
V&V techniques – per activity

- Requirements
 - Inspection of requirement document (static)
 - GUI prototype (static)
 - Product prototype (dynamic)
- Design
 - Inspection of design document (static)
- Implementation
 - Test (unit, integration, application) (dyn)
 - Inspection of source code (static)

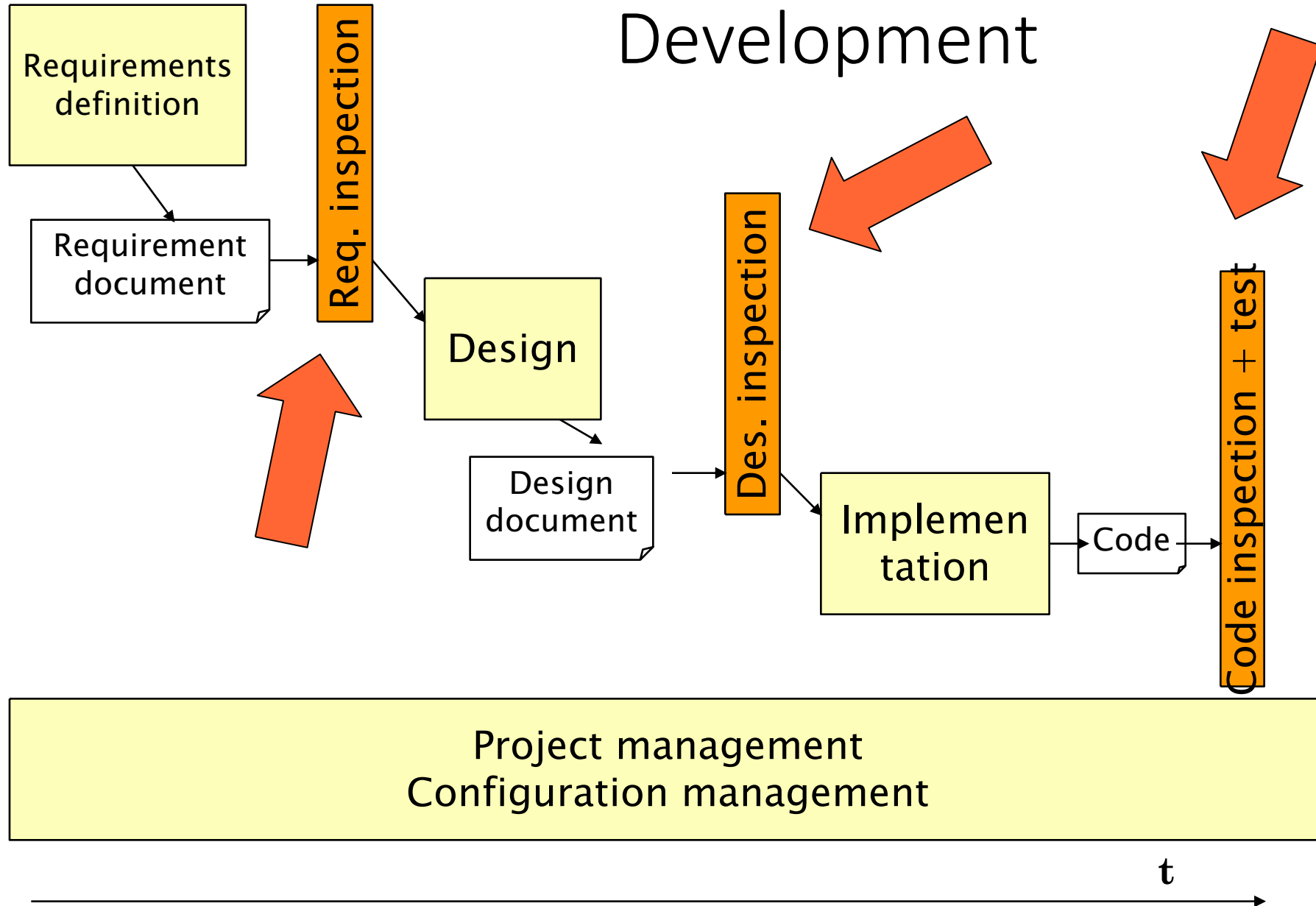
Inspections

Inspections

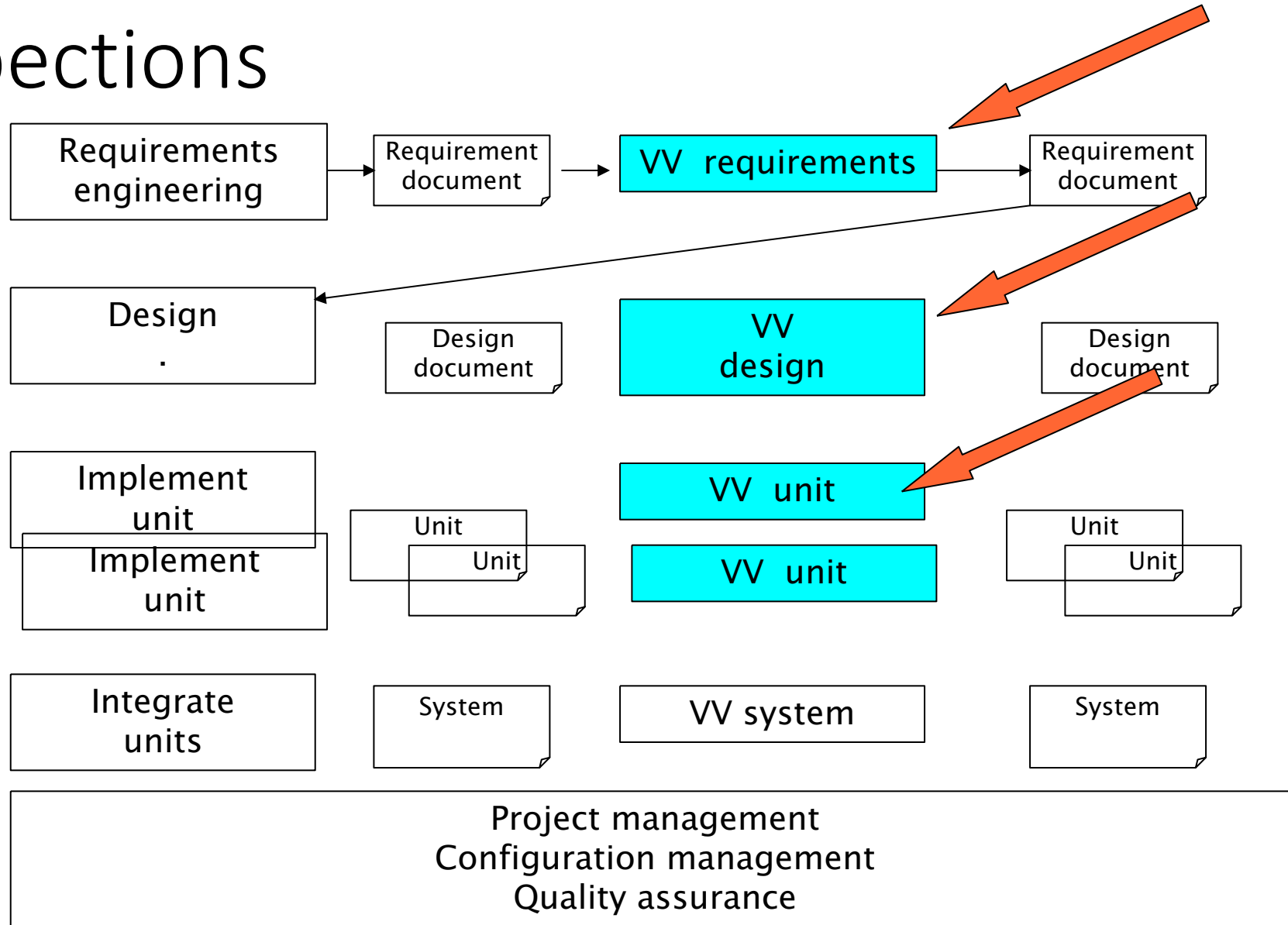
- Static
 - inspections
 - source code analysis
- Dynamic
 - testing



Development



Inspections



Inspection

- Consists in
 - reading documents/code
 - By a group of people (3+, group dynamics)
 - With goal of finding defects (no correction)
- Variants of inspections
 - Reading techniques, walkthroughs, reviews
- Can find many defects
 - Test concentrates one defect at a time

Inspection

- Advantages

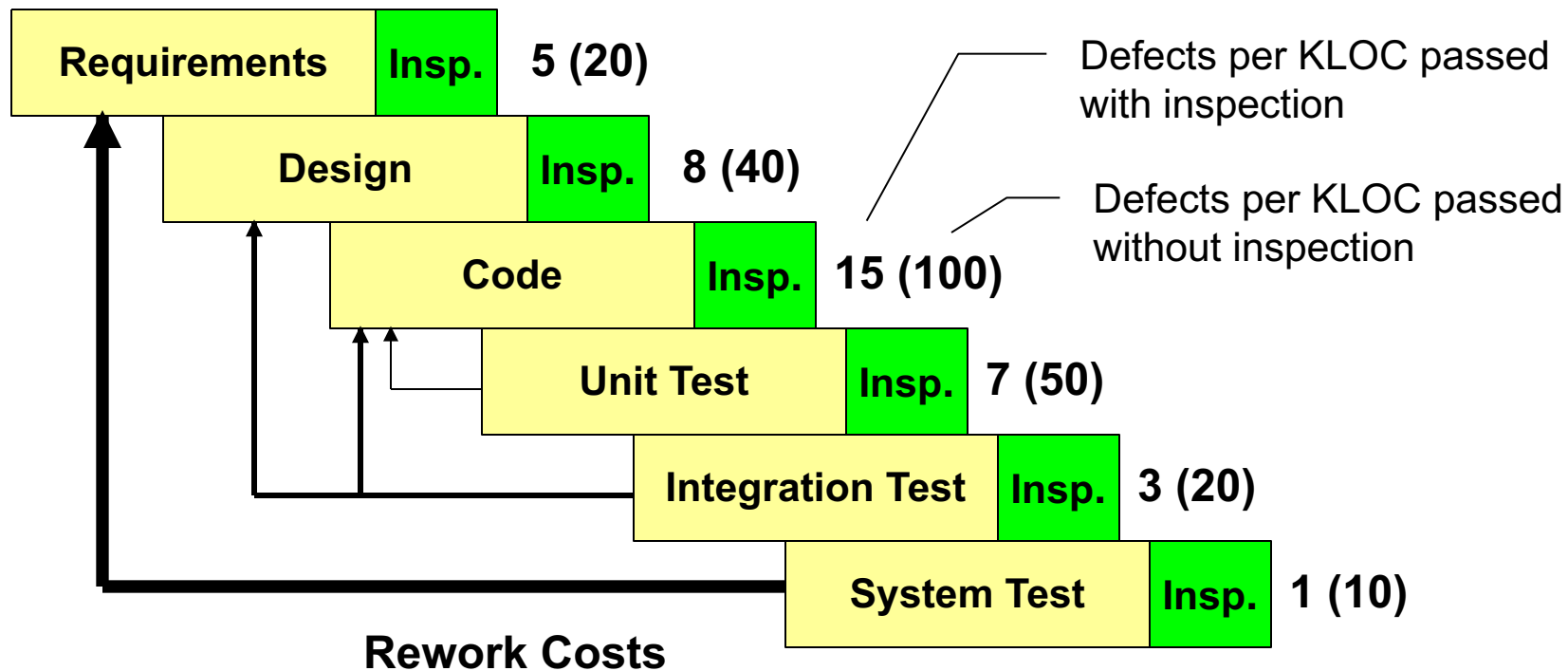
- Can be applied to documents
 - Requirements, design, test cases, ..
- Can be applied to code
 - Does not require execution environment
- Is very effective
 - Reuses experience and know of people on domain and technologies
 - Has more global view (vs test: one defect at a time)
 - Uses group dynamics

- Limits

- More suitable for functional aspects
- Requires time (effort and calendar time)

Benefits

- Early defect detection improves product quality and reduces avoidable rework (down to 10-20%)
 - Data from industry averages [Capers Jones, 1991]
 - more data available at www.cebase.org



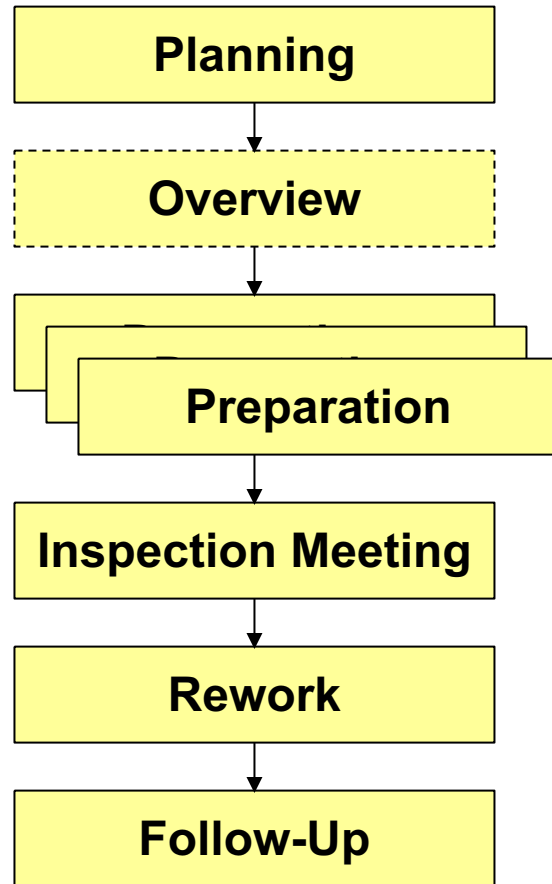
Inspection vs. testing

- Complementary techniques
 - Both should be used in V and V

Roles in group

- Moderator:
 - Leads inspection process and notably inspection meeting
 - Selects participants, prepares material
 - Usually not from project that produces document to be inspected
- Readers:
 - Read document to be inspected
- Author
 - Answers to questions that arise
- Scribe
 - Writes inspection log

Fagan Inspection Process



- Select team, arrange materials, schedule dates
- Present process, product
- *Become familiar with the product*
- Team analysis to find defects
- Correct defects
- Verify fixes, collect data

Inspectors are often unprepared.
How to make visible the quality of preparation?

Process

- Overview
 - Quickly present inspection to group goals and document to be inspected
- Preparation
 - Read individually (applying inspection technique)
- Meeting
 - Group reads, discusses issues, agrees on problems. Scribe logs problems. Moderator keeps focus, keeps pace, stops (long) discussions
- Rework
 - Author fixes defects/problems
- Follow up
 - Repeat inspection or close and pass to next phase

Prerequisites for successful inspections

- Commitment from management
 - Effort invested upfront, that “does not produce anything”
- Find defects, not fix them
- Document under inspection meets quality standards
- Results not used to evaluate people (and notably author)
- Constructive approach
 - Group aims to produce best possible document
 - No “kill the author” game
 - No “relax and chat” meetings

Techniques vs. document

- Ad hoc (code, requirements, design)
 - Just read it
- Defect taxonomy (code, requirements, design)
 - Categories of common defects
- Checklist (code, requirements, design)
 - Questions/controls to be applied
- Code
 - Author or reader 'executes' code
 - Reader reconstructs goal of code from code
 - Reader defines and applies some test cases
- Requirements
 - Scenario based reading
 - Defect based
 - Perspective based
- Design
 - Traceability matrix
 - Scenario execution

Defect Taxonomies for Requirements

- One level
[Basili et al., 1996]
 - Omission
 - Incorrect Fact
 - Inconsistency
 - Ambiguity
 - Extraneous Information
- Two levels
[Porter et al., 1995]
 - Omission
 - Missing Functionality
 - Missing Performance
 - Missing Environment
 - Missing Interface
 - Commission
 - Ambiguous Information
 - Inconsistent Information
 - Incorrect or Extra Functionality
 - Wrong Section

Checklists for Requirements

- Based on past defect information
- Questions refine a defect taxonomy

[Ackerman et al., 1989]

- Completeness
 - 1. Are all sources of input identified?
 - ...
 - 12. For each type of run, is an output value specified for each input value?
 - ...
- Ambiguity
 - 18. Are all special terms clearly defined?
 - ...
- Consistency
 - ...

Checklists for code

- Depends on programming language
- Depends on previous results of inspections

Inspection checks

Fault class	Inspection check
Data faults	Are all program variables initialised before their values are used? Have all constants been named? Should the lower bound of arrays be 0, 1, or something else? Should the upper bound of arrays be equal to the size of the array or Size -1? If character strings are used, is a delimiter explicitly assigned?
Control faults	For each conditional statement, is the condition correct? Is each loop certain to terminate? Are compound statements correctly bracketed? In case statements, are all possible cases accounted for?
Input/output faults	Are all input variables used? Are all output variables assigned a value before they are output?
Interface faults	Do all function and procedure calls have the correct number of parameters? Do formal and actual parameter types match? Are the parameters in the right order? If components access shared memory, do they have the same model of the shared memory structure?
Storage management faults	If a linked structure is modified, have all links been correctly reassigned? If dynamic storage is used, has space been allocated correctly? Is space explicitly de-allocated after it is no longer required?
Exception management faults	Have all possible error conditions been taken into account?

Rates (code inspections)

- 500 LOC/hour (overview)
- 125 LOC/hour (preparation)
- 90-125 LOC/hour (meeting)
 - Ex. 500 LOCs, 4 people, 40 person hours
 - Overview 1hr X 4 = 4 person hours
 - Preparation 4hr X 4 = 16 person hours
 - Meeting 5hr X 4 = 20 person hours

Scenario based reading

- Ask inspectors to create an appropriate abstraction
 - Help to understand the product
- Ask inspectors to answer a series of questions tailored to the abstraction
Inspectors follow different scenarios each focusing on specific issues

Defect-Based Reading

[Porter et al., 1995]

- A scenario-based reading technique to detect defects in requirements expressed in a formal notation (SCR)
- Each scenario focuses on a specific class of defects
 - data type inconsistencies
 - incorrect functionality
 - ambiguity/missing functionality
- Excerpt from incorrect functionality scenario
 - 1. *For each functional requirement identify all input/output data objects:*
 - *questions ...*
 - 2. *For each functional requirement identify all specified system events:*
 - *(a) Is the specification of these events consistent with their intended interpretation?*
 - 3. *Develop an invariant for each system mode:*
 - *questions ...*

Perspective-Based Reading

[Basili et al., 1996]

- A scenario-based reading technique to detect defects in requirements expressed in natural language
 - extended later for design and source code
- Each scenario focuses on reviewing the document from the point of view of a specific stakeholder
 - User (abstraction required: user tasks descriptions)
 - Designer (abstraction required: design)
 - Tester (abstraction required: test suite)

For each requirement/functional specification, generate a test or set of tests that allow you to ensure that an implementation of the system satisfies the requirement/functional specification. Use your standard test approach and technique, and incorporate test criteria in the test suite. In doing so, ask yourself the following questions for each test:

questions ...

Testing

Testing

- Static
 - inspections
 - source code analysis
- Dynamic
 - testing



Testing

- Dynamic technique, requires execution of executable system or executable unit
 - system test
 - unit test

Testing

- The process of operating a system or component under specified conditions observing or recording the results to detect the differences between actual and required behavior (= failures)

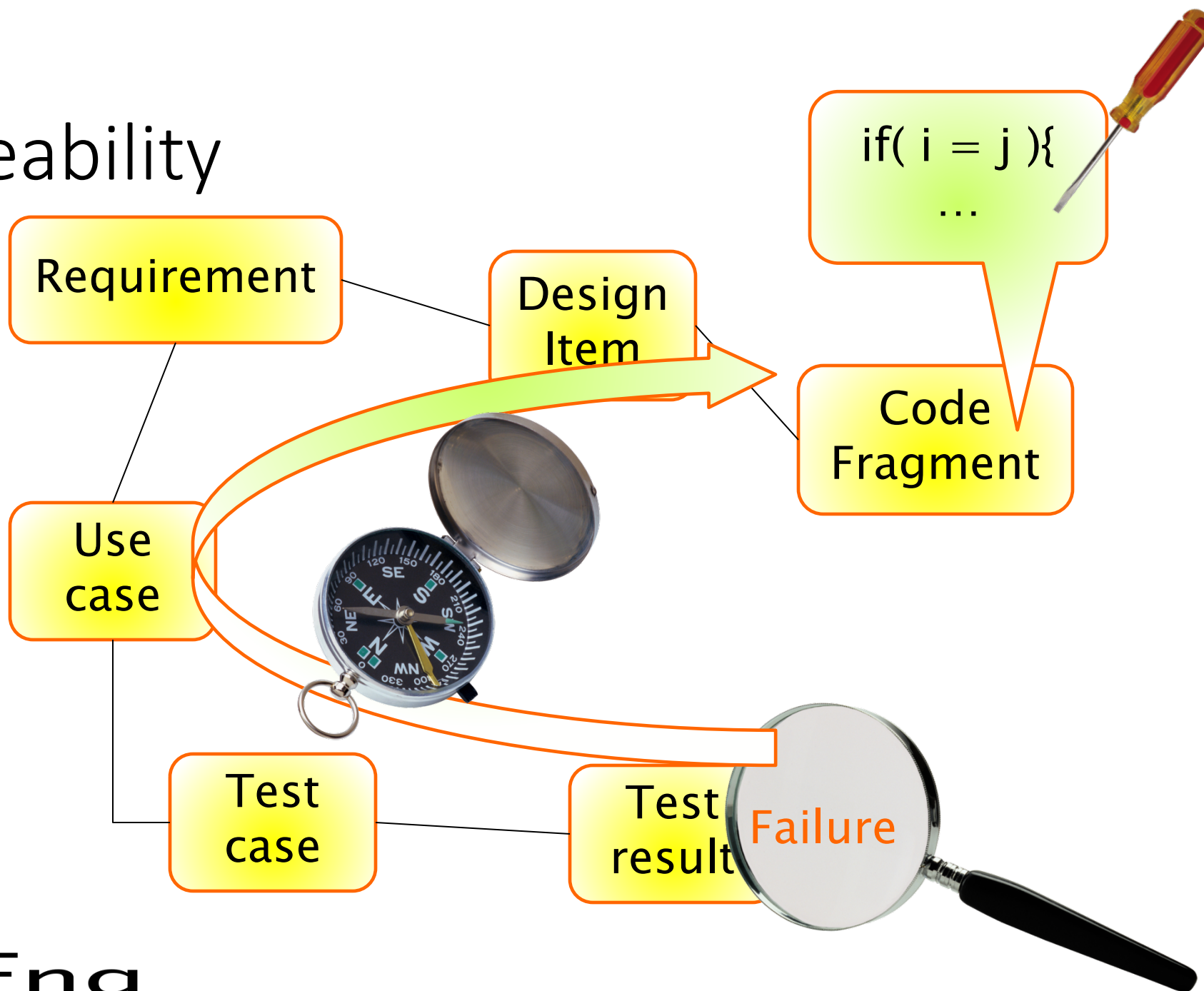
Purpose of test

- The purpose of testing process is to find defects in the software products
 - A test is successful if it reveals a defect

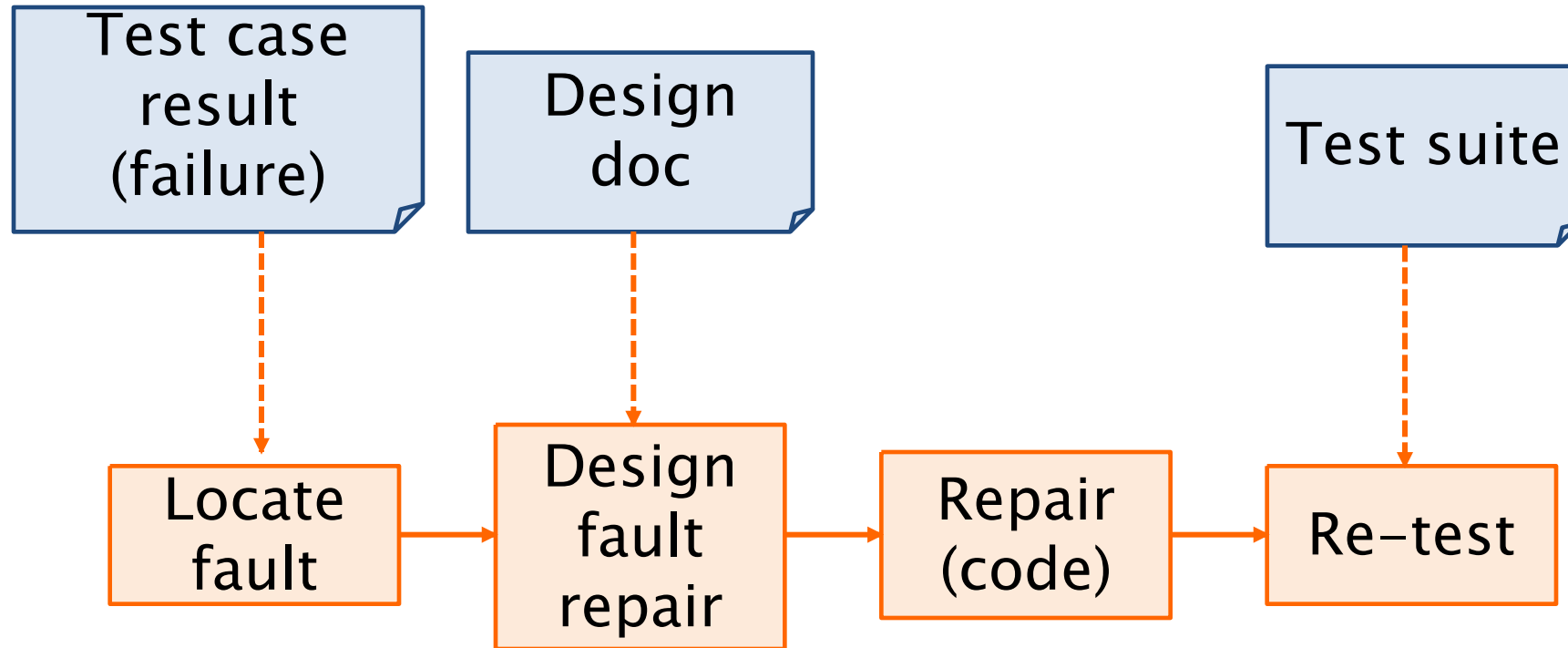
Testing vs. debugging

- Defect testing and debugging are different activities
 - May be performed by different roles in different times
- Testing tries to find failures
- Debugging searches for and removes the corresponding fault(s)

Traceability



Debugging



Test case

- Certain stimulus applied to executable (system or unit), composed of
 - name
 - input (or sequence of)
 - expected output
- With defined constraints/context
 - ex. version and type of OS, DBMS, GUI ..

Test suite

- Set of (related) test cases

Test case log

- Test case ref. +
 - Time and date of application
 - Actual output
 - Result (pass / no pass)

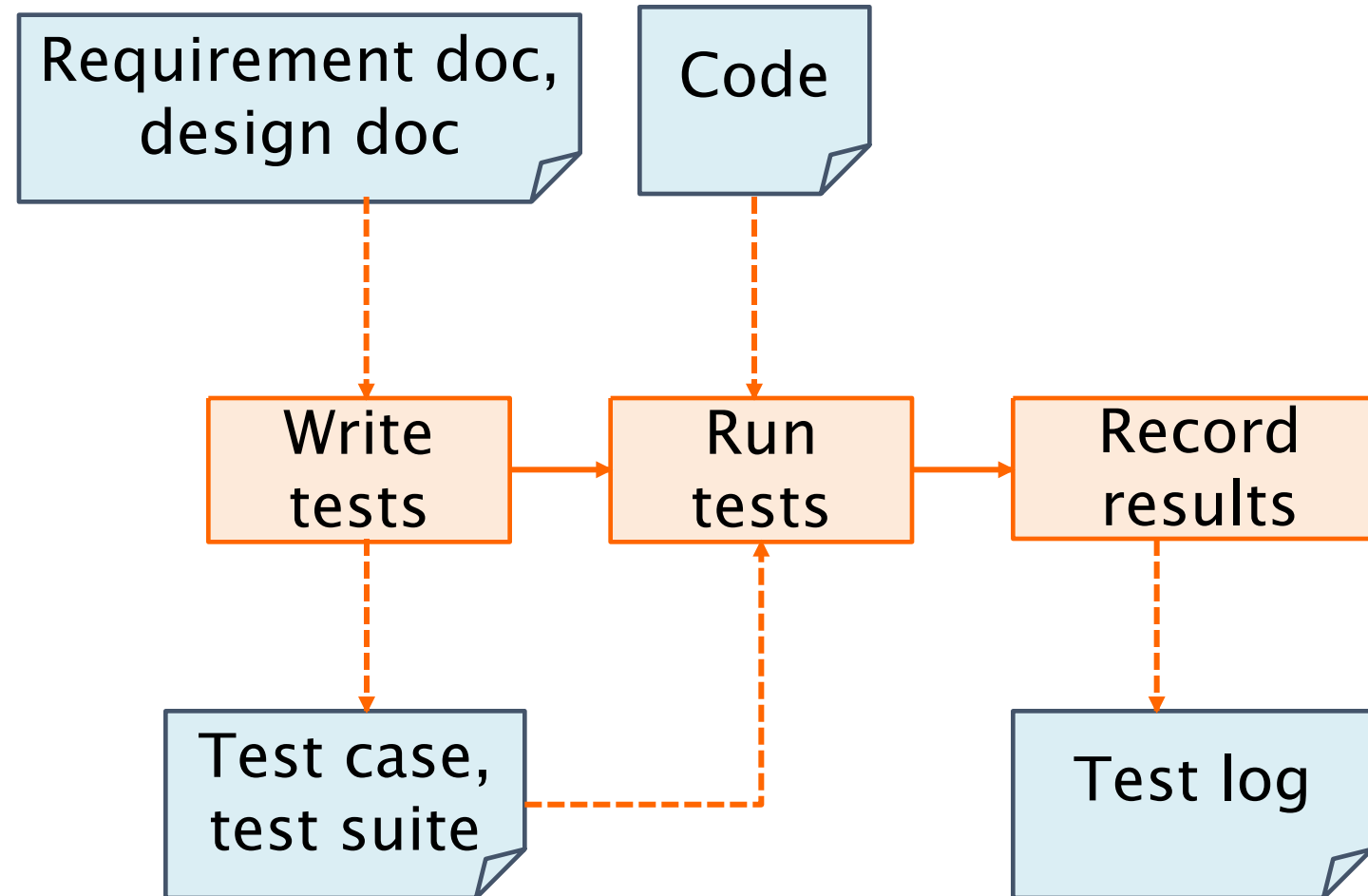
Ex.

- Function **add(int x, int y) : number**
- Test case:
 - T1(1,1; 2)
 - T2(3,5; 8)
- Test suite
 - TS1{T1, T2}
- Test log
 - T1, 16-3-2013 9:31, result 2, success
 - T2, 16-3-2013 9:32, result 9, fail

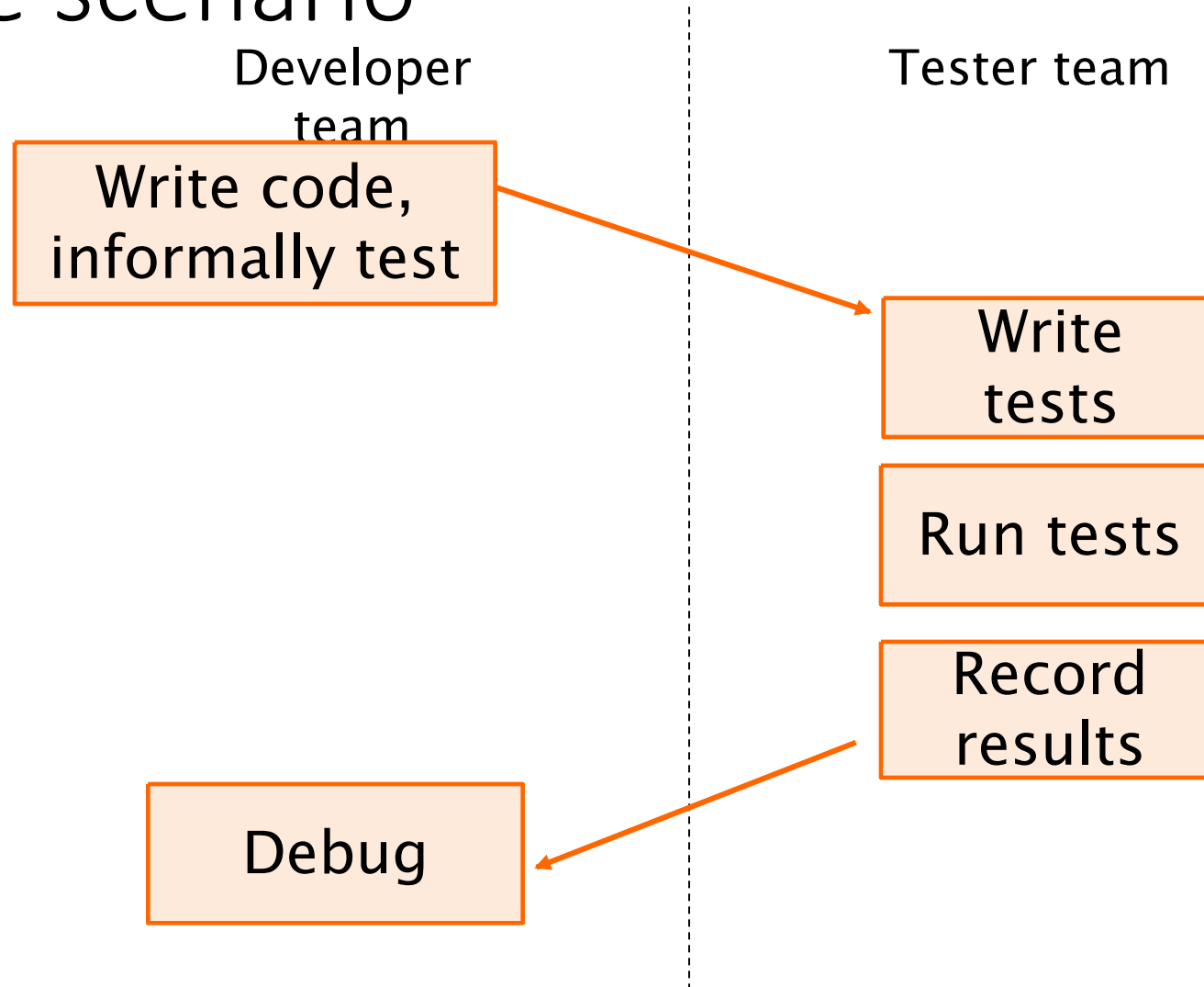
Test activities

- Write test cases
 - Test case, test suite
- Run test case (test suite)
- Record results
 - Test case log

Test activities



Possible scenario



Scenario 2

Developer team

Write code,
informally test

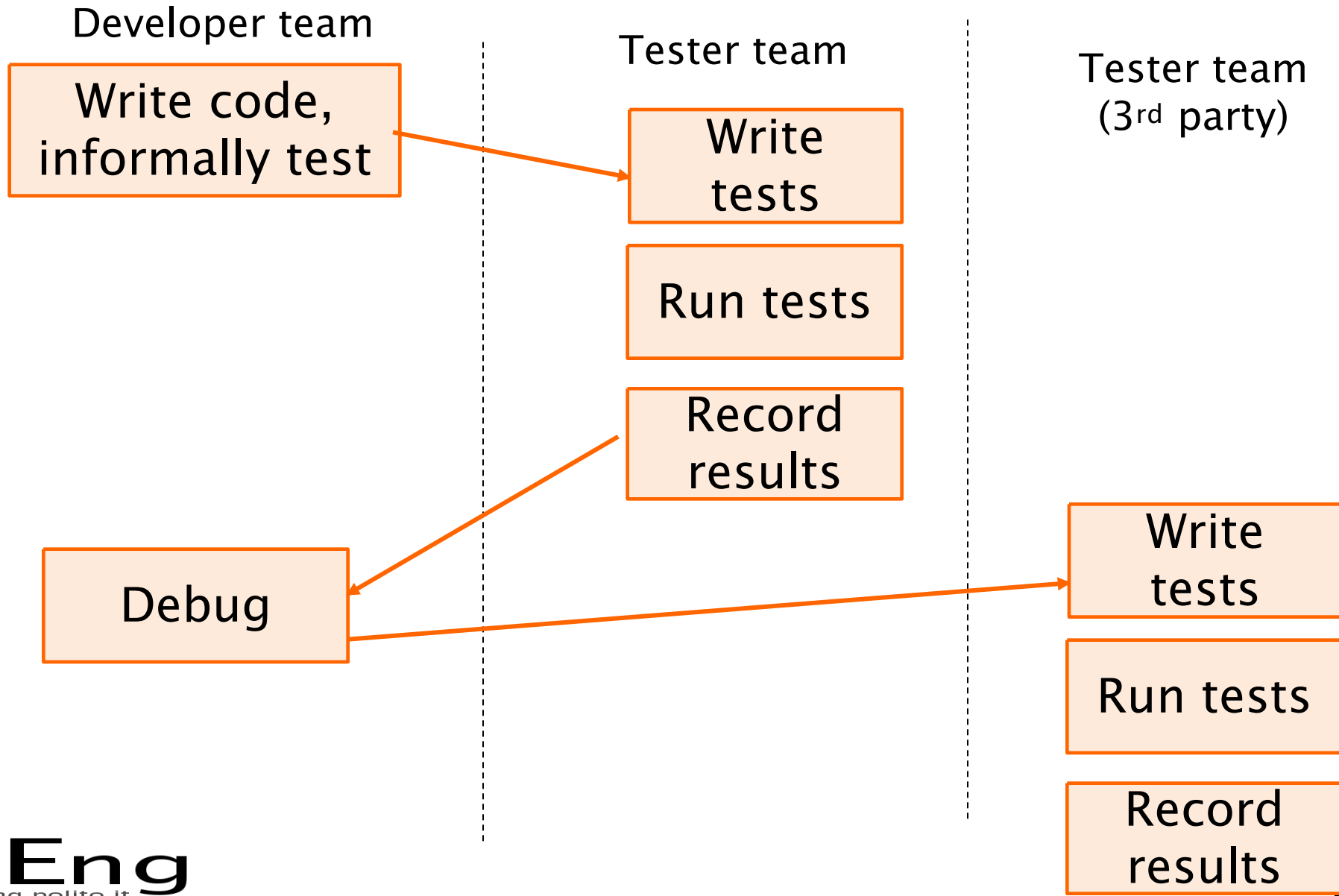
Write
tests

Run tests

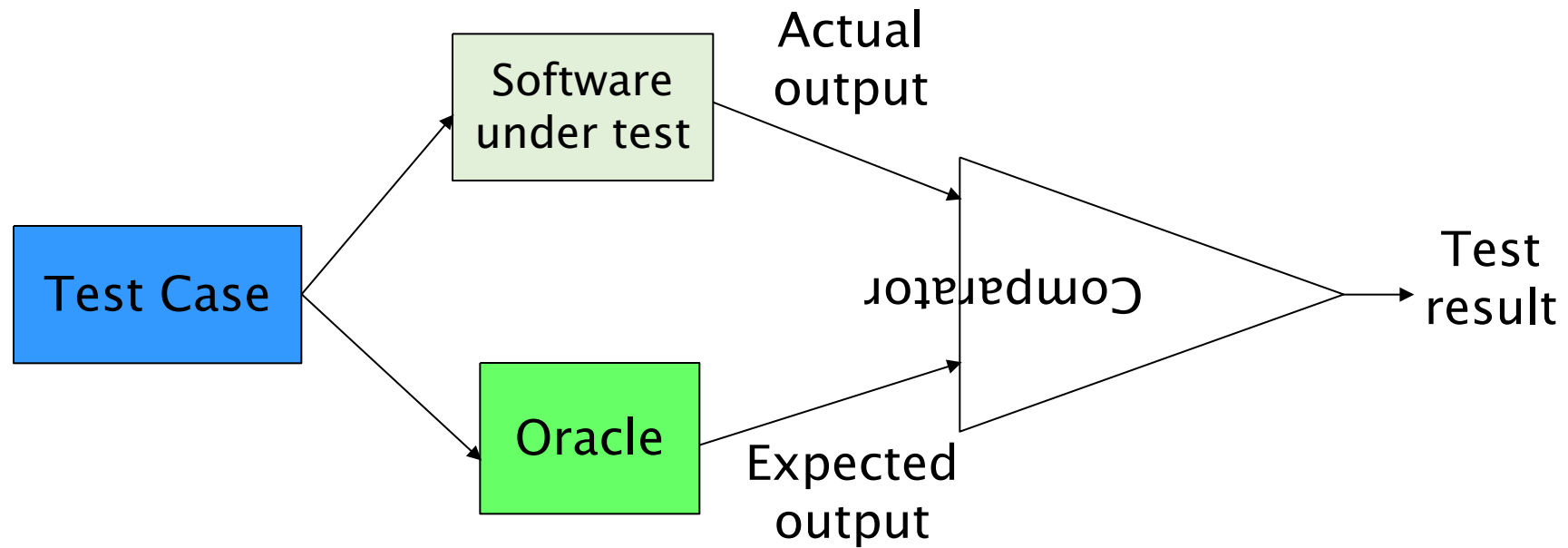
Record
results

Debug

Scenario 3



Oracle



Oracle

- The ideal condition would be to have an automatic oracle and an automatic comparator
 - The former is very difficult to have
 - The latter is not always available (ex GUI)
- A human oracle is subject to errors
- The oracle is based on the program specifications (which can be wrong)

Oracle

- Necessary condition to perform testing:
 - Know the expected behavior of a program for a given test case (oracle)
- Human oracle
 - Based on req. specification or judgment
- Automatic oracle
 - Generated from (formal) req. specification
 - Same software developed by other parties
 - Previous version of the program (**regression**)

Theory and constraints

Correctness

- Correct output for all possible inputs
- Requires exhaustive testing

Digital vs analog

Analog device

- Continuity hypothesis
- Ex bridge.
If bridge can sustain 1 ton weight (test case) and 2 tons weight (another test case), the bridge will sustain all weights between 1 and 2 tons

Digital program

- Digital, no continuity hypothesis
- Ex function `add(x, y);`
if `add` works correctly for `add(1,2)` (test case)
and `add(1,10)` (test case)
there is no warranty that it will work for 1,3 1,4 1,5 etc

Pentium case – 1994

- Error in division function in the FPU
(missing entries in a lookup table)
- 1 failure out of 9 billion possible cases
- Cost 474 million USD to replace the processors

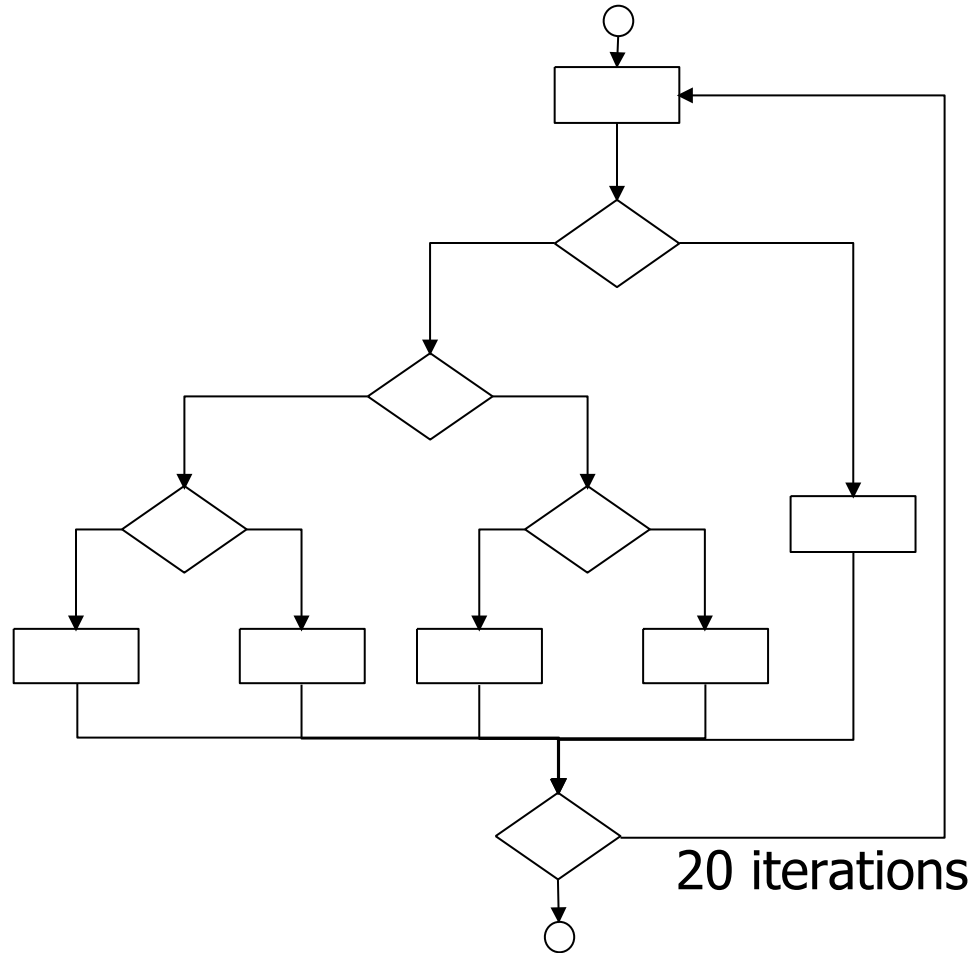
Exhaustive testing

- Write and run all possible test cases
(== execute all possible inputs)

Exhaustive test - example

- function: $Y = A + B$
- A and B integers, 32 bit
- Total number of test cases : $2^{32} * 2^{32} = 2^{64} \approx 10^{20}$
- 1 ms to run a test case $\Rightarrow$ 3 billion years
 - (and the time for writing test cases is not considered)

Exhaustive test



- 5 possible paths per iteration
- 20 iterations
- Overall: $5^{20} \approx 10^{14}$ possible paths
- 1 ms/test $\Rightarrow$ 3170 years

Exhaustive test

- Exhaustive test is not possible
- So, goal of test is finding defects, not demonstrating that system is correct (that would require exhaustive testing)
- Goal of test (and VV in general) is assuring a *good enough* level of confidence

Dijkstra thesis

- *Testing can only reveal the presence of errors, never their absence*

E. W. Dijkstra. Notes on Structured Programming.

In *Structured Programming*, O.-J. Dahl, E. W. Dijkstra, and C. A. R. Hoare, Eds. Academic, New York, 1972, pp. 1–81.

How to select test cases?

- Exhaustive testing impossible (except in trivial cases)
- Key point is in how to select test cases
- Criterion to select test cases is evaluated by
 - Reliability
 - Validity

Basic concepts

- D : program domain (input)
- $d \in D$, $P(d)$ is the program output
- $OK(d) \Leftrightarrow P(d)$ corresponds to oracle
- Test: $T \subseteq D$
- $SUCC(T) \Leftrightarrow \forall t \in T, OK(t)$

J. B. Goodenough and S. L. Gerhart. Toward a Theory of Test Data Selection. *IEEE Transactions on Software Engineering*, June 1975, pp. 26–37.

Criteria

- Tests can be selected by means of criteria
- C : selection criterion for tests
- $\text{COMPLETE}(T,C)$: T is selected by C

Validity

- $\exists d \in D, \neg \text{OK}(d) \Rightarrow (\exists T \subseteq D) \text{ COMPLETE}(C,T) \wedge \neg \text{SUCC}(P,T))$
- *Valid Criterion:*
 - C is *valid* if and only if whenever P is incorrect C selects at least one test set T which is not successful for P .

Reliability

- $\forall T1, \forall T2 \subseteq D,$
 $\text{COMPLETE}(C, T1) \wedge \text{COMPLETE}(C, T2) \Rightarrow \text{SUCC}(T1) \Leftrightarrow \text{SUCC}(T2)$
- *Reliable Criterion:*
 - *C* is *reliable* if and only if either every test selected by *C* is successful or no test selected is successful.

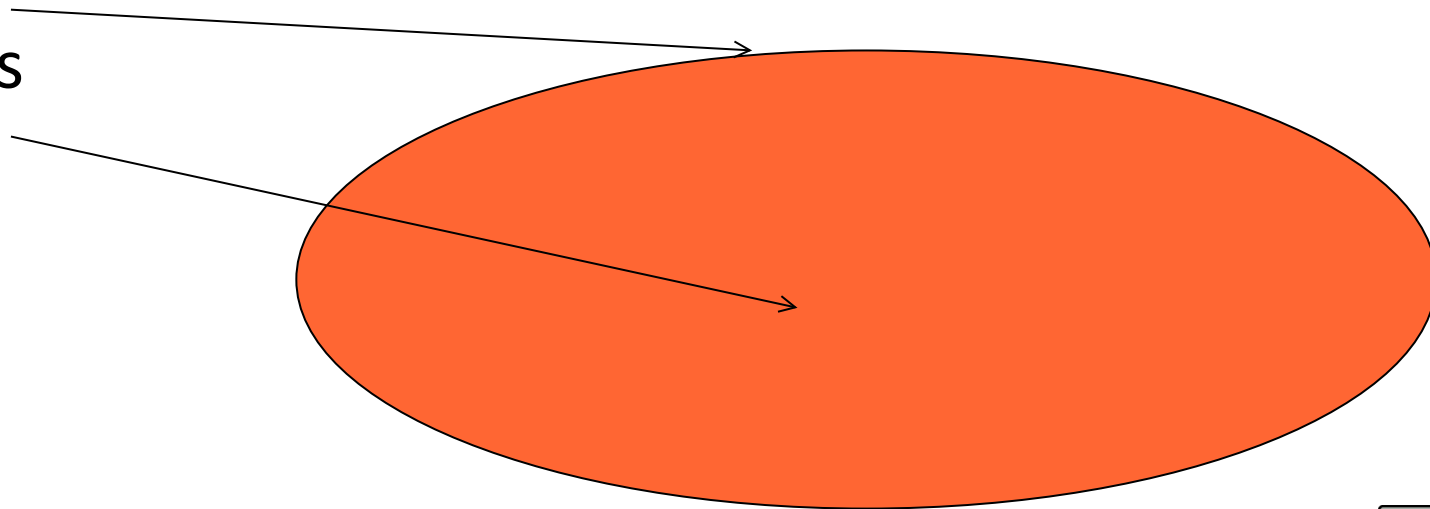
Ex: Pentium case

- Reliable: all test cases succeed in I-F, all test cases fail in F
- Valid: selects both F and I

I: input space

$F \subseteq I$: input that fails

I-F : input that does not fail



Fundamental theory

- Theorem

$$(\exists T \subseteq D)(\text{COMPLETE}(T, C) \wedge \text{RELIABLE}(C) \wedge \text{VALID}(C) \wedge \text{SUCC}(T)) \Rightarrow (\forall d \in D) \text{OK}(d)$$

- The success of a test T selected by a reliable and valid criterion implies the correctness of T

Uniformity

- Criterion uniformity focuses on program specification
 - Not only program
- A criterion C is uniformly valid and uniformly reliable if and only if C selects only the single test set $T = D$

E. J. Weyuker and T. J. Ostrand. Theories of Program Testing and the Application of Revealing Subdomains. *IEEE Transactions on Software Engineering*, May 1980, pp. 236–246.

Howden theorem

- For an arbitrary program P it is impossible to find an algorithm that is able to generate a finite ideal test (that is selected by a valid and reliable criterion)

W.Howden. Reliability of the Path Analysis Testing Strategy. IEEE Transactions of Software Engineering, 2(3), September 1976, pp. 208-215

Brainerd Landweber

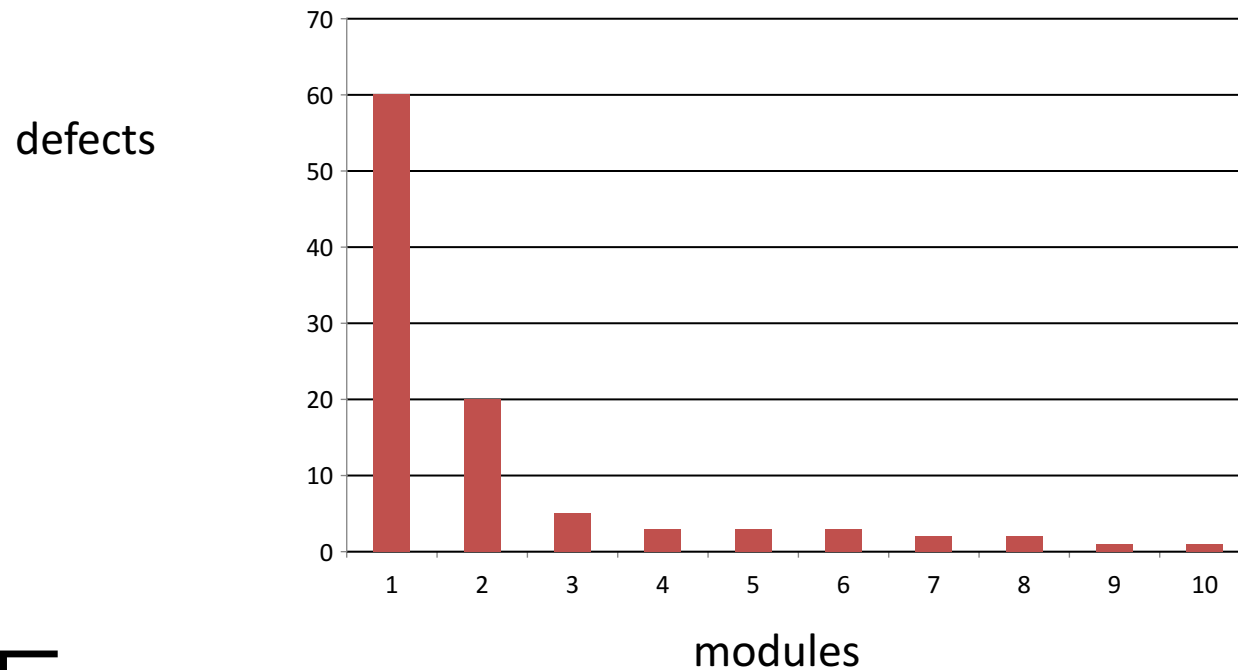
- Given two programs the problem of deciding whether they compute the same function is undecidable
- Therefore even if we have access to the archetype program we cannot demonstrate the equivalence of a new program

Weinberg's law

- A developer is unsuitable to test his/her own code
 - A developer is not emotionally willing to find defects
 - If a developer misunderstands a problem, he cannot find such error
- Testing should be performed by
 - A separate QA team
 - Peers

Pareto-Zipf law

- Approximately 80% of defects come from 20% of modules
- It is better to concentrate on the faulty modules



Summary

- From correctness point of view
 - Use criteria to define test cases that are
 - Reliable
 - Valid
- From psychological point of view
 - Use a policeman mindset
 - Any part of a software (including tests) is guilty in principle (may contain defects)
- From risk management point of view
 - Test in function of the risk
 - Safety critical software
 - Mission critical software

Test classification

- Per item under test
 - Unit, integration, system
 - Regression
- Per approach
 - Black box (functional)
 - White box (structural)
 - Reliability assessment/prediction
 - Risk based (safety security)
- Per formality
 - Exploratory / informal
 - formal

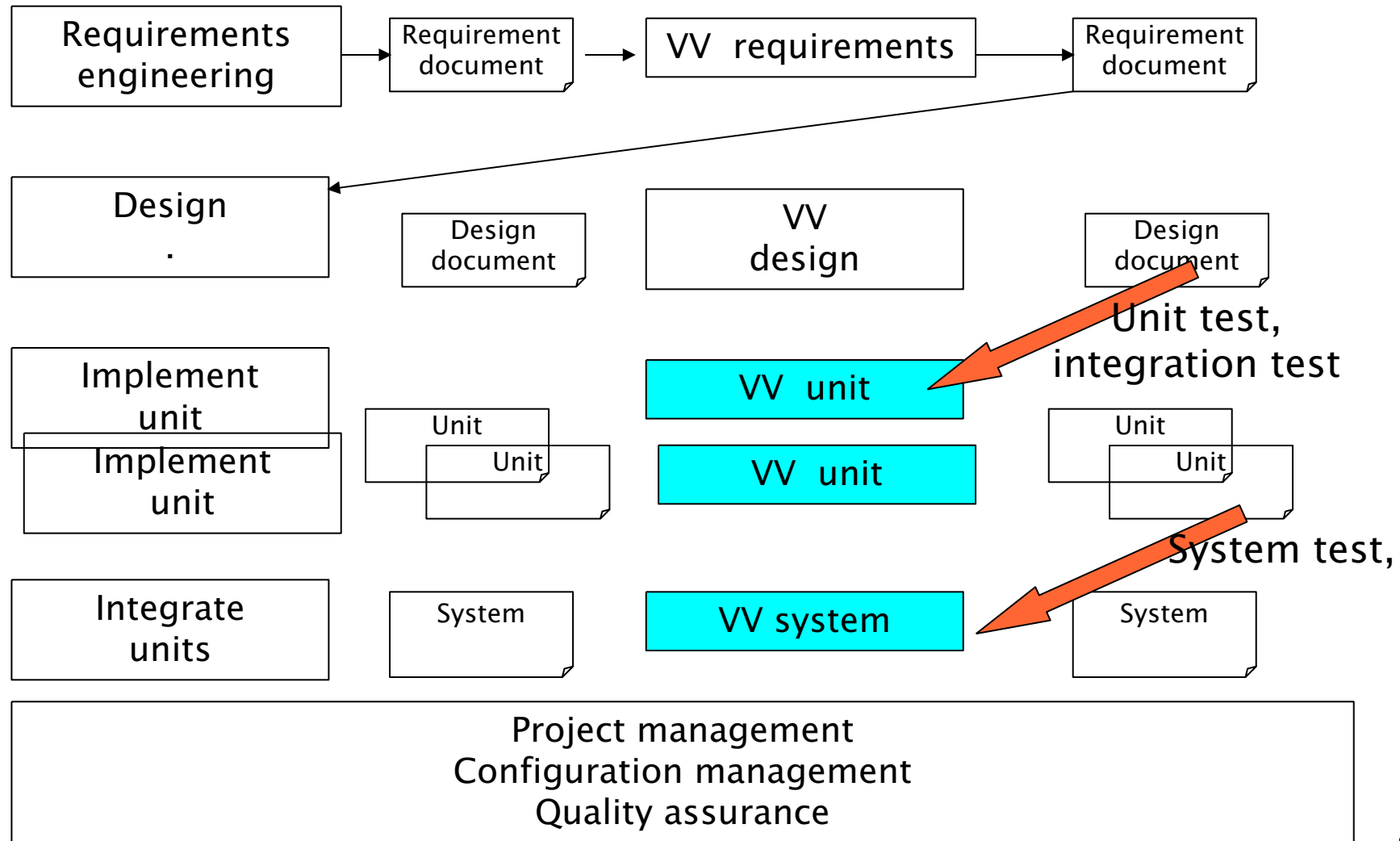
Test per item under test

- Unit tests
 - Individual modules
- Integration tests
 - Some modules working together (partial system)
- System tests
 - All modules together (complete system)
 - API level
 - GUI level

Test per formality

- Informal testing / exploratory testing
 - Tests are not formalized (just applied, test case and test result are lost)
 - Technique used to generate the test is not explicit
- (Formal) Testing
 - Tests are formalized
 - A technique is used (and documented) to generate test cases

Test per item



Test per approach

- Given an item to test, approach can be
 - Black box / functional (Requirements driven)
 - Are the requirements of the object satisfied?
 - White box (Structure driven)
 - Is the object built as it should?
 - Reliability / statistic
 - Does it satisfy the customer need? (use most common operational scenarios)
 - Risk based
 - Is it vulnerable to most likely risks?

Testing classification (2)

	Item		
	Unit	Integration	System
Functional / black box	X	X	X
Structural / white box	X		
Reliability			X
Risks			X

Test classification and coverage

Approach	Item tested		
	Unit	Integration	System
Requirements-driven	100% unit requirements	100% product requirements	100% system requirements
Structure-driven	85% logic paths	100% modules	100% components
Statistics-driven			90-100% of usage profiles
Risk-driven	As required	As required	100% if required

Coverage

- Entities considered by at least one test case / total entities
- 'Entity' depends on type of test, can be:
 - Test cases
 - Requirement
 - Function
 - Structural element
 - Statement, decision, module

Coverage

Item tested	Coverage	
Unit (class)	methods	Consider methods of the class >= one test case per method
	Black box, partitions	Consider a method Define partitions >= One test case per partition
	Black box, boundary	Define partitions >= One test case per boundary
	Black box, random	Generate randomly test cases

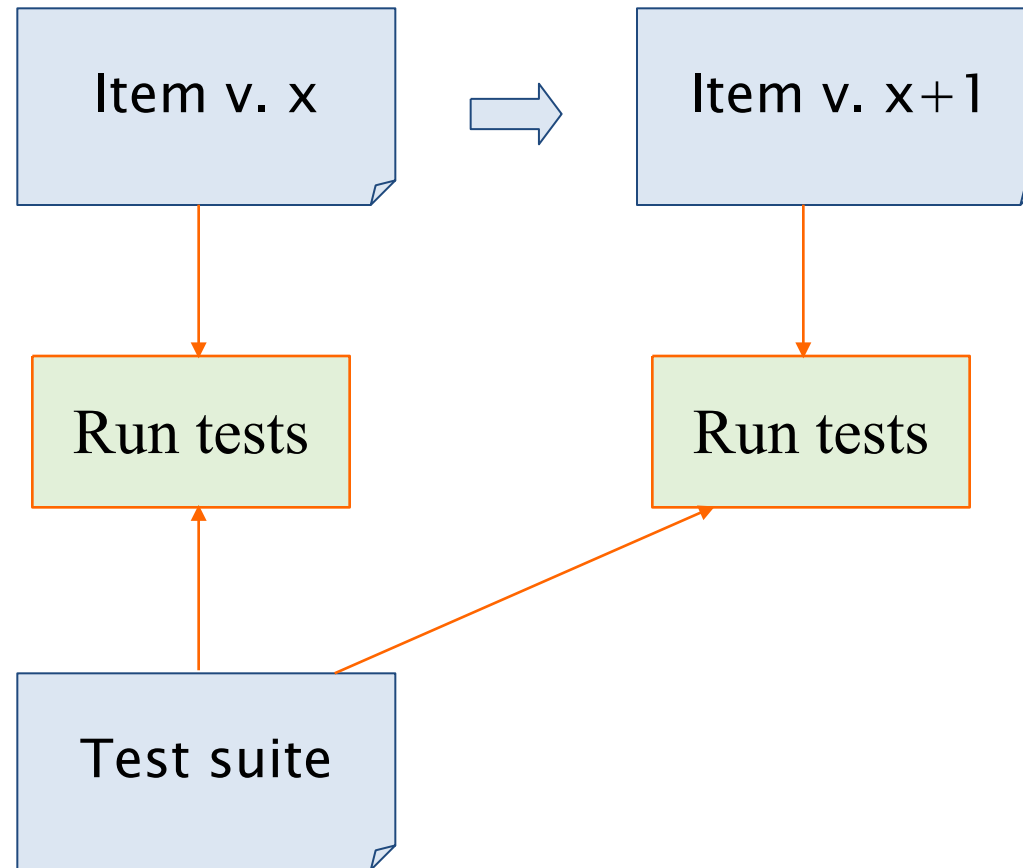
Coverage

Object tested	Coverage	
Integration (some classes)	dependencies	At least one test case per dependency

Coverage

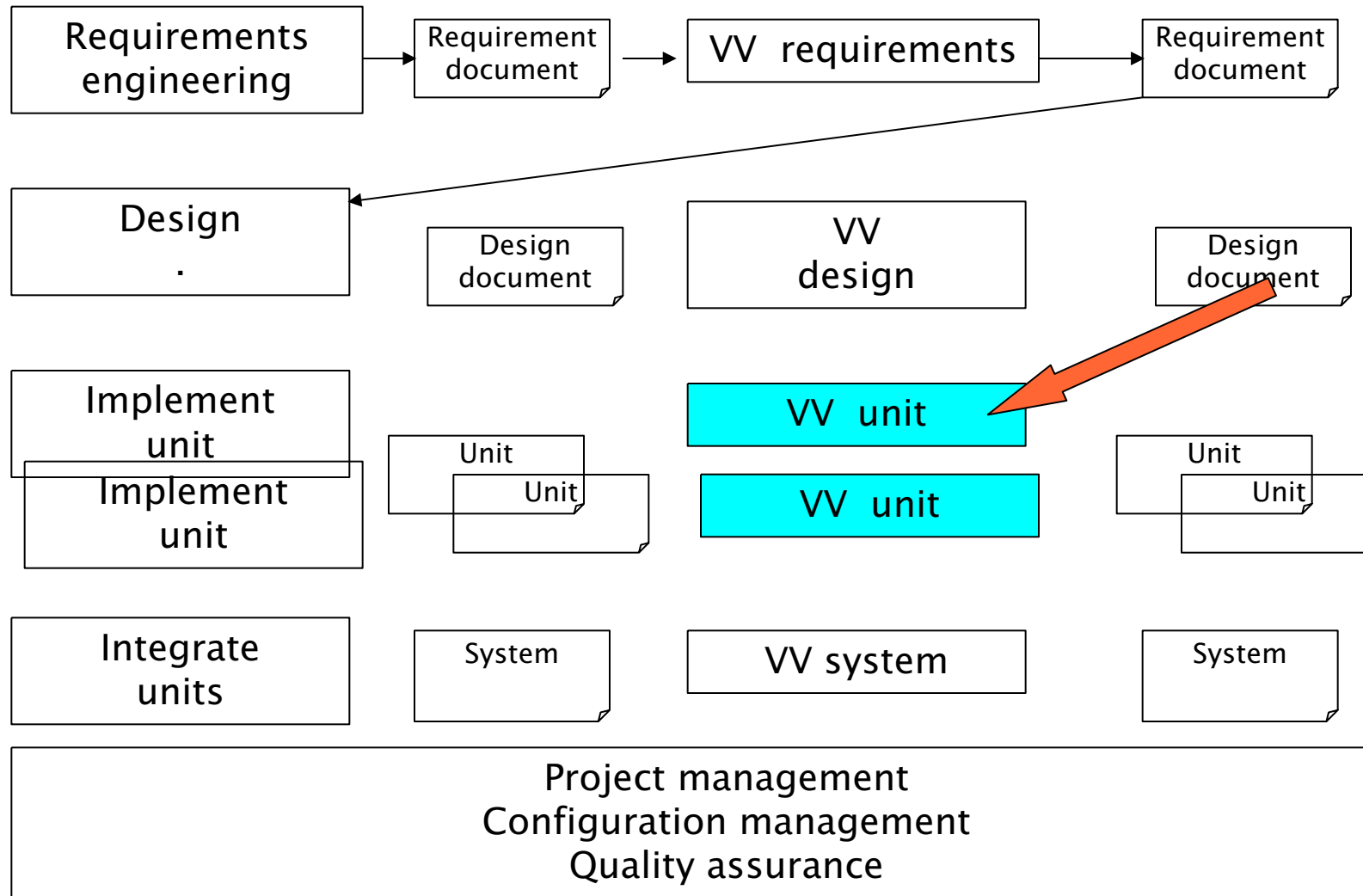
Object tested	Coverage	
System (all classes)	Functional requirements	At least one test case per requirement (may correspond to testing methods of 'main' class at end of integration testing)
	Scenarios	At least one test case per scenario
	Non functional requirements	At least one test case per non functional requirement (efficiency)

Regression testing



Unit test

Unit Test



Unit test

- Test of one independent unit
 - Unit:
 - function (procedural languages)
 - class and its methods (OO languages)

Unit test

- Black box (functional)
 - Random
 - Equivalence classes partitioning
 - Boundary conditions
- White Box (structural)
 - Coverage of structural elements

Unit test – black box

Random

- Function **function squareRoot(x: number): number**
- Extract randomly x
 - T1 (3.0 ; $\sqrt{3}$)
 - T2 (1000.8 ; $\sqrt{1000.8}$)
 - T3 (-1223.7; error)
- Function **function invert(x: number): number**
- Extract randomly x
 - T1 (1.0 ; 1.0)
 - T2 (-2.0 ; -0.5)

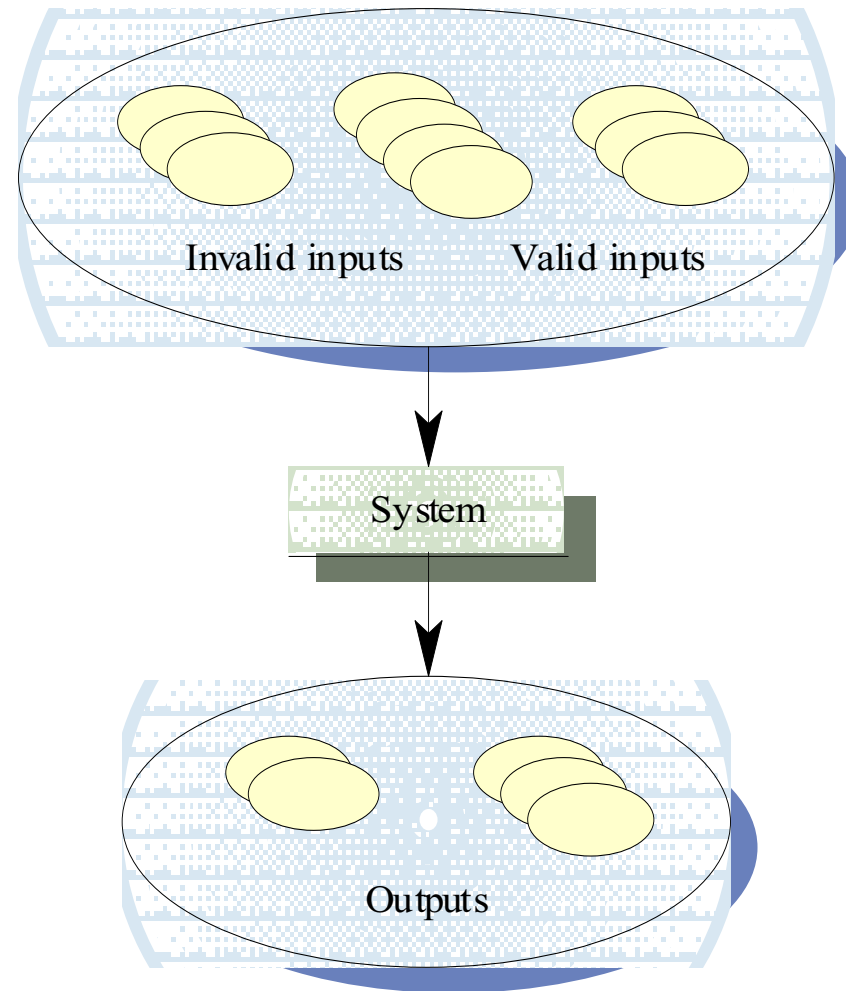
Random

- Pros
 - Independent of requirements
- Cons
 - Requires many test cases (easy to define the inputs, requires Oracle to compute the expected output)

Equivalence classes partitioning

- Divide input space in partitions
 - that have similar behavior from point of view of (requirements for) unit
 - Take one / two test cases per partition
- Boundary conditions
 - Boundary between partitions
 - Take test cases on the boundary

Equivalence classes



Equivalence classes

- A class corresponds to set of valid or invalid inputs for a *predicate* on the input variables
 - If a test in a class has not success the other tests in the same class may have the same behavior
 - Valid: acceptable
 - Invalid: not acceptable (produces exception, error)

Criteria, predicates, partitions

- Criterion: attribute
- Predicate: condition on attribute, defines a partition (== eq class)
- Ex
 - Criterion: age
 - Predicate: age > 100 (valid)
age < 25 (valid)
age < 0 (invalid)
age > 150 (?)

Predicates

- Common predicates:
 - Single value: age = 33
 - Interval: age between 0 and 200
 - Boolean: married = true or false
 - Discrete set: marital status = single, married, divorced

Predicates and eq classes

Predicates	Classes	Example
Single value	Valid value, invalid values < value_x Invalid values > value_x	Age = 33 Age < 33 Age > 33
Interval	Inside interval, Outside one side Outside, other side	Age > 0 and age < 200 Age > 200 Age < 0
Boolean	True false	Married = true Married = false
Discrete set	Each value in set One value outside set	Status = single Status = married Status = divorced Status = jksfhj

Selection of test cases

- Every equivalence class must be covered by one test case at least
 - A test case for each invalid input class
 - Each test case for valid input classes must cover as many (remaining) valid classes as possible

Equivalence classes

function squareRoot (x: number) : number;

- Criterion: sign of x
- Partitions
 - Sign is positive T1 (1 ; $\sqrt{1}$)
 - Sign is negative T2 (-1 ; error)
- Boundary: zero, infinite
 - Zero and close
T3 (0; $\sqrt{0}$) T4(0.01; $\sqrt{0.01}$) T5(-0.01; error)
 - 'Infinite' and close
T6 (maxdouble; $\sqrt{\text{maxdouble}}$) T7 (maxdouble+0.01; err)
T8 (mindouble; $\sqrt{\text{mindouble}}$) T7 (mindouble-0.01; err)

- Previous examples considered only one criterion
- What if many?
- Need to consider all combinations of predicates

Equivalence classes

function convert(s: string) : number;
converts a sequence of chars (max 6) into an integer number.
Negative numbers start with a '-'

Criterion to define the class	Equivalence classes and test cases	
String represents a well formed integer	Yes T1("123"; 123)	No T2("1d3" ; error)
Sign of number	Positive T1("123" ; 123)	Negative T3("-123" ; -123)
Number of characters	≤ 6 T1("123" ; 123)	> 6 T4("1234567"; err)

Equiv. classes- combinatorial

WF integer	sign	N char	
yes	Pos	≤ 6	T1("123"; 123)
		> 6	T4("1234567"; err)
	Neg	≤ 6	T3("-123"; -123)
		> 6	T5("-123456"; err)
no	Pos	≤ 6	T2("1d3"; err)
		> 6	T6("1sed345"; err)
	Neg	≤ 6	T7("-1ed"; err)
		> 6	T8("-1ed234"; err)

Boundary - combinatorial

WF integer	sign	N char	
yes	Pos	≤ 6	"0" "999999"
		> 6	"1 000000" "9999999"
	Neg	≤ 6	"-0" "-999999"
		> 6	"-9999999"
no	Pos	≤ 6	" "
		> 6	" " (7 blanks)
	Neg	≤ 6	"_"
		> 6	"_ "

Equiv classes and state

- When a module has state
 - the state has to be considered to define the partitions
 - State may be difficult to read/create
 - Requires a sequence of calls

Equiv classes and state

- **function ave3(i: number): number;**
- Computes average of last three numbers passed, excluding the negative ones
 - Criteria
 - state: n elements received
 - int i: positive, negative

Equiv classes and state

N elements	i	Test
0	NA	
1	Pos	T1(10; 10)
	Neg	T2(-10; ?)
2	Pos	T3(10,20 ; 15)
	Neg	T4(-10,-20; ?)
3	Pos	T5(10,2,6; 6)
	Neg	T6(-10,-2,-6; ?)
>3	Pos	T7(1,2,3,4; 3)
	Neg	T8(-1,-2,-3,-4; ?)

Test of OO classes

- Have state
- Many functions to be tested
- Identify criteria and equivalence classes
- Apply them to each function

Test of OO classes

```
abstract class AbstractEventsQueue {  
    /**  
     * Cancels all events  
     */  
    abstract reset(): void;  
    /**  
     * Pushes a new event with a specified time tag into the queue. Discards events with negative or zero-time tag and  
     * with time tag already existing.  
     * @param timeTag The time tag of the event to be pushed.  
     * @throws InvalidTag if the time tag is invalid or already exists.  
     */  
    abstract push(timeTag: number): void;  
    /**  
     * Returns and cancels the event with the lowest time tag. Raises an exception if the queue is empty.  
     * @returns The time tag of the event with the lowest time tag.  
     * @throws EmptyQueue if the queue is empty.  
     */  
    abstract pop(): number;  
}
```

Test of OO classes

- Function push()

Empty	Repeated elements	Sign > 0	Test case
yes	yes	Yes	Reset(); Push(10); Push(10); Pop() → 10; Pop(); → EmptyQueue
		No	Reset();Push(-10); Push(-10); Pop(); → EmptyQueue
	no	Yes	
		No	
no	yes	Yes	
		No	
	no	Yes	
		No	

Test of OO classes

- Function reset()

Empty	Repeated elements	Sign > 0	Test cases
yes	yes	Yes	NA
		No	NA
	no	Yes	reset(); pop() → EmptyQueue
		No	NA
no	yes	Yes	NA
		No	NA
	no	Yes	NA
		No	NA

Unit test black box - summary

- Functional test of units (functions, classes) generates test cases starting from the specification of the unit
- Key techniques are
 - Random
 - Equivalence classes partitioning
 - Boundary conditions

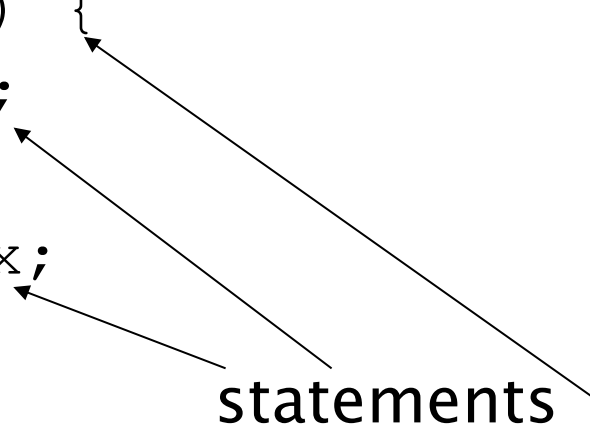
Unit test - White box

Unit test

- Black box (functional)
 - Random
 - Equivalence classes partitioning
- White Box (structural)
 - Coverage of structural elements
 - Statement
 - Decision, condition (simple, multiple)
 - Path
 - Loop

Statement coverage

```
function abs(x: number) : number {  
    if (x >= 0) {  
        return x;  
    } else {  
        return -x;  
    }  
}
```



statements

Two test cases to cover all statements T1(1; 1)
T2(-1; 1)

Statement coverage

Try to execute all statements in the program

Measure:

statement coverage =

$\frac{\text{\#statements covered}}{\text{\#statements}}$

Problem: statement?

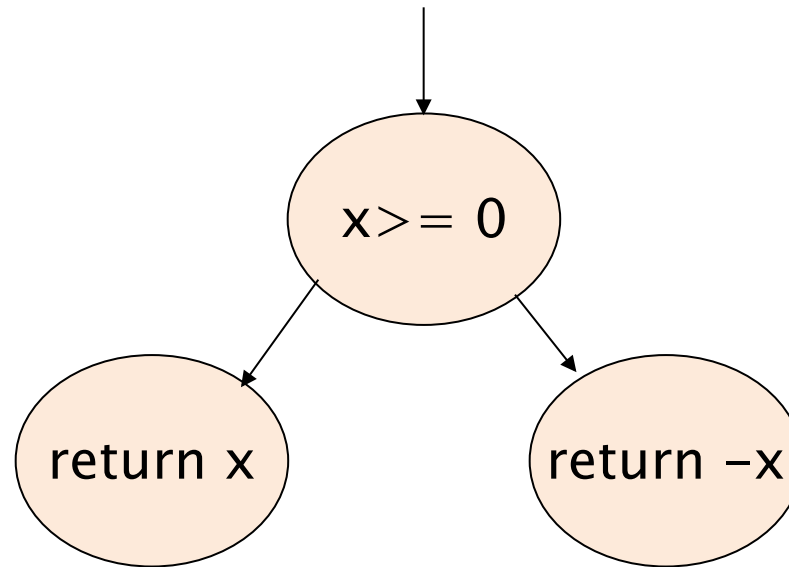
```
function abs(x: number) : number {  
    if (x >= 0) {  
        return x;  
    } else {  
        return -x;  
    }  
}
```

Transform program in control flow

- Node:
 - atomic instruction
 - decision
- Edge: transfer of control
- and basic blocks
 - Nodes can be collapsed in basic blocks
 - A basic block has only one entry point at initial instruction and one exit point at final instruction

Control flow graph

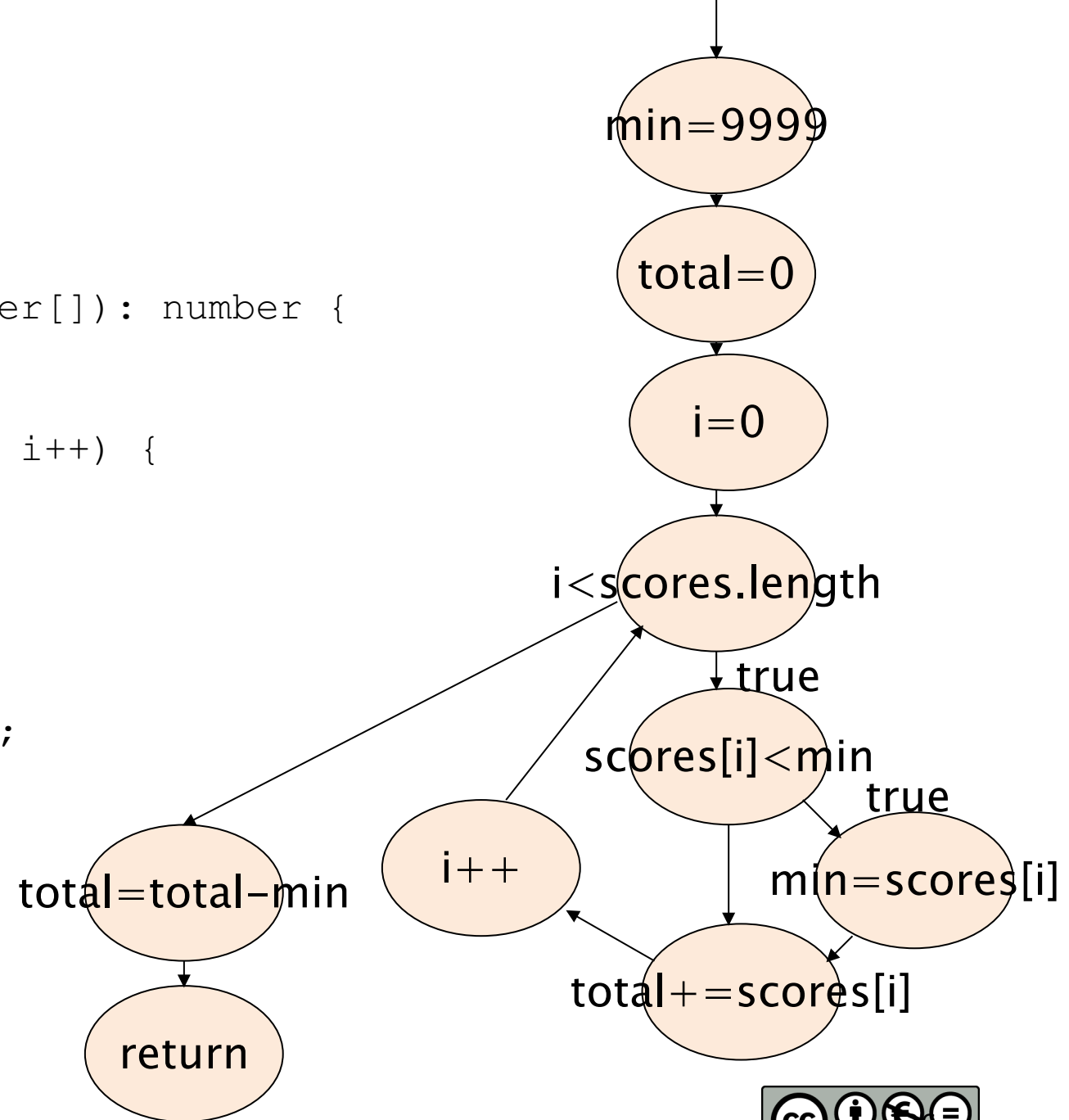
```
function abs(x: number): number {  
    if (x >= 0) {  
        return x;  
    } else {  
        return -x;  
    }  
}
```



Control flow graph

```
function homeworkAverage(scores: number[]): number {  
  let min = 99999;  
  let total = 0;  
  for (let i = 0; i < scores.length; i++) {  
    if (scores[i] < min)  
      min = scores[i];  
    total += scores[i];  
  }  
  total = total - min;  
  return total / (scores.length - 1);  
}
```

T1: input: scores[10.0, 10.0]
Exp Output: 10.0

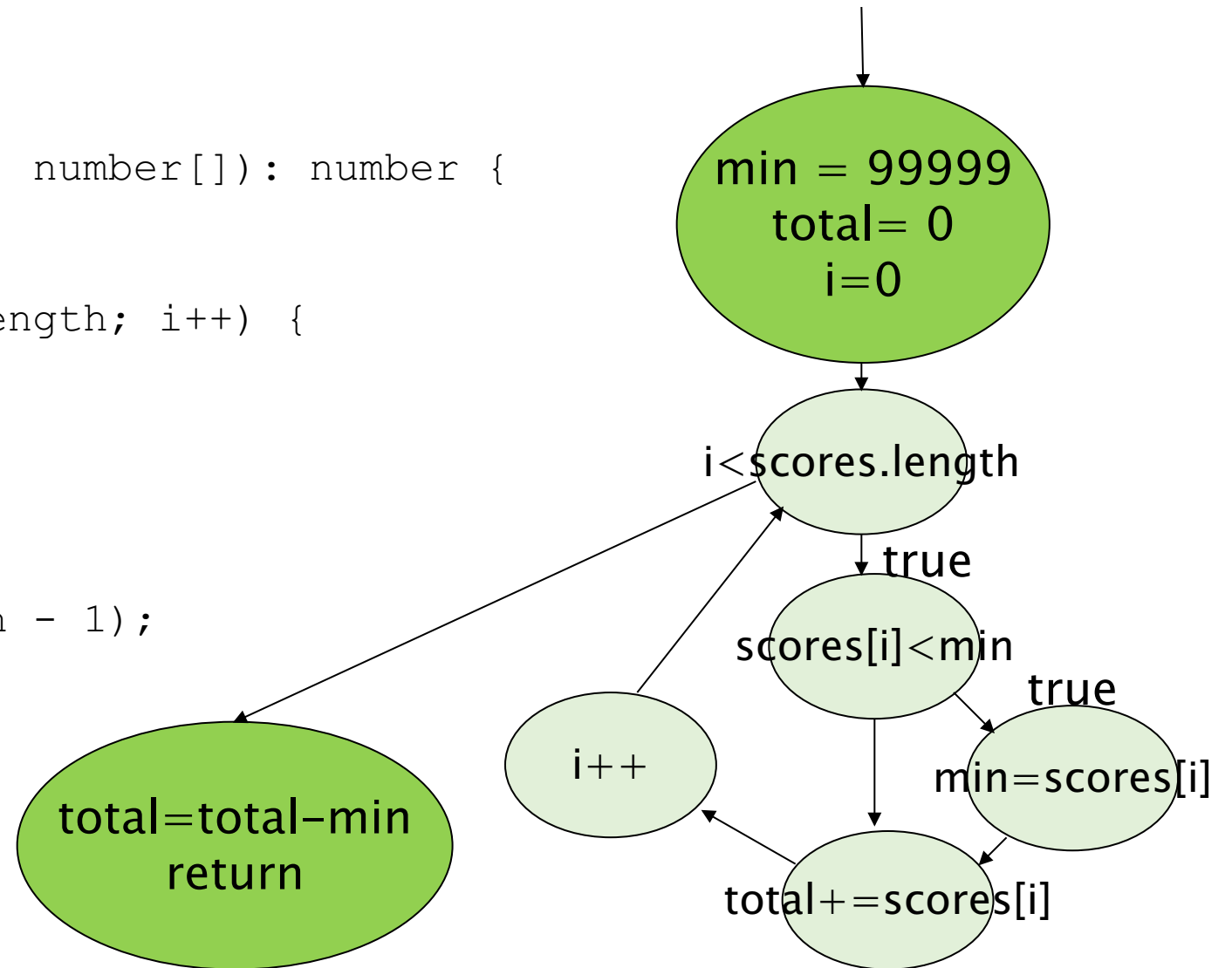


Basic blocks

```
function homeworkAverage(scores: number[]): number {  
  let min = 99999;  
  let total = 0;  
  for (let i = 0; i < scores.length; i++) {  
    if (scores[i] < min)  
      min = scores[i];  
    total += scores[i];  
  }  
  total = total - min;  
  return total / (scores.length - 1);  
}
```

T1(scores[10.0,10.0] ; 10.0)

100% node
coverage

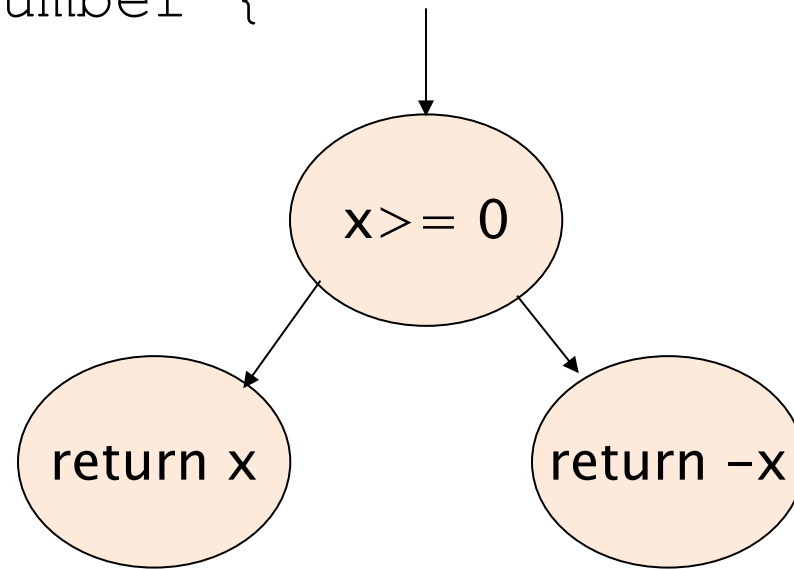


Measure: Node coverage

- Node coverage = number of nodes executed / total number of nodes
- For each test
- Cumulative: for a test suite

Node coverage

```
function abs(x: number): number {  
  if (x >= 0) {  
    return x;  
  } else {  
    return -x;  
  }  
}
```



T1 (1; 1)

T2 (-1; 1)

Statement coverage = node coverage

Statement coverage

- Node coverage $\Leftrightarrow$ Statement coverage
- Basic block cov $\Leftrightarrow$ Statement coverage

Decision coverage

Try to cover all decisions in the program with true and false

Decision:

- boolean expression

Measure:

- $\text{decision coverage} = \frac{\text{\#decisions covered}}{\text{\#decisions}}$

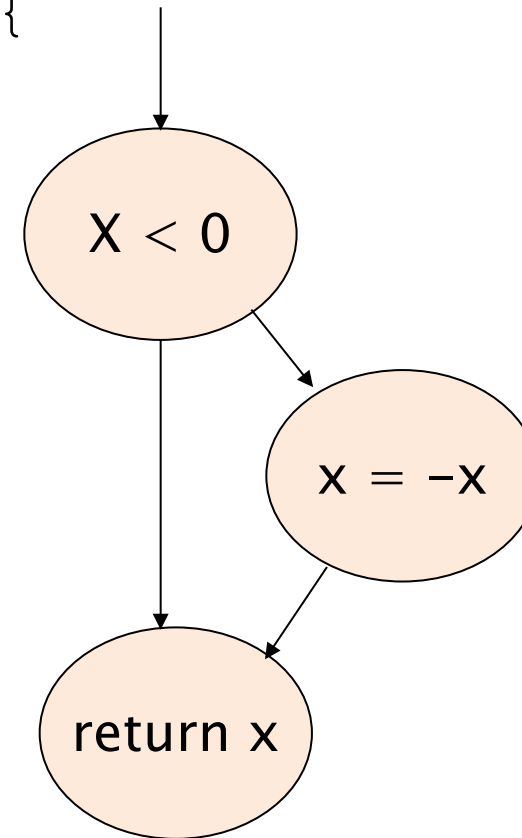
Decision coverage

- Edge coverage $\Leftrightarrow$ Decision coverage

Edge coverage

```
function abs(x: number): number {  
    if (x < 0)  
        x = -x;  
    return x;  
}
```

T1 (1;1)
T2 (-1; 1)

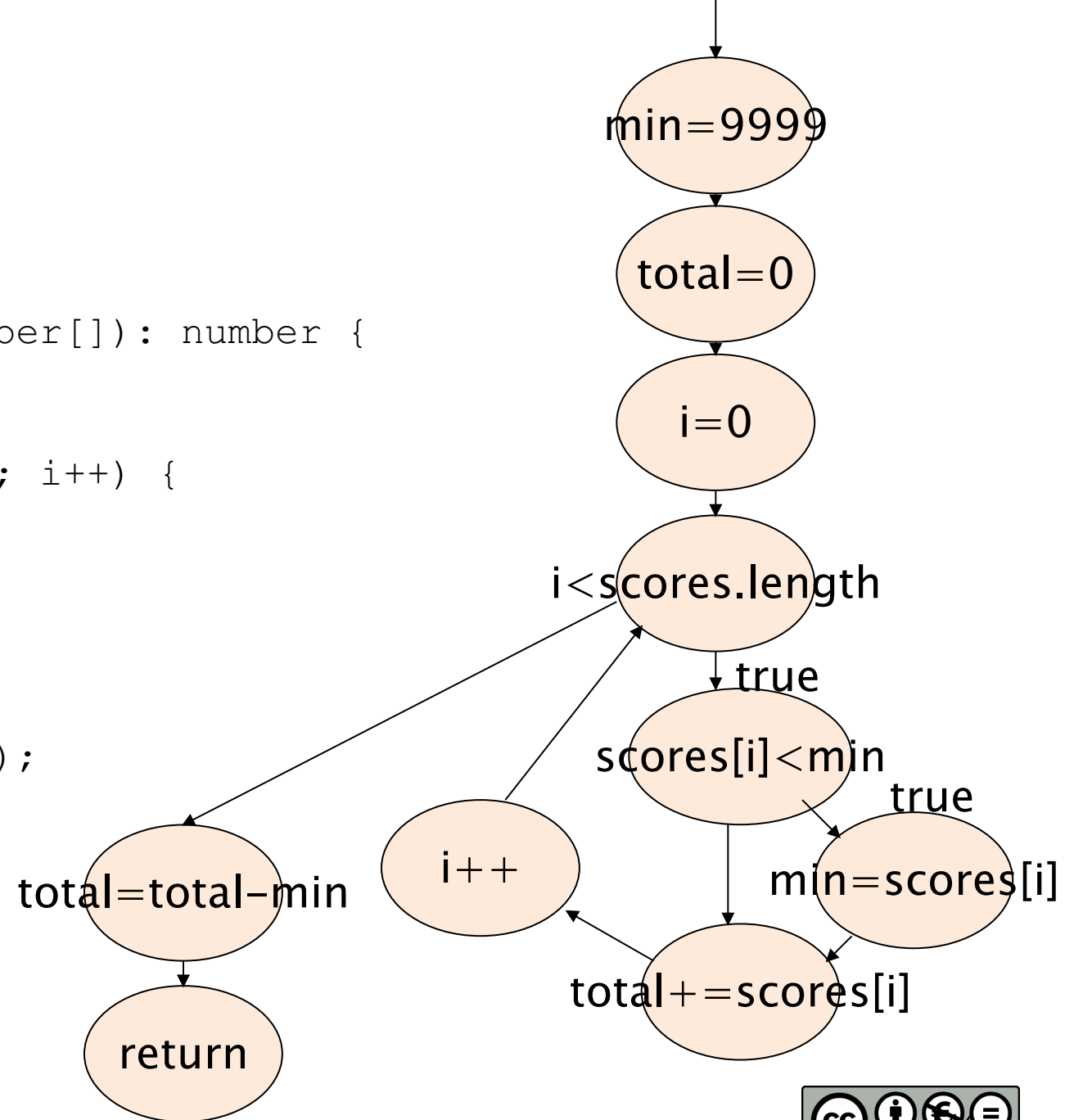


Edge coverage

```
function homeworkAverage(scores: number[]): number {  
  let min = 99999;  
  let total = 0;  
  for (let i = 0; i < scores.length; i++) {  
    if (scores[i] < min)  
      min = scores[i];  
    total += scores[i];  
  }  
  total = total - min;  
  return total / (scores.length - 1);  
}
```

T1({10.0}; 10.0)

T2({1,2}; 1.5)



Relations

Edge coverage implies node coverage
not viceversa

Condition coverage

A decision can be made of a combination of terms (== conditions)

Ex:

```
boolean isMarried;
```

```
boolean isRetired;
```

```
int age;
```

```
if (age>60 and isRetired or isMarried)
```

Condition coverage

- Simple condition coverage
 - Each condition set at least once to T and F
- Multiple condition coverage
 - All combinations T/F are tried

Simple condition coverage

```
{  
    let isMarried: boolean;  
    let isRetired: boolean;  
    let age: number;  
    let discountRate: number;  
  
    if ((age > 60 && isRetired) || isMarried) {  
        discountRate = 30;  
    } else {  
        discountRate = 10;  
    }  
}
```

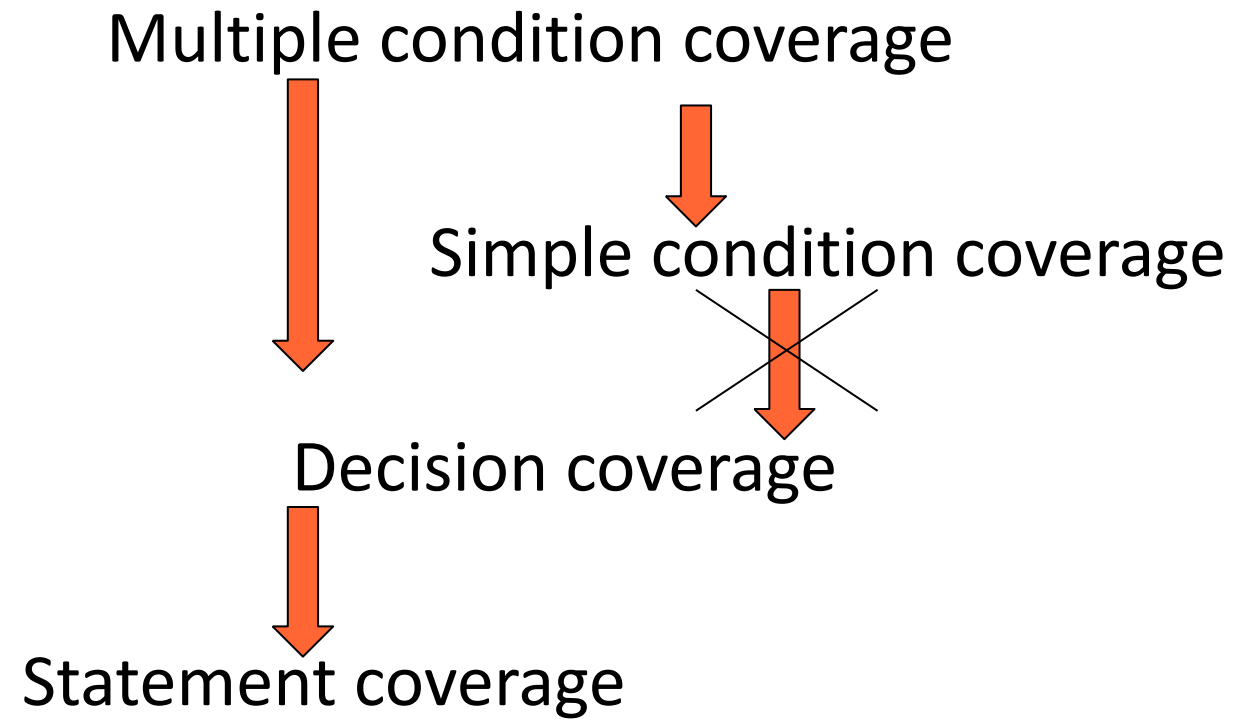
Test case	age>60	isRetired	isMarried	Decision
T1	T	T	T	T
T2	F	F	F	F

Multiple condition coverage

```
{  
  let isMarried: boolean;  
  let isRetired: boolean;  
  let age: number;  
  let discountRate: number;  
  
  if ((age > 60 && isRetired) || isMarried) {  
    discountRate = 30;  
  } else {  
    discountRate = 10;  
  }  
}
```

Test case	age>60	isRetired	isMarried	Decisio
T1	T	T	T	T
T2	T	T	F	T
T3	T	F	T	T
T4	T	F	F	F
T5	F	T	T	T
T6	F	T	F	F
T7	F	F	T	T
T8	F	F	F	F

Relations



Test case	age>60	isRetired	isMarried	Decision
T1	T	T	T	T
T2	T	T	F	T
T3	T	F	T	T
T4	T	F	F	F
T5	F	T	T	T
T6	F	T	F	F
T7	F	F	T	T
T8	F	F	F	F

- T2 and T7 provide simple condition coverage, but no decision coverage

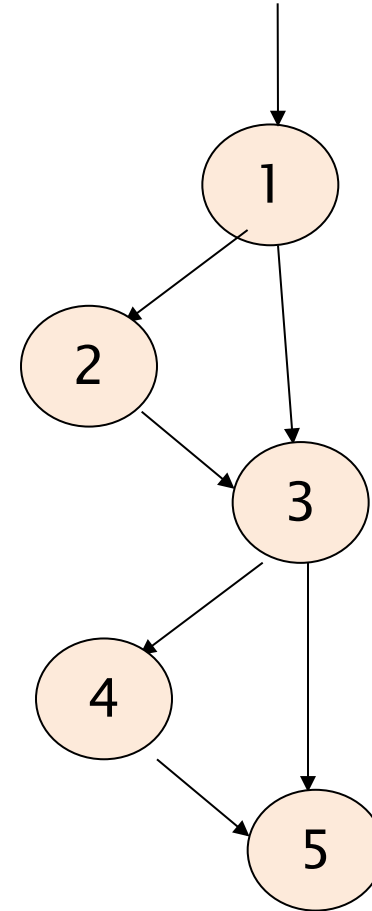
- T1 and T8 achieve both
 - Simple condition coverage
 - Decision coverage

Path coverage

- Path = sequence of nodes in a graph
- select test cases such that every path in the graph is visited

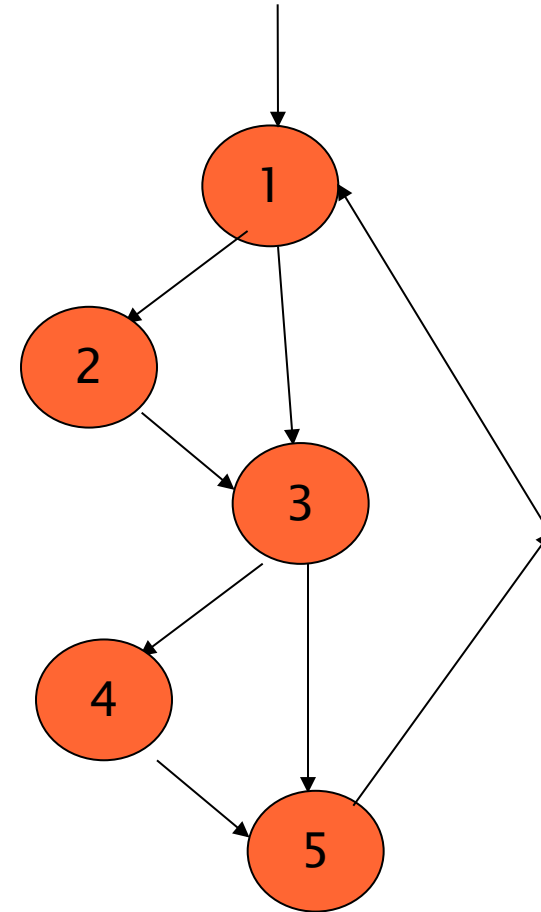
Path coverage

- Ex. 4 paths in this simple graph
 - 1,2,3,4,5
 - 1,3,5
 - 1,3,4,5
 - 1,2,3,5



Path coverage

- Combinatorial explosion with cycle
 - 1,3,5
 - 1,3,5,1,3,5
 - 1,3,5,1,3,5,1,3,5
 - Etc ..
 - $N_{paths} = 4^{n_{loops}}$



Path coverage

- In most cases unfeasible if graph is cyclic
- Approximations
 - Path-n
 - Path-4 == loop 0 to 4 times in each loop
 - Loop coverage
 - In each loop cycle 0, 1 , >1 times

Loop coverage

- select test cases such that every loop boundary and interior is tested
 - Boundary: 0 iterations
 - Interior: 1 iteration and > 1 iterations
- Coverage formula: $x/3$

Loop coverage

- Consider each loop (for, while) separately
- Write 3 test cases
 - No enter the loop
 - Cycle once in the loop
 - Cycle more than once

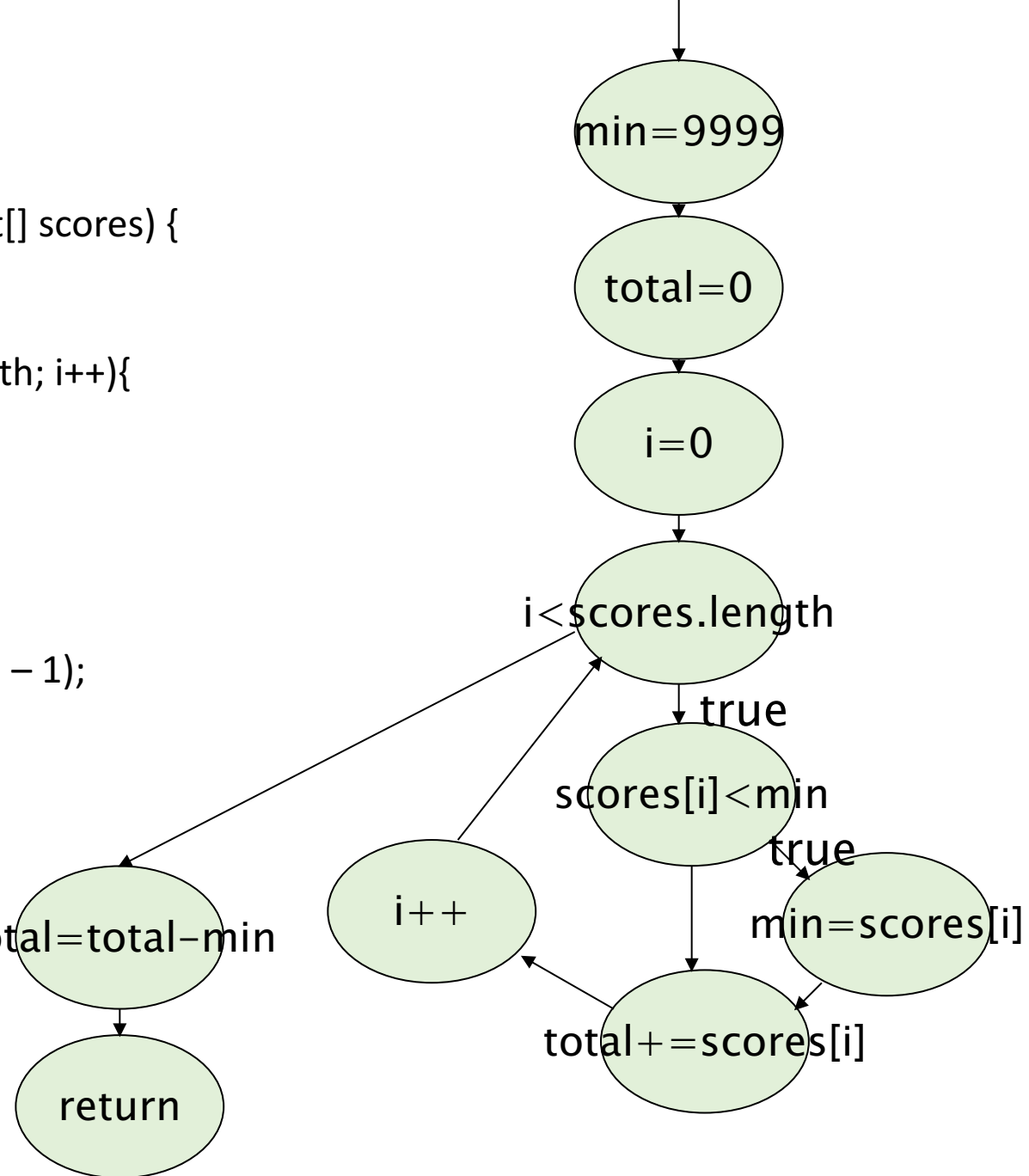
Loop coverage

```
float homeworkAverage(float[] scores) {  
    float min = 99999;  
    float total = 0;  
    for (int i = 0; i < scores.length; i++){  
        if (scores[i] < min)  
            min = scores[i];  
        total += scores[i];  
    }  
    total = total - min;  
    return total / (scores.length - 1);  
}
```

T1({1}; 1) loops 1

T2({1,2,3}; 2) loops 2

T3({}; ?) loops 0



Tools

- To write and run test cases
 - Ex JUnit , Jest
- To compute coverage
 - Ex. Cobertura, Clover, Jacoco, Jest

Jest

The screenshot shows a VS Code editor with a project structure in the Explorer on the left. The project includes a `src` directory with `mathFunctions.ts` and a `test` directory with `mathFunctions.test.ts`. The `mathFunctions.test.ts` file is open in the editor, showing a series of Jest tests for the `abs` and `homeworkAverage` functions. The tests cover various inputs, including positive and negative numbers, zero, and arrays of different lengths and values. The bottom panel shows the terminal output of running `jest % npm test`, which displays a successful test run with 7 tests passed.

```
src > test > TS mathFunctions.test.ts > ...
1 import { abs, homeworkAverage } from '../mathFunctions';
2
3 describe('abs', () => {
4   test('returns the absolute value of a positive number', () => {
5     expect(abs(10)).toBe(10);
6   });
7
8   test('returns the absolute value of a negative number', () => {
9     expect(abs(-20)).toBe(20);
10  });
11
12  test('returns zero if zero is passed', () => {
13    expect(abs(0)).toBe(0);
14  });
15 });
16
17 describe('homeworkAverage', () => {
18   test('calculates average excluding the lowest score', () => {
19     expect(homeworkAverage([90, 100, 80])).toBe(95);
20   });
21
22   test('handles an array with one score', () => {
23     expect(homeworkAverage([100])).toBe(0);
24   });
25
26   test('returns the average of multiple identical scores', () => {
27     expect(homeworkAverage([100, 100, 100])).toBe(100);
28   });
29
30   test('returns zero if the array is empty', () => {
31     expect(homeworkAverage([])).toBe(0);
32   });
33 });
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
hardo@yanez jest % npm test
> jest@1.0.0 test
> jest
PASS src/test/mathFunctions.test.ts
  abs
    ✓ returns the absolute value of a positive number (2 ms)
    ✓ returns the absolute value of a negative number
    ✓ returns zero if zero is passed (1 ms)
  homeworkAverage
    ✓ calculates average excluding the lowest score (2 ms)
    ✓ handles an array with one score
    ✓ returns the average of multiple identical scores
    ✓ returns zero if the array is empty
Test Suites: 1 passed, 1 total
Tests: 7 passed, 7 total
Snapshots: 0 total
Time: 0.684 s, estimated 1 s
Ran all test suites.
hardo@yanez jest %
```

Ln 1, Col 55 Spaces: 2 UTF-8 LF TypeScript Prettier

Istanbul + Coverage Gutters

The screenshot displays a development environment with VS Code and a web browser. The VS Code interface shows the Explorer, Search, and Run and Debug panels. The main editor displays the file `mathFunctions.test.ts` with Jest test cases. The Test Results panel shows the test suite `src/test/mathFunctions.test.ts` passing all tests. The Coverage panel shows the Istanbul coverage report for `mathFunctions.ts`.

Test Results:

```
> npm run test --coverage
> jest@1.0.0 test
> jest
PASS src/test/mathFunctions.test.ts
  abs
    ✓ returns the absolute value of a positive number (1 ms)
    ✓ returns the absolute value of a negative number
    ✓ returns zero if zero is passed (1 ms)
  homeworkAverage
    ✓ calculates average excluding the lowest score
    ✓ handles an array with one score
    ✓ returns the average of multiple identical scores
    ✓ returns zero if the array is empty (1 ms)
```

Coverage Report:

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	100	100	100	100	
mathFunctions.ts	100	100	100	100	

Test Suites: 1 passed, 1 total
Tests: 7 passed, 7 total
Snapshots: 0 total
Time: 0.723 s, estimated 1 s
Ran all test suites.
hardo@yanez jest %

The web browser shows the Istanbul coverage report for `mathFunctions.ts` at `http://127.0.0.1:3000/coverage/cov-report/mathFunctions.ts.html`. The report shows 100% coverage for all metrics: Statements (14/14), Branches (5/5), Functions (2/2), and Lines (13/13).

```
export function abs(x: number): number {
  if (x >= 0) {
    return x;
  } else {
    return -x;
  }
}

export function homeworkAverage(scores: number[]): number {
  let min = Infinity;
  let total = 0;
  for (let i = 0; i < scores.length; i++) {
    if (scores[i] < min) {
      min = scores[i];
      total += scores[i];
    }
  }
  total = total - min;
  return scores.length > 1 ? total / (scores.length - 1) : 0;
}
```

Summary

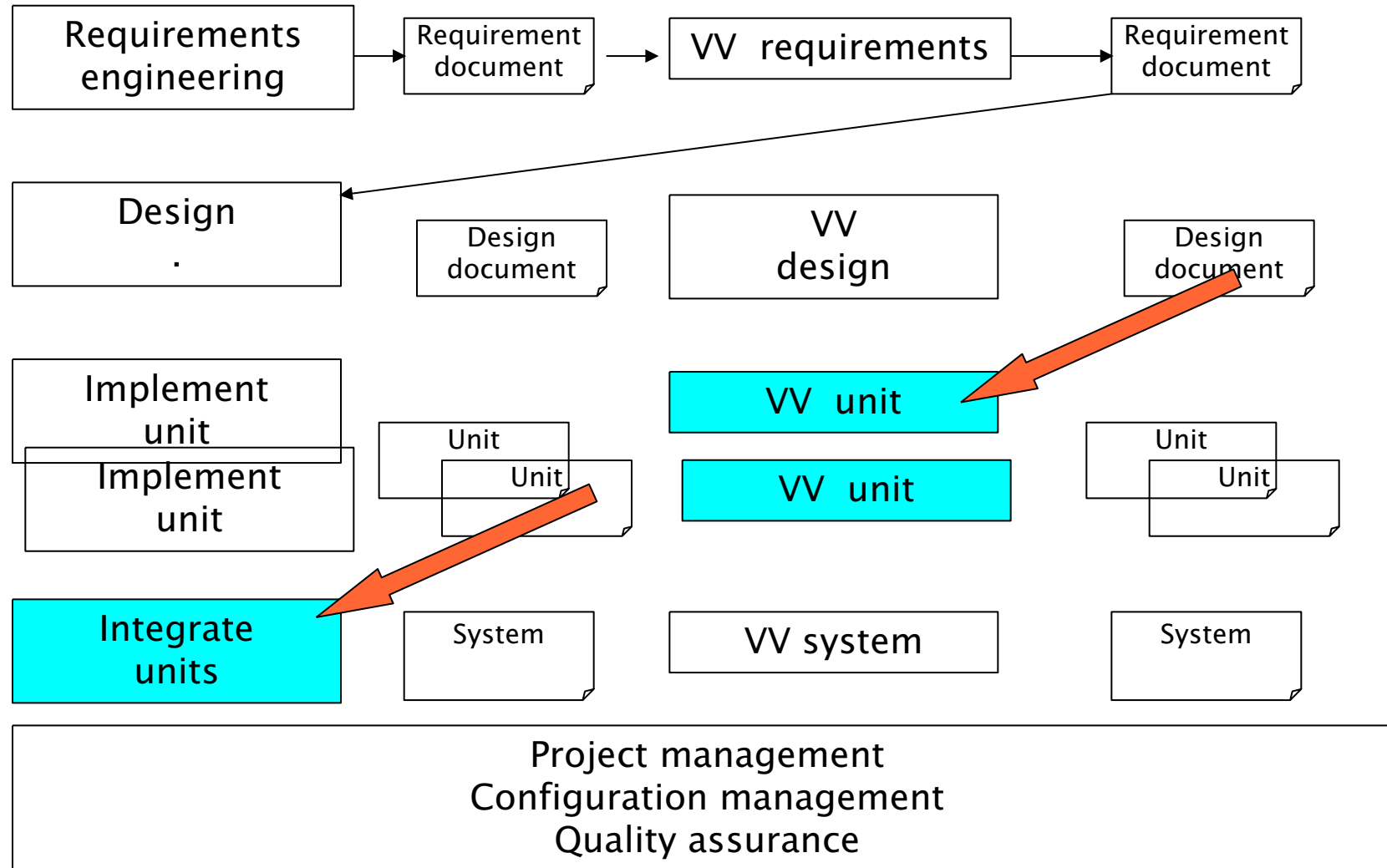
- Structural / white box testing starts from the code, and uses several coverage objectives
 - Statements
 - Decisions
 - Conditions (simple, multiple)
 - Path
 - Loop

Summary

- White box testing is typically made in the development environment and supported by tools to compute coverage

Integration test

Integration test

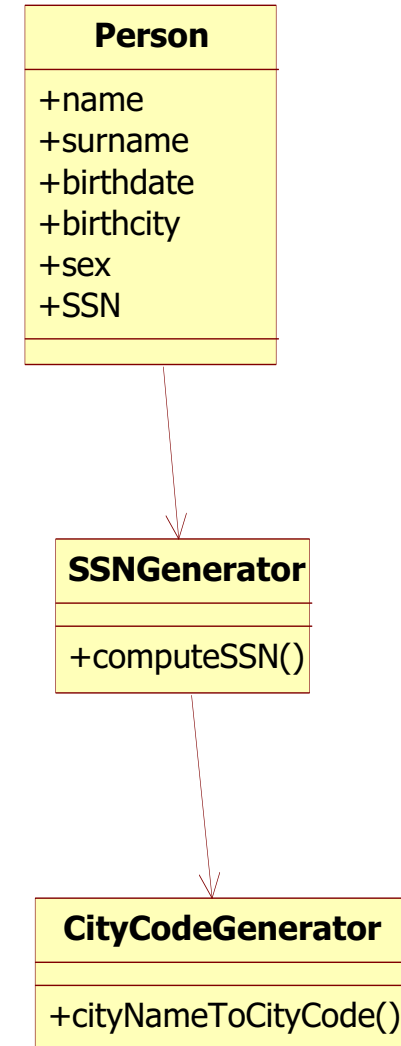


Integration test

- Test of some dependent units
 - Unit:
 - function (procedural languages)
 - class and its methods (oo languages)

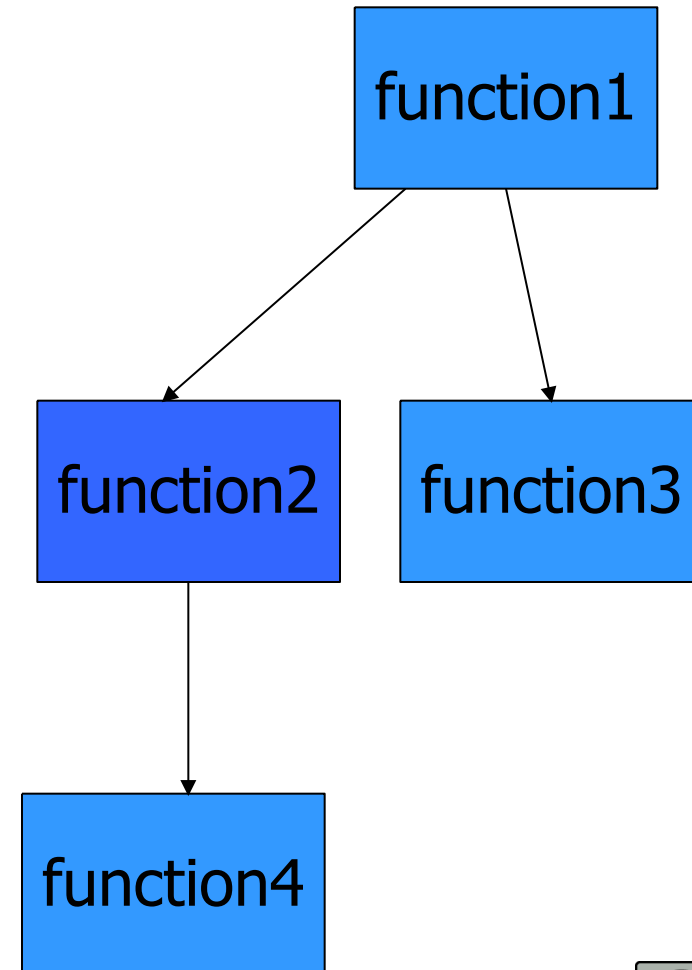
The problem

- Some units need others.
How to test them?



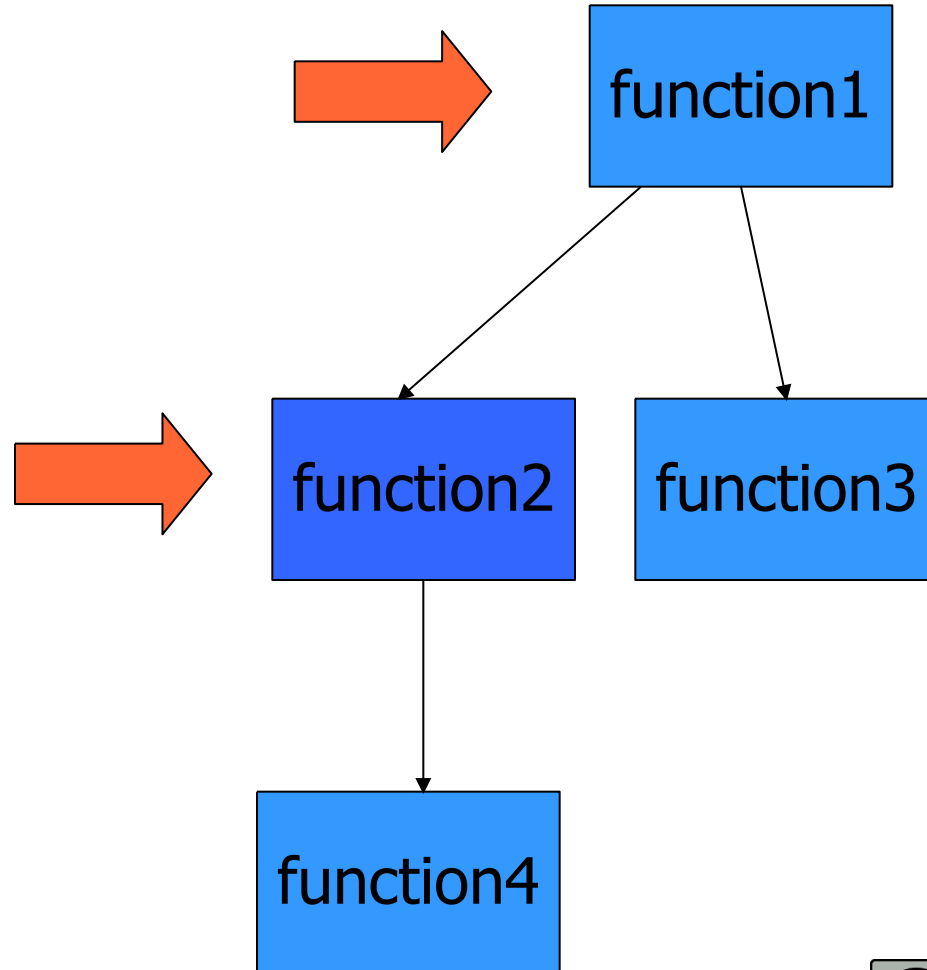
The dependency graph

```
function1() {  
    // call function2();  
    // call function3();  
    // some other code  
}  
function2() {  
    //some code call function4();  
}
```



The problem -2

- Function3, function4 have no dependency
 - Unit test techniques can be applied
- Function1, function2 have dependencies, how to test them?



In fact two (related) problems

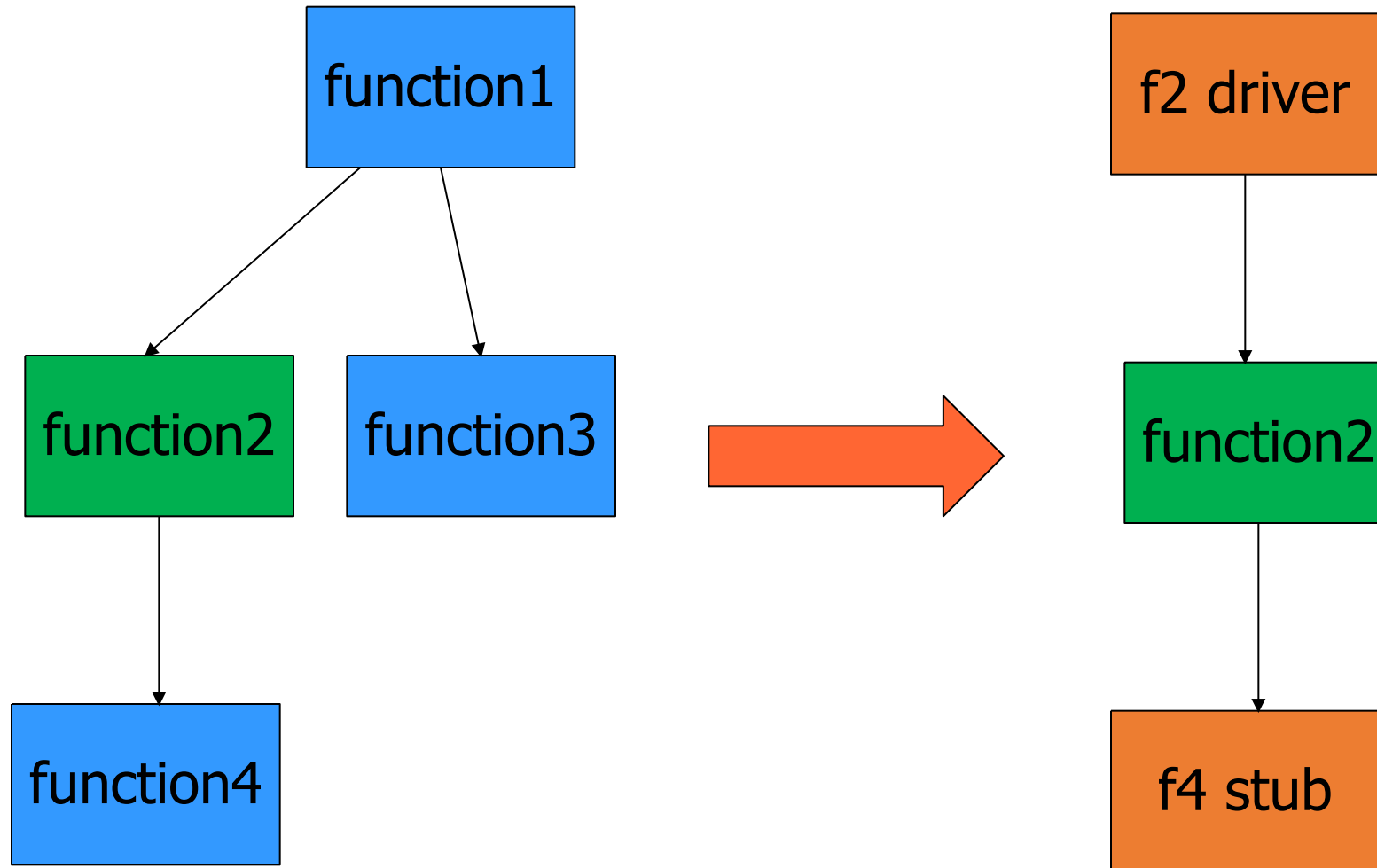
- How to test 'independently' a function that has dependencies?
 - Ex Function1()
- How to test the dependency?

How to test independently a function with dependencies

Technique

- Stubs
- Try to eliminate the dependency using substitutes

Goal: test function2



Stub, driver

- Driver
 - Unit (function or class) developed to pilot another unit
- Stub
 - Unit developed to substitute another unit (fake unit)
- Also called mock ups, mocks

Ex. driver (Jest)

```
import { Converter } from './Converter';

describe('Converter', () => {
  let converter: Converter;

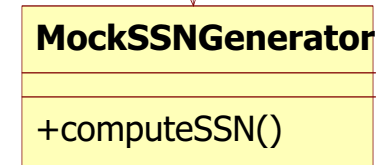
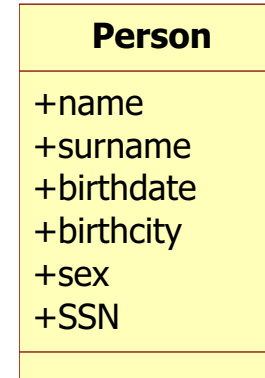
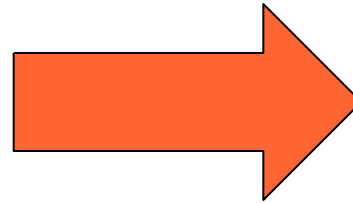
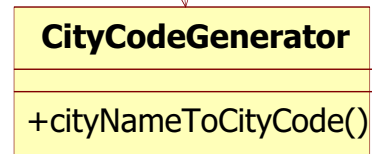
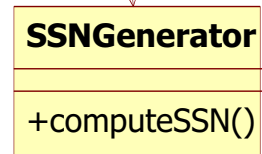
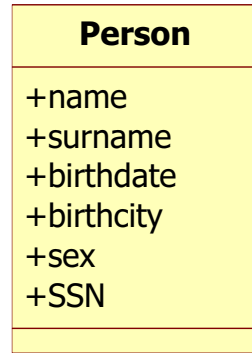
  beforeEach(() => {
    converter = new Converter();
  });

  test('throws an exception when the input
  array length exceeds 6 elements', () => {
    const str = ['1', '2', '3', '4', '5',
    '6', '7'];
    expect(() => {
      converter.convert(str);
    }).toThrow("Input array length
    exceeds 6 elements.");
  });
});
```

```
export class Converter {
  public convert(str: string[]): number {
    if (str.length > 6) {
      throw new Error("Input array length exceeds 6 elements.");
    }
    let number = 0;
    let digit;
    let i = 0;
    if (str[0] === '-') {
      i = 1;
    }
    for (; i < str.length; i++) {
      digit = parseInt(str[i], 10);
      if (isNaN(digit)) {
        throw new Error("Invalid character for conversion.");
      }
      number = number * 10 + digit;
    }
    if (str[0] === '-') {
      number = -number;
    }
    if (number > 32767 || number < -32768) {
      throw new Error("Number out of range.");
    }
    return number;
  }
}
```


Stub

- Substitute of module, whose result is known
 - Ex. unit = SSNfunction to compute social security number from name/family name etc.
 - Stub of SSNfunction = returns the needed SSN for a certain test case
- Ex. unit = catalog of products, contains thousands of them
 - StubCatalog = returns a certain product for a given test case



Stub, embedded systems

- Substitues sensors / actuators with pure software units
 - Ex, heating control system
 - Temperature sensor stub, rain sensor stub etc

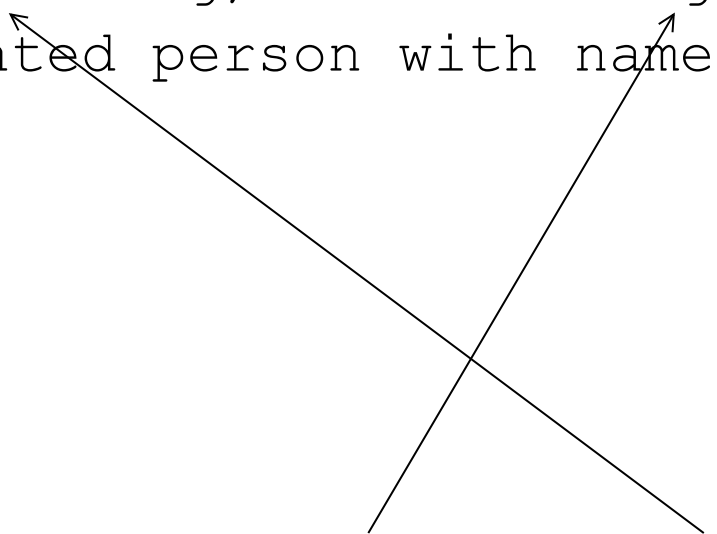
Test dependencies between functions

Dependency defect

- Two units work perfectly when isolated
- Defect happens when connected

Ex. Dependency defect

```
class Person {  
    constructor(surname: string, name: string) {  
        console.log(`Created person with name: ${name}, surname:  
                    ${surname}`);  
    }  
}  
  
function main() {  
    const person = new Person("John", "Wright");  
}
```



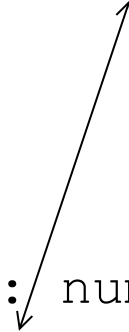
Ex. Dependency defect

```
/**
 * Calculates the surface area of a triangle given its height and base.
 * @param height The height of the triangle.
 * @param base The base of the triangle.
 * @returns The surface area of the triangle.
 */
function triangleSurface(height: number, base: number): number {
    return 0.5 * base * height;
}
function main() {
    console.log("Area with integer values:", triangleSurface(10, 4)); // int instead of float
    console.log("Area with floating-point values:", triangleSurface(3.1, 4.2)); // exchanged base and height
}
```

Ex Dependency defect

- The Mars Polar Lander case (1999)

```
function function(metricMeasure: number): number {  
    //...  
}  
  
function main() {  
    let imperialMeasure: number;  
    function(imperialMeasure);  
}
```

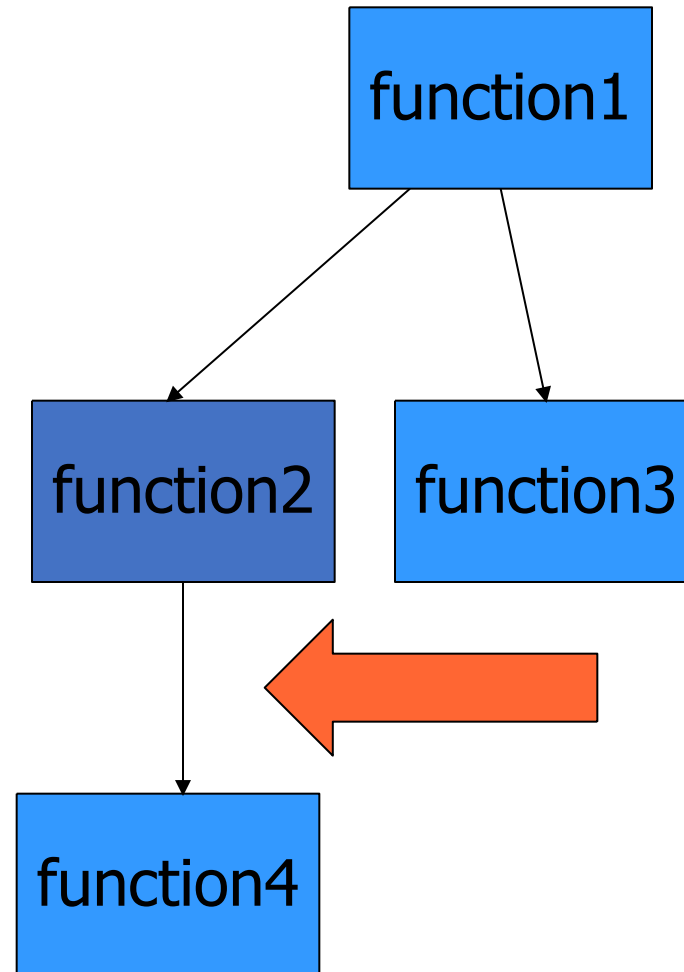


- UK team develops unit assuming metric input
- US team calls unit assuming imperial input
- Difference is small, noticed only at 'landing'

Technique

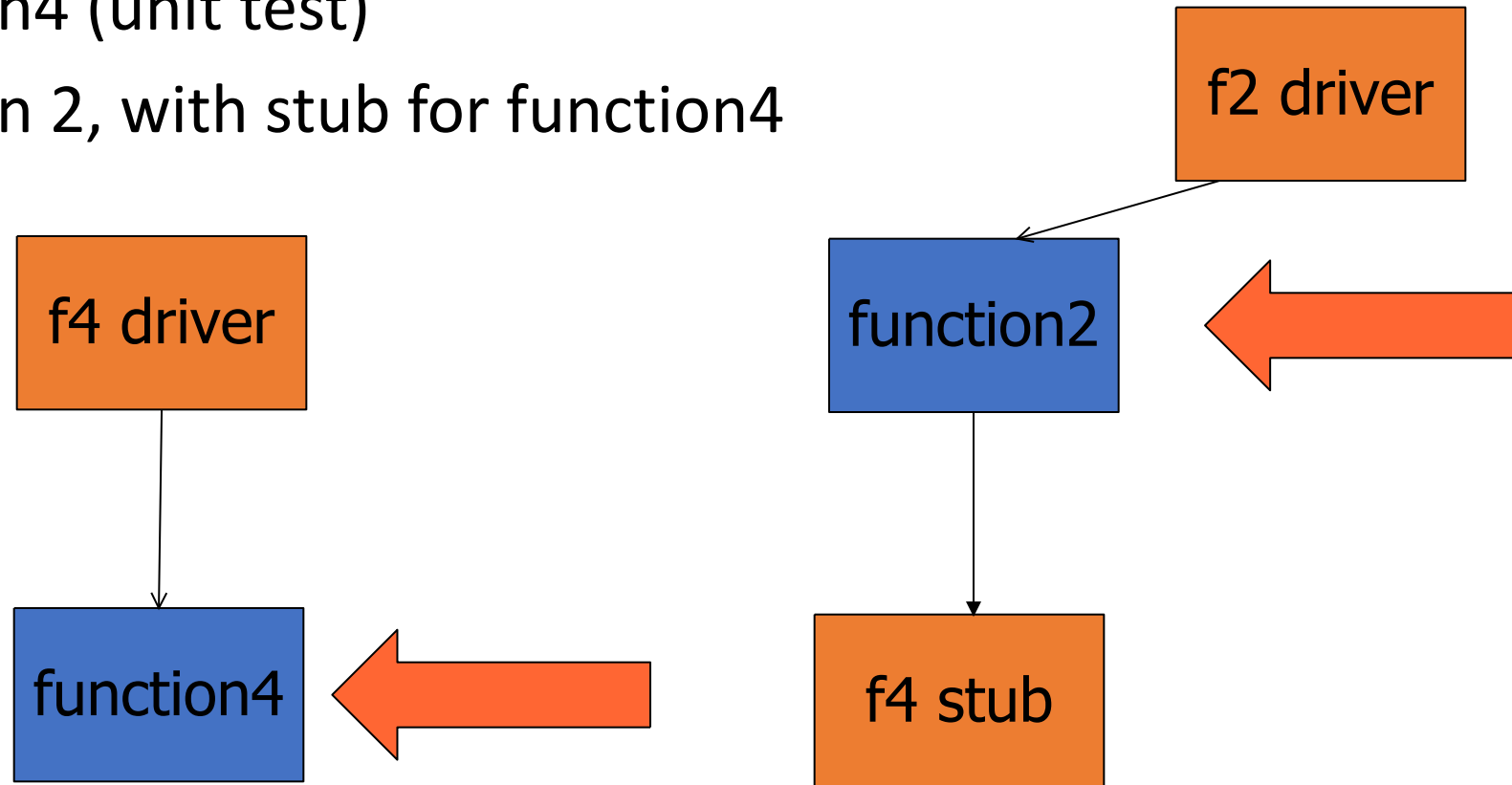
- Two units are tested in isolation first
- Integrate them, test again
 - Focus on the dependency
- In case of more units, integrate incrementally
- In any case, avoid BIG BANG integration

Goal: test dependency function2-function4

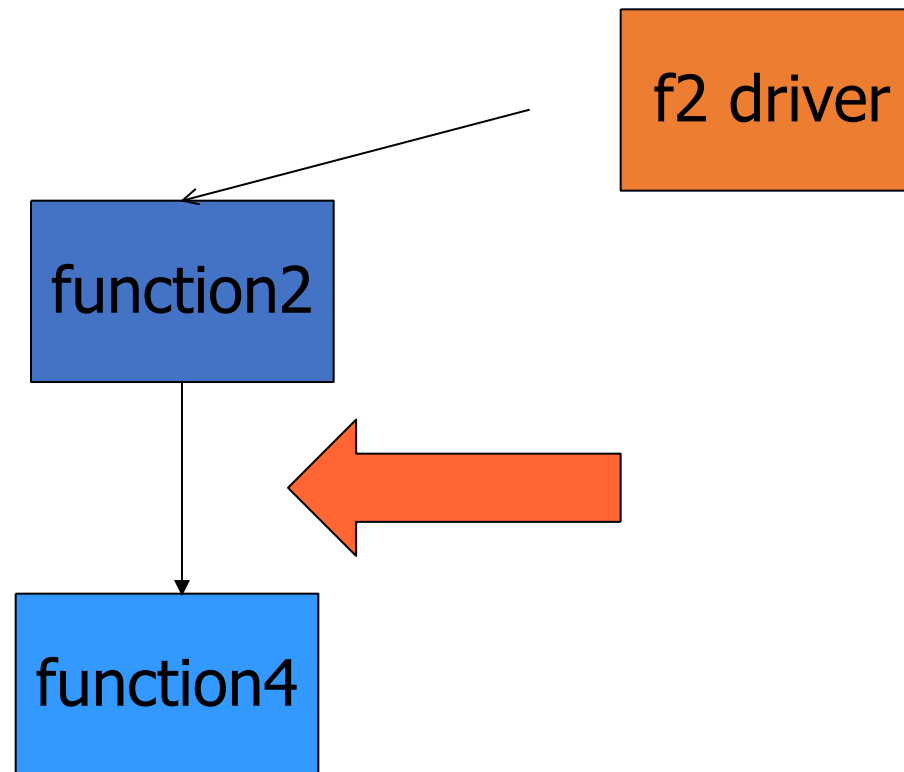


1 test function4 (unit test)

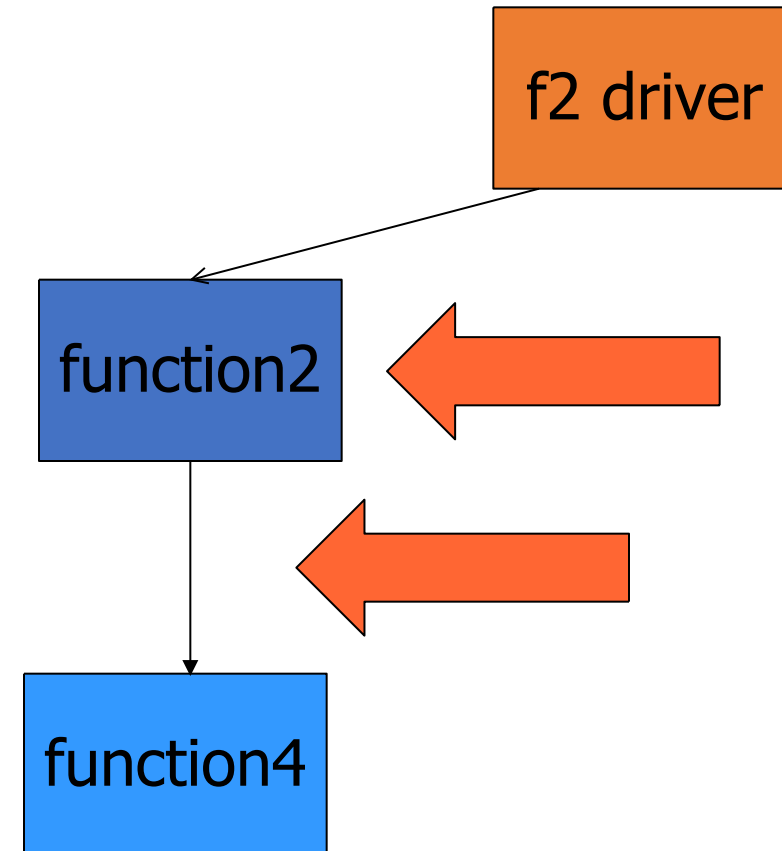
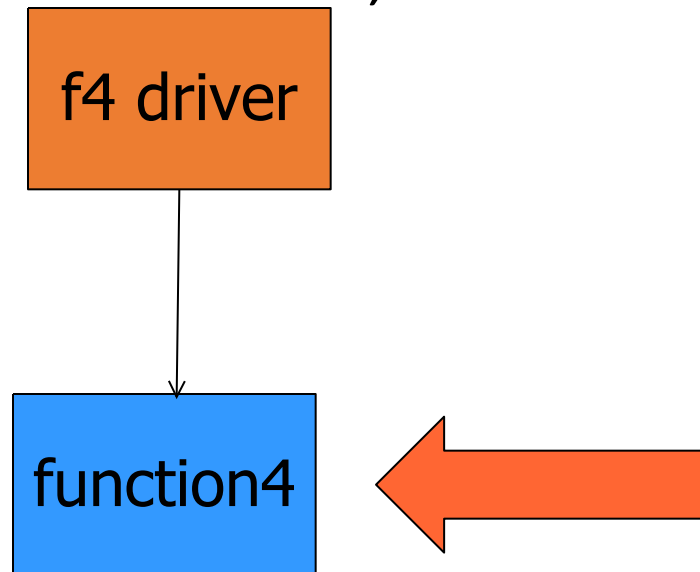
2 test function 2, with stub for function4



3 integrate function 2 and 4, test the dependency



- Other simpler option
- 1 test function4 (unit test)
- 2 integrate function2, test both

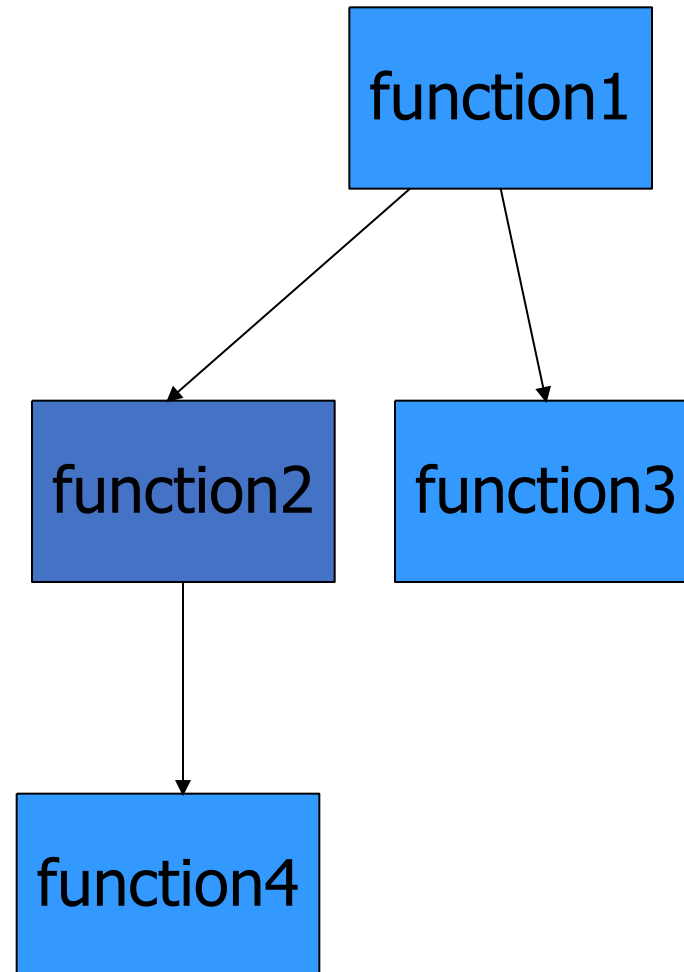


Incremental integration

- Goal:
 - Add one unit at a time, test the partial aggregate
- Pro:
 - Defects found, most likely, come by last unit/interaction added
- Con:
 - More tests to write, stubs/drivers to write

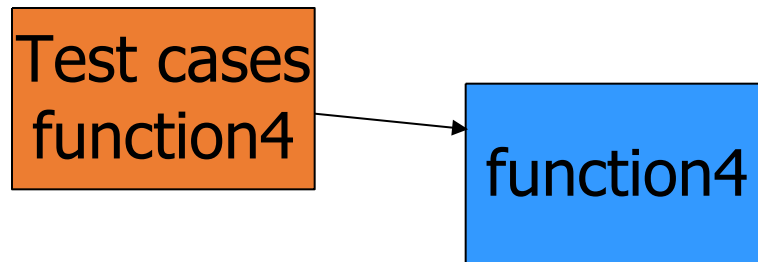
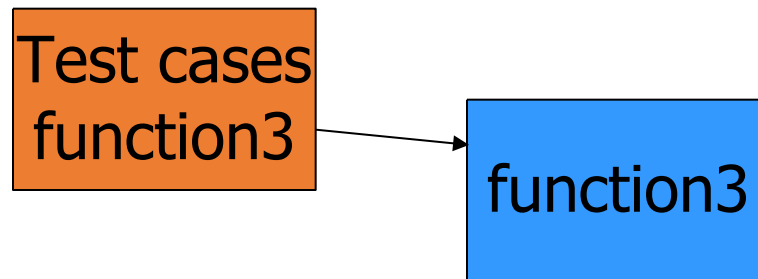
Incremental integration

- Top down
- Bottom up
 - Defined relatively to the dependency graph

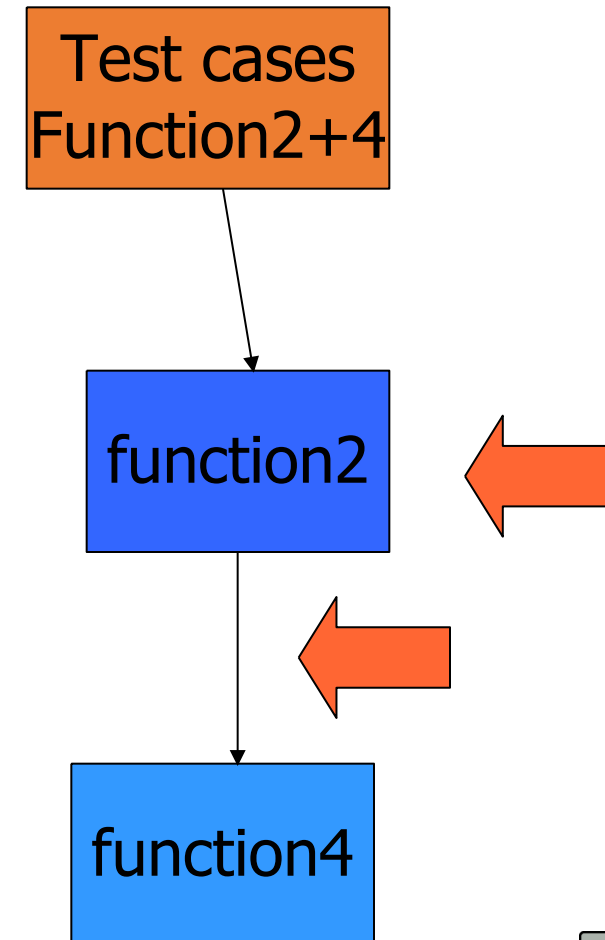


Bottom up

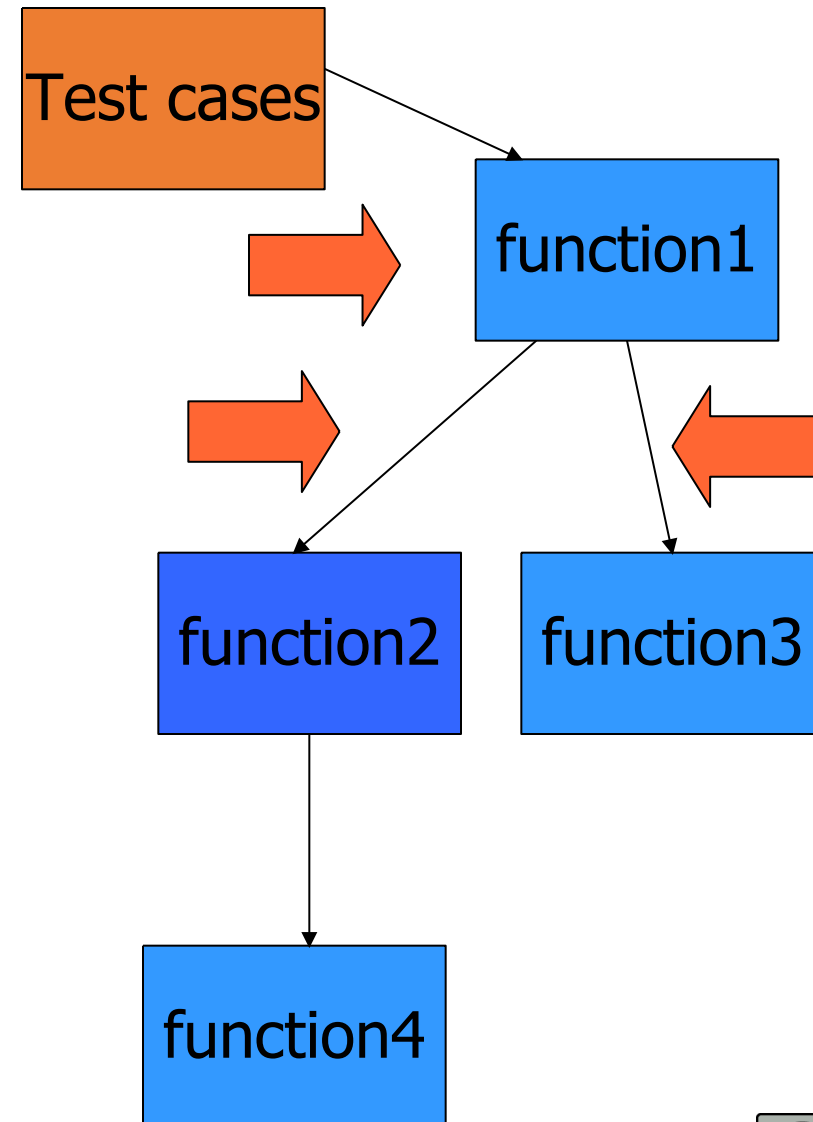
- Step 1
 - function4, function3 have no dependencies. Test them in isolation (unit test)



- Step 2
 - Test function2+function4 as-if they were one unit
 - if defect is found, it should come
 - from function2 or
 - from interaction function1-function4
 - Not from function 4, that is now 'trusted'

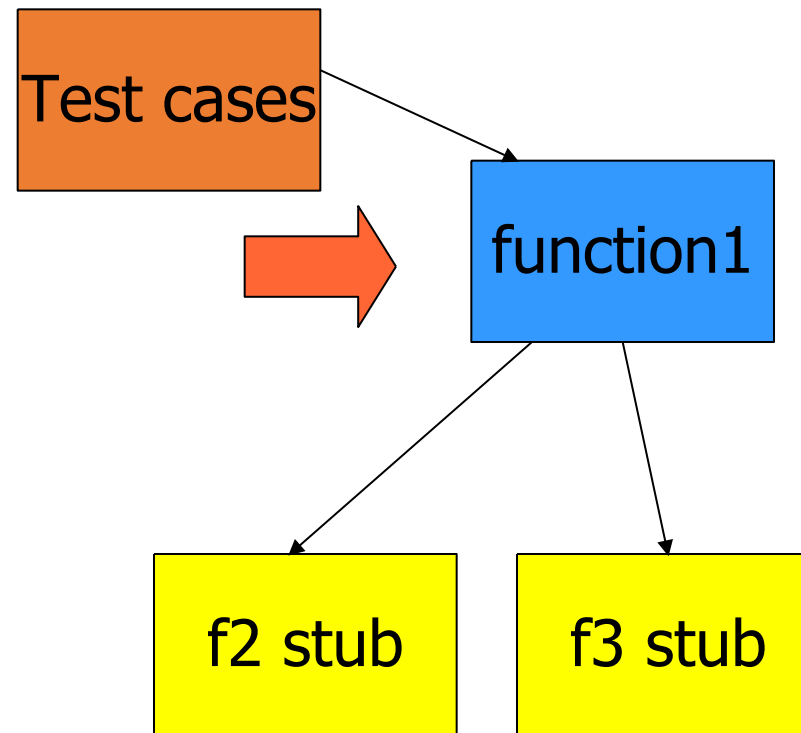


- Step 3
 - Test all
 - if defect is found, it should come
 - from function1 or
 - from interaction function1-function2
 - from interaction function1-function3
 - Not from function 2+4, and function3, that are now 'trusted'



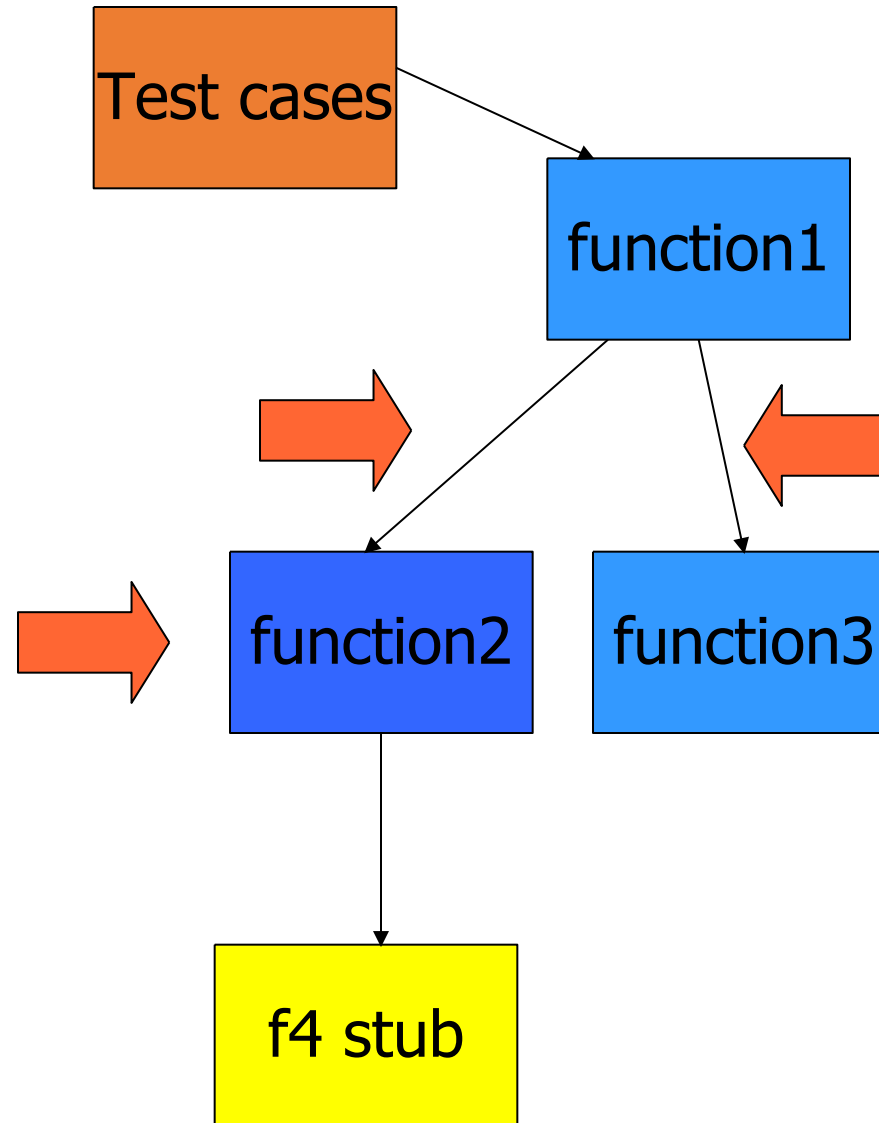
Top down

- Step 1



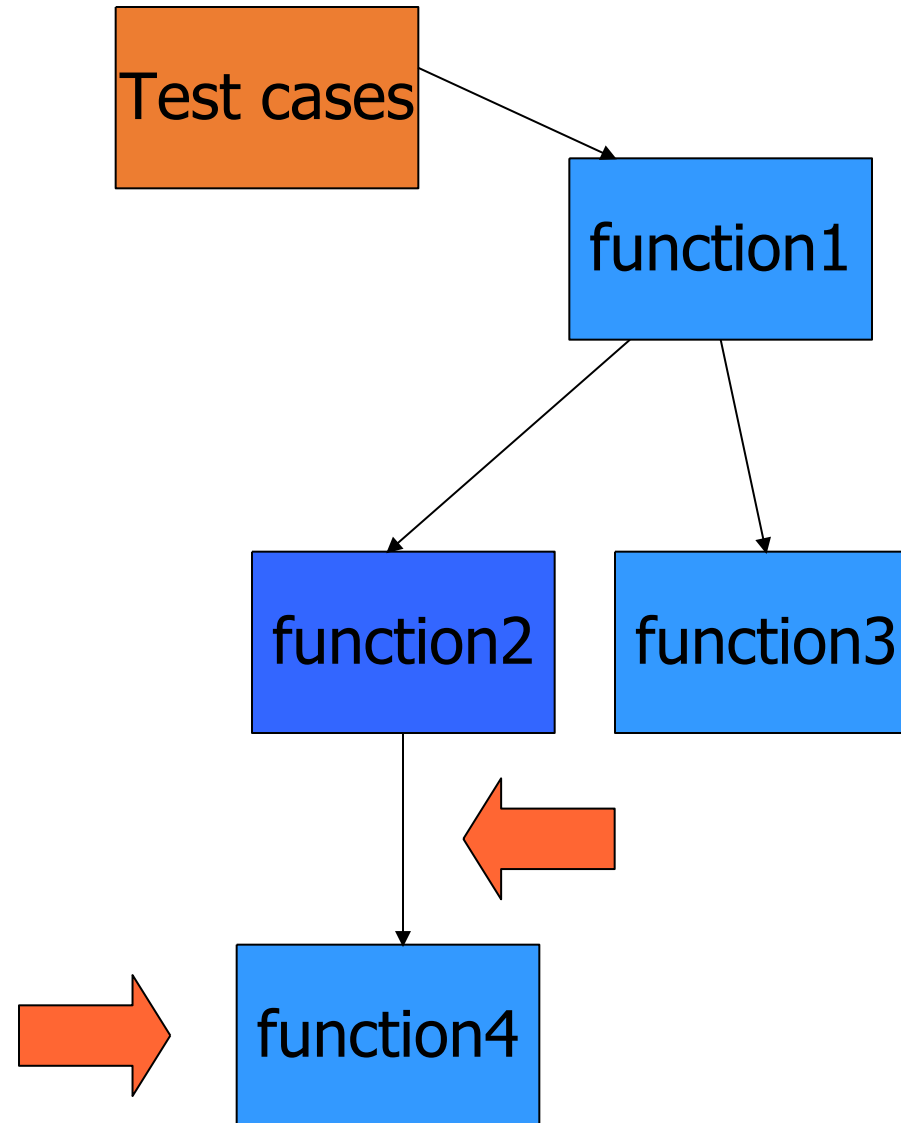
Top down

- Step 2



Top down

- Step 3



Top-down

- Pros
 - Allows early detection of architectural flaws
 - A limited working system is available early
- Cons
 - Requires the definition of stubs for all lower-level units
 - Suitable only for top-down development
 - Lower levels not directly observable

Bottom-up

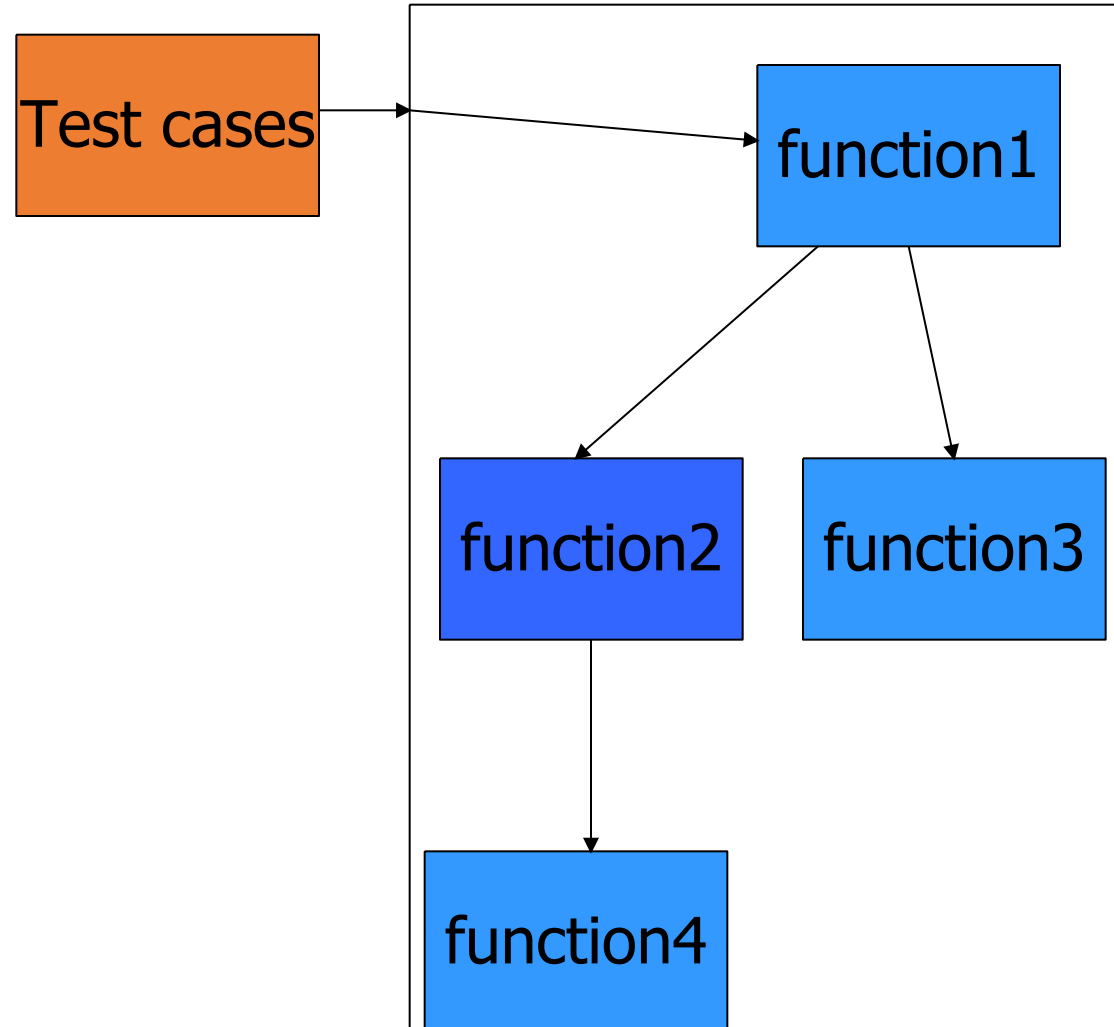
- Pros
 - Testing can start early in the development process
 - Lower levels are directly observable
- Cons
 - Requires the definition of drivers for all lower level units

In practice

- Incremental integration is a mix of top down and bottom up, trying to
 - Minimize stubs creation
 - Compromise with availability of units
 - (units are developed in an order suitable to the integration sequence decided)

Big bang integration

- All functions are developed
- Test is applied directly to the aggregate as-if it were a single unit



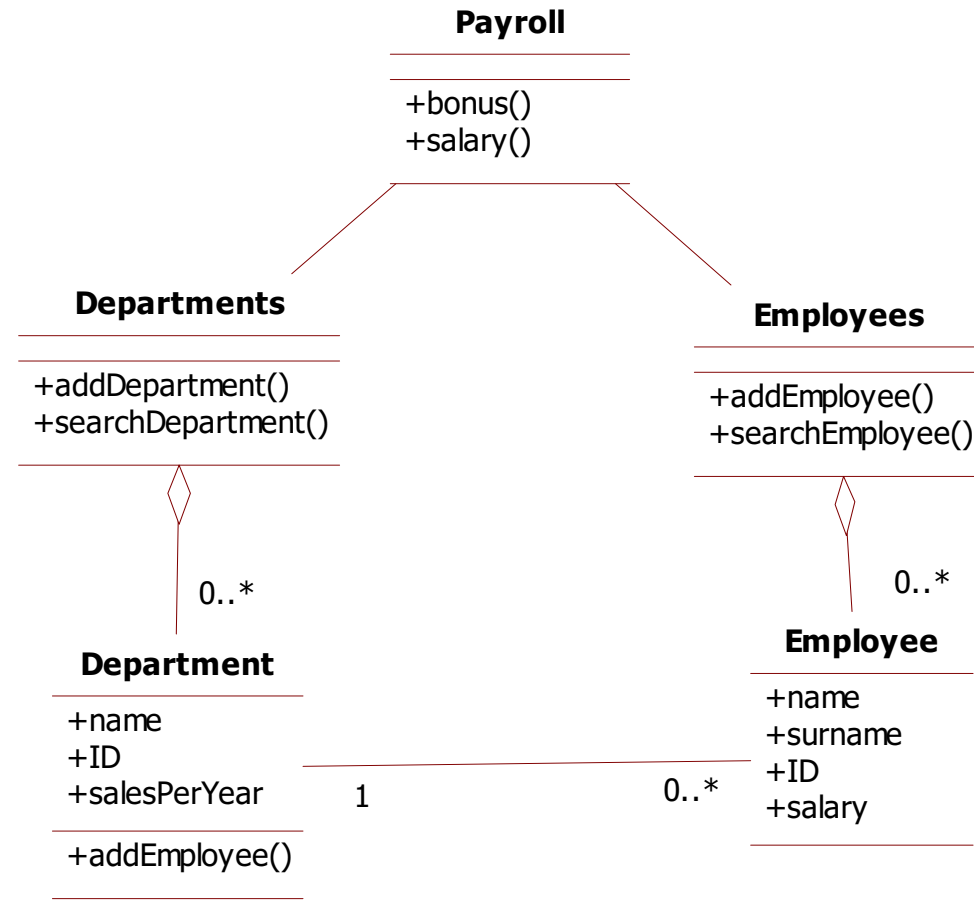
Problems

- When a defect is found, how to locate it?
 - Could be generated
 - by any function
 - function1, 2, 3, 4
 - by any interaction
 - function1 to function2, function1 to function3, function2 to function4
 - Interaction problems: bad parameter passed, parameter passed in wrong order, in wrong timing

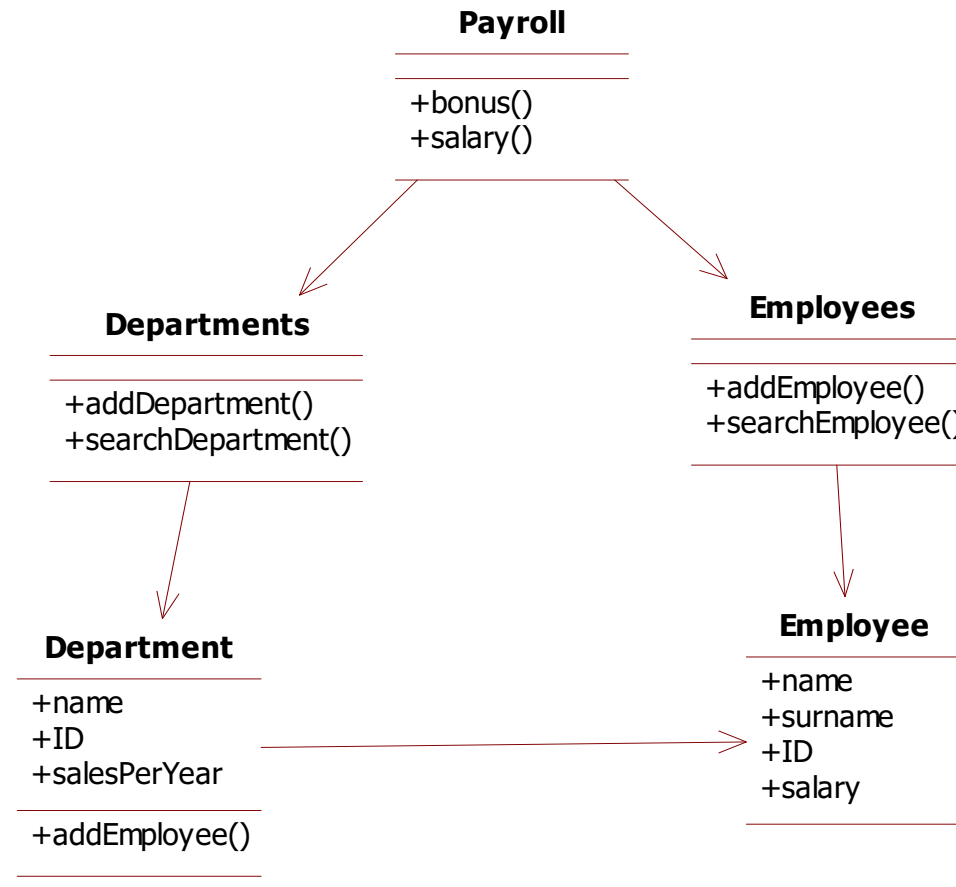
Example

- A company has employees and departments, each employee belongs to a department
- Payroll
 - salary(ID) – returns salary given employee ID
 - bonus(amount) – gives bonus 'amount' to employees with higher sales record
- Employee
 - name, surname, ID, salary
- Department
 - name, id, salesPerYear
 - addEmployee
- Employees
 - add employee
 - search employee, returns employee given ID
- Departments
 - add department
 - search department, returns department given ID

Class diagram (1)



Dependencies (1)



Integration

- Bottom up
 - Employee
 - Employees Department
 - Departments
 - Payroll
- Top Down
 - Payroll
 - Departments, Employees

BU - 1

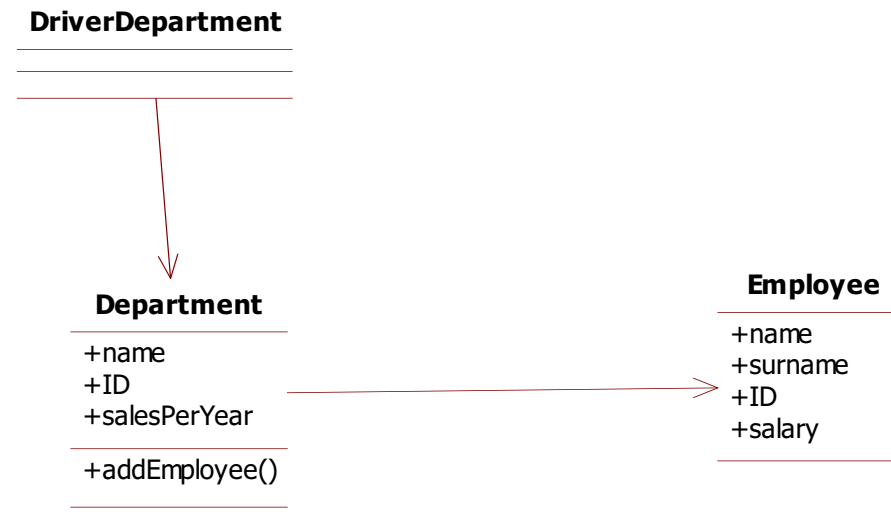
DriverEmployee



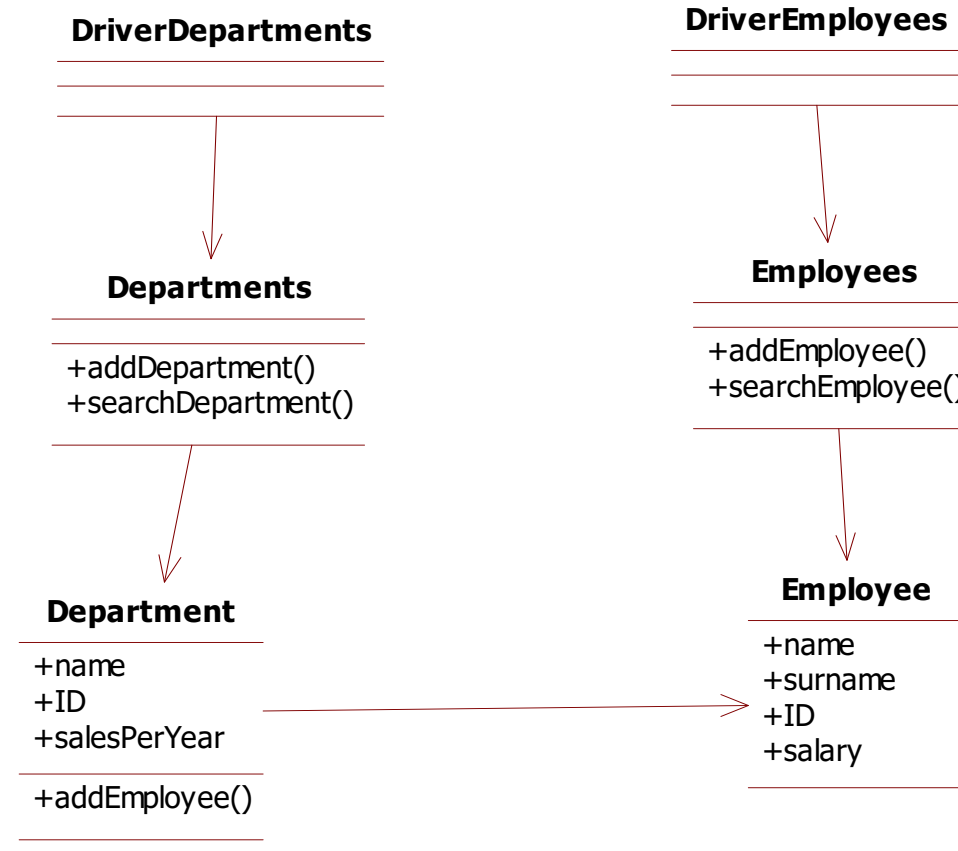
Employee

+name
+surname
+ID
+salary

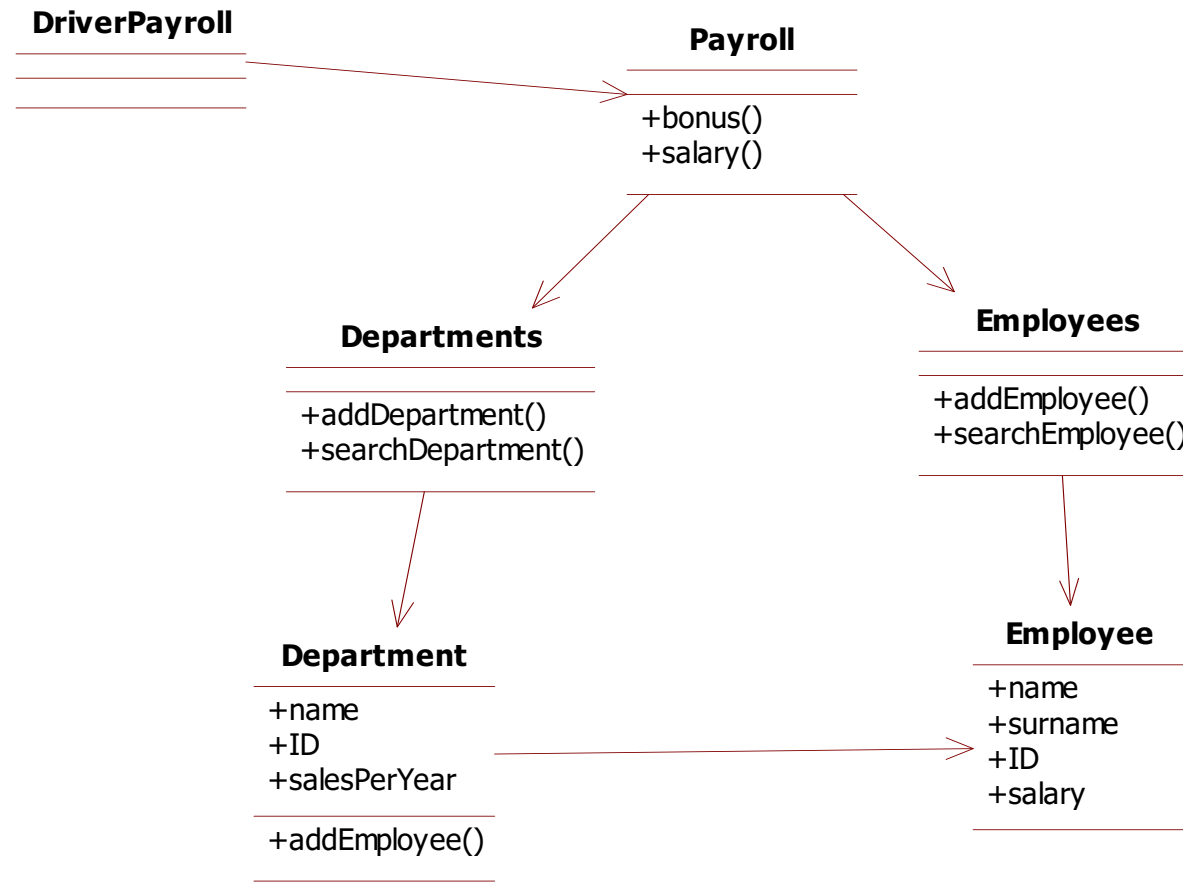
BU - 2



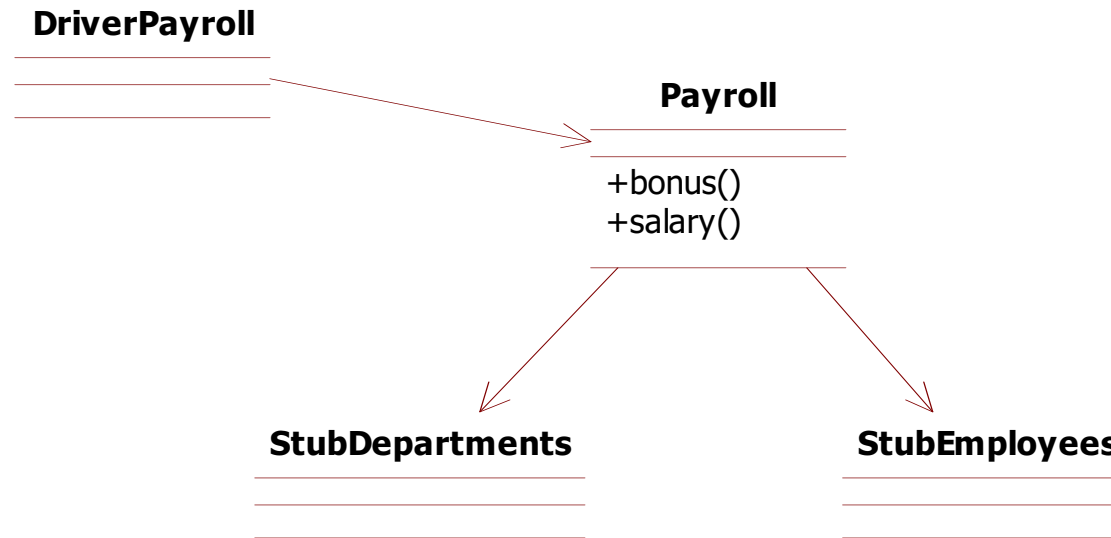
BU - 3



BU - 4

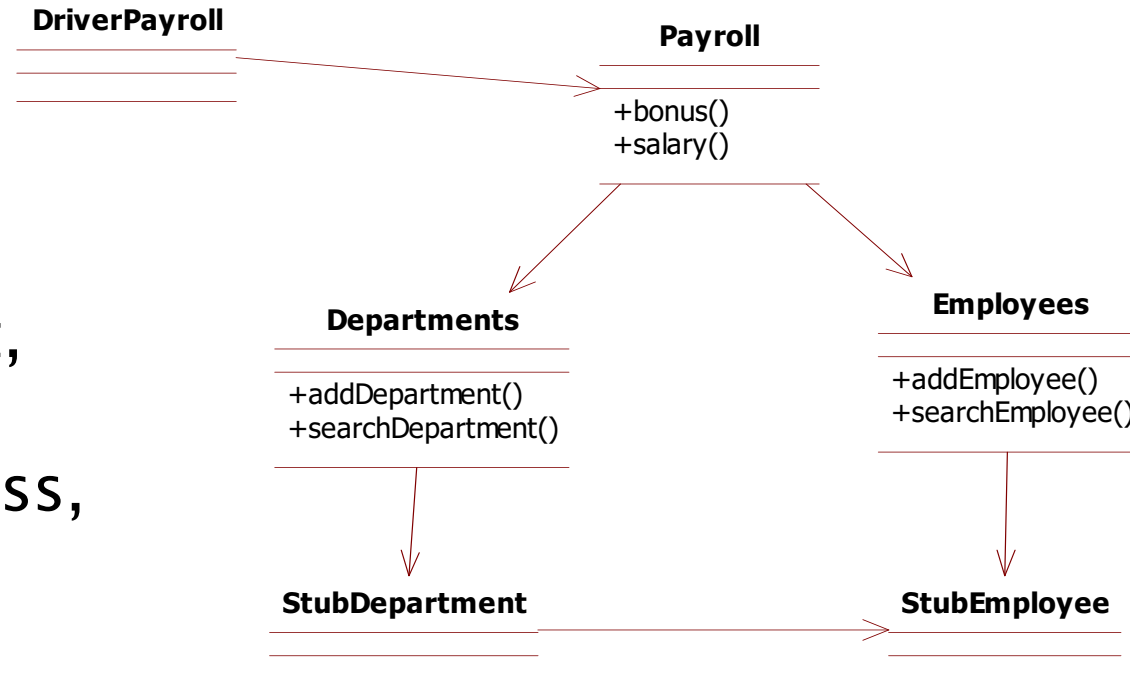


TD - 1

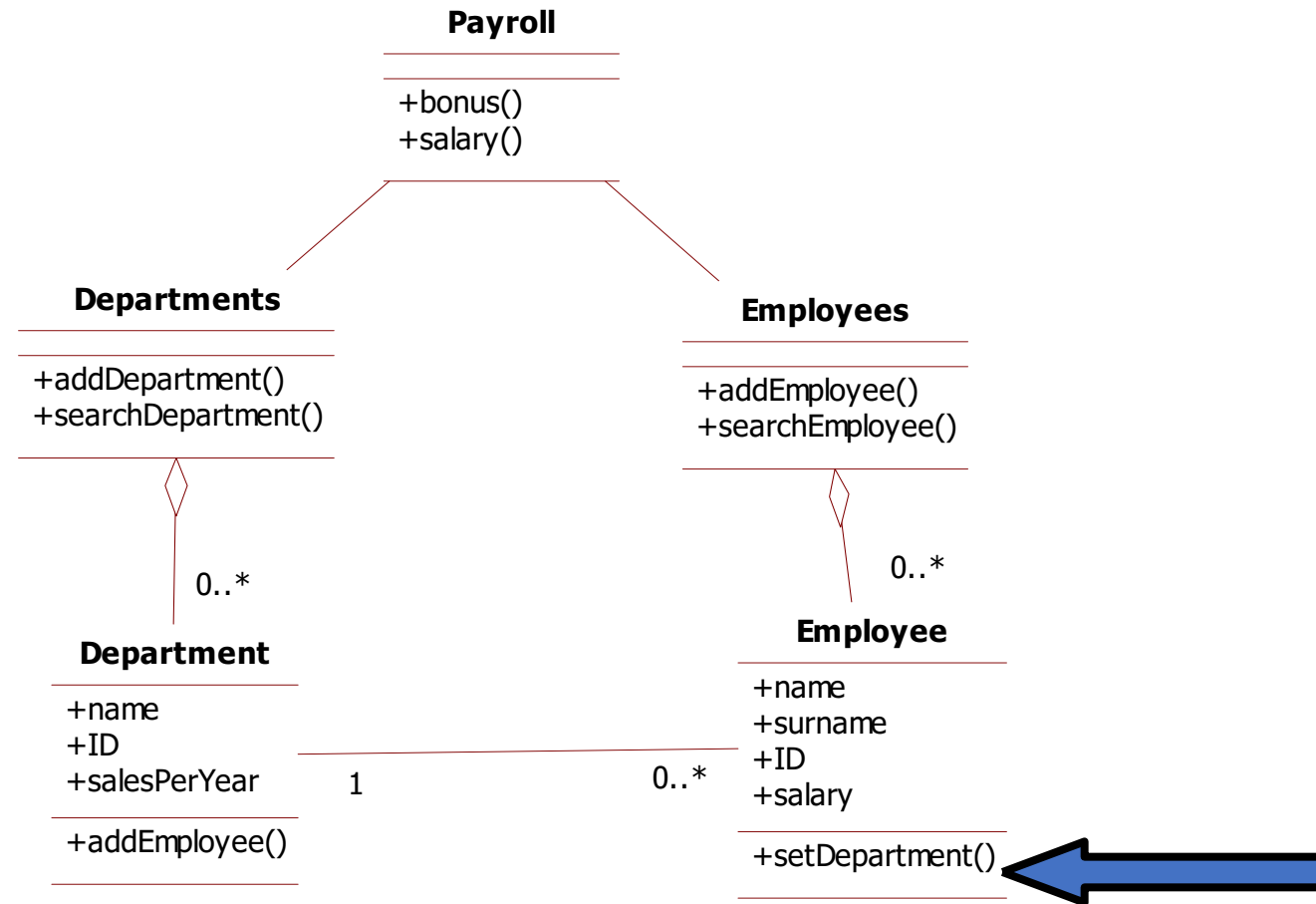


TD - 2

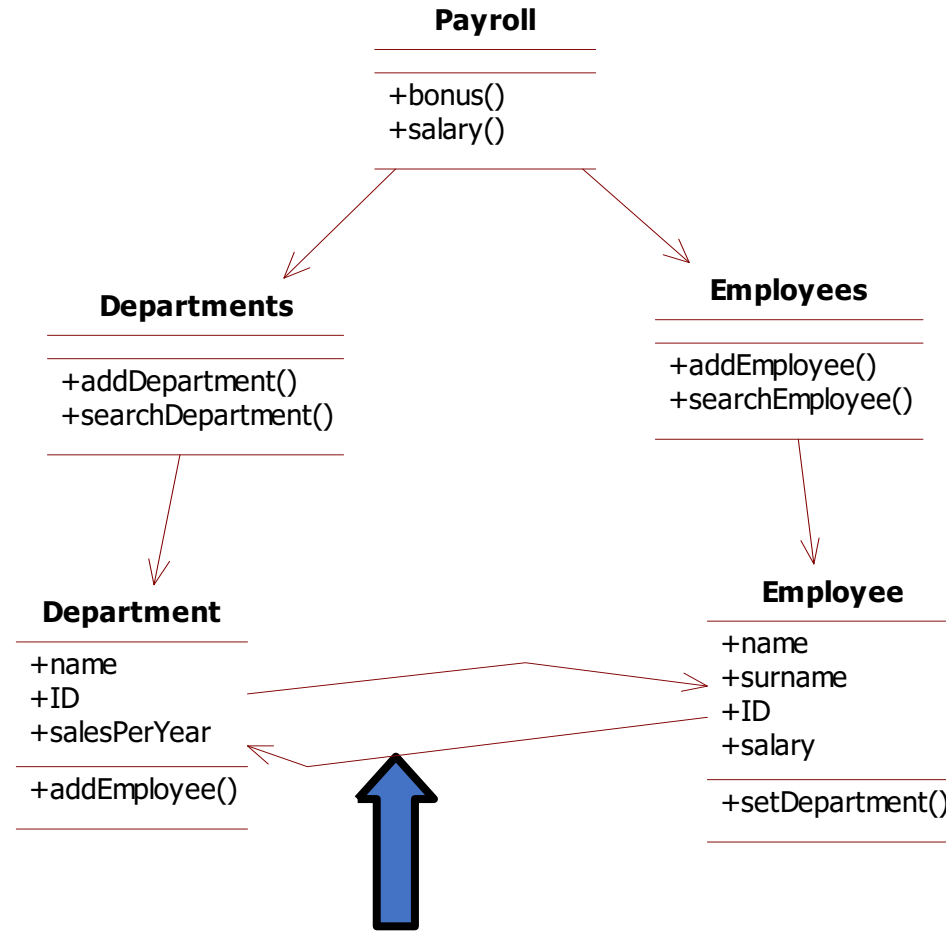
StubDepartment,
StubEmployee
probably useless,
have similar
complexity of
Department and
Employee



Class Diagram (2)

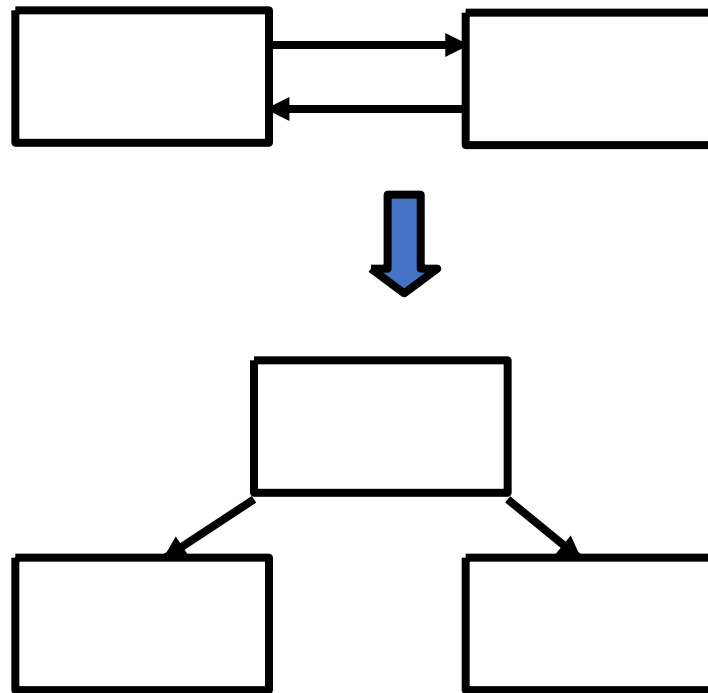


Dependencies (2)

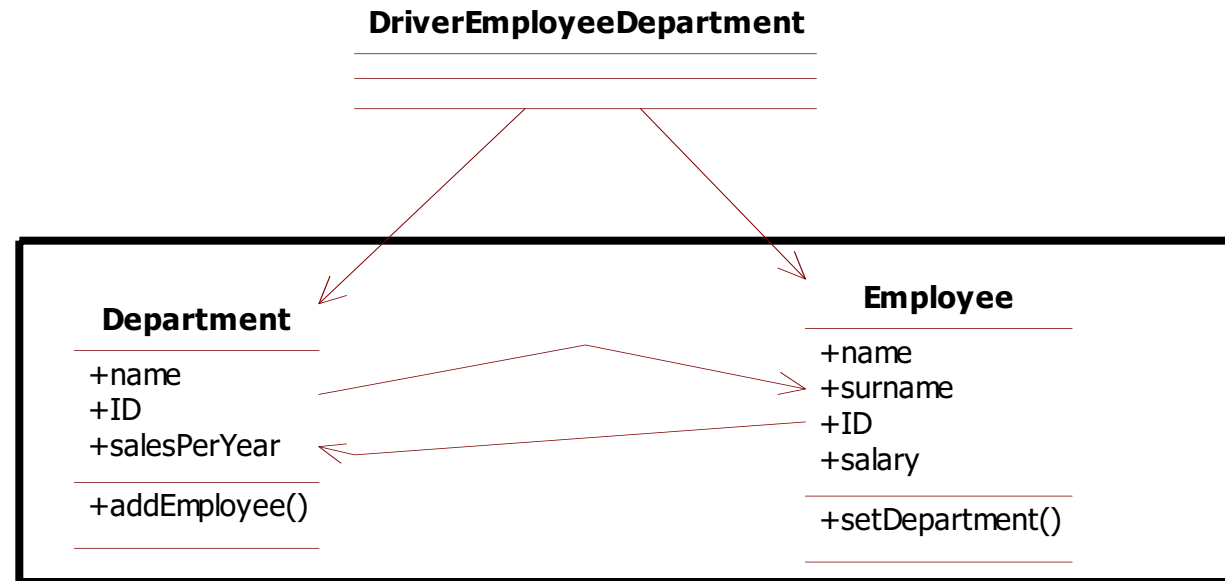


Integration

1. Consider classes with dependency loop as single class
Not feasible if large/complex classes
2. Change design, split loop



Case 1 - BU



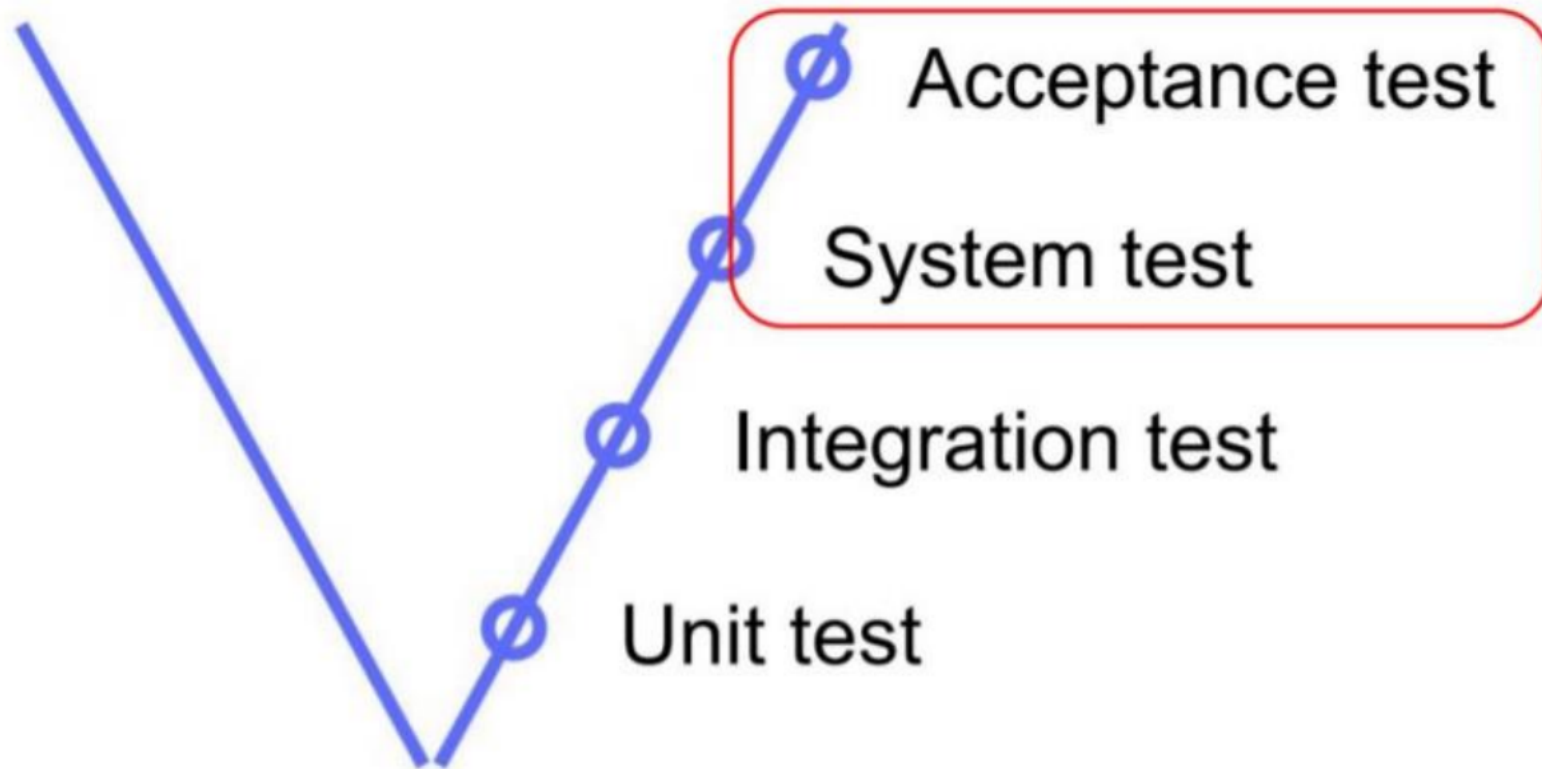
- Next steps as for BU case

Hw sw integration (embedded sw)

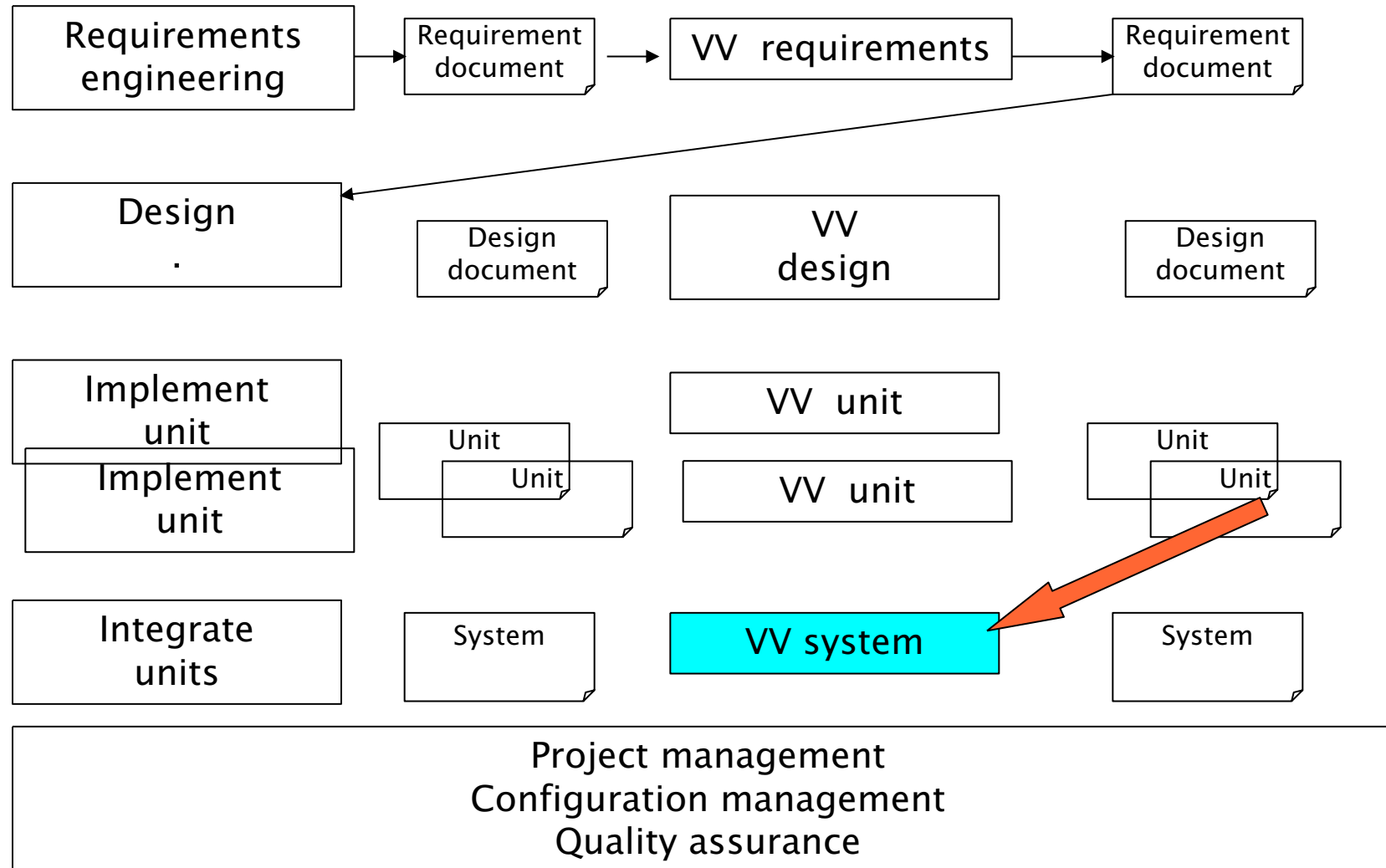
1. Test software units with stubs replacing hardware (actuators, sensors)
ex: sensor temperature → sw stub
development environment (ex MS Windows on Intel cpu)
2. Integrate software + hardware
target environment (ex Android on ARM device)

- Usually writing stubs for sensors / actuators implies defining a simulated / emulated model of the context of the sw application (as defined by context diagram and system design)

System test



System test



System test

- Test of all units composing an application
 - Unit:
 - function (procedural languages)
 - class and its methods (oo languages)

System test

- Focuses on
 - Functional properties (functional test)
 - Aka, the last stage of integration testing
 - Non-functional properties (..ilities)
- Considers different platforms
 - Development
 - Production
- Considers different players
 - Developers / testers
 - End users

System test and platforms

The platform

- Environment where an application runs, defined by
 - Operating system
 - Database
 - Network
 - Memory
 - CPU
 - Libraries
 - Other applications installed
 - Other users
 - ...

Platform and test

- An element (system, unit, ..) can be tested on
 - Target / operation / production platform
 - Where the element will run for day by day use
 - **Cannot** be used for development
 - Risk of corrupting data
 - Availability
 - Development platform
 - Where the element is developed
 - Cannot be (in most cases) equal to the target platform

Platforms, examples

- Embedded system
 - ABS for car, heating control system, mobile phone
 - Development platform is typically PC, external devices simulated/emulated
 - Production platform: car, house, phone
- Information system
 - Bank account management, student enrollment management
 - Development platform is PC or workstation, database replicated in simplified form
 - Production platform: actual database

What can change in system testing

- Who performs it
 - End user (beta testing)
 - Developer
- On what platform
 - Development
 - Target

System test and players

System test and players

- Developer, development platform
- Developer, target platform
- End user, development platform
- End user, target platform

- System test performed by end users is also called
 - Acceptance testing
 - Context: custom development, acquirer formally accepts the system, if the test is positive, the project ends, payments can proceed
 - Test cases: written by the acquirer
 - Beta testing
 - Context: mass market products, a subset of end users try the product before mass market release
 - Test cases: Informal

System test and test types

System test

- Test of functional properties
- Test of non functional properties

System test

- Test of functional properties (or functional requirements)
- Starting point is requirements document
 - Coverage of functional requirements
 - Coverage of use cases / scenarios
- Consider usage profile (the most common, typical ways of using the system)
 - Cfr. Unit and integration test, goal is coverage, using all functions, all code.

Usage profiles

- Not all functions / use cases are equal
 - In many commercial products end users use 5-10% of all available functions
- Test more / test first the functions that are used more

Usage profile

Function / scenario	Frequency of usage
F1	10
F2	10
F3	80

User profiles

Advanced user profile	Function / scenario	Frequency of usage
	F1	20
	F2	20
	F3	60

Beginner user profile	Function / scenario	Frequency of usage
	F1	0
	F2	10
	F3	90

System test

- Test of non functional requirements
 - Non functional properties are usually system, (emerging) properties. In many cases only testable when system is available
 - See efficiency, reliability

Non-functional properties

- **Usability, reliability, portability, maintainability, efficiency** (see ISO 9126)
- **Configuration:** the commands and mechanisms to change the system
- **Recovery:** the capability of the system to react to catastrophic events
- **Stress:** reliability of the system under limit conditions
- **Security:** resilience to non authorized accesses

System test - variants

- Acceptance testing
 - Data and test cases provided by the customer, on target platform
- Beta-testing
 - Selected group of potential customers

Test, in summary

	Functional/non functional	Who tests	Platform	Techniques
Unit test	Functional / structural	Developer or test group	development	BB, WB
Integration test	Functional	Developer or test group	development	Incremental TD or BU
System test	Functional + non functional	Developer or test group or user	development, target	Requirement coverage Use cases coverage Usage profiles Nf properties

Testing classification (2)

	Phase		
	Unit	Integration	System
Functional / black box	X	X	X
Structural / white box	X	dependencies	
Reliability			X
Risks			X

Coverage

- Entities considered by at least one test case / total entities
- 'Entity' depends on type of test
 - Test cases
 - Requirement
 - Function
 - Structural element
 - Statement, decision, module

Coverage

Object tested	Coverage	
Unit (class)	Methods / functions	Consider methods of the class At least one test case per method
	Black box, partitions	Consider a method Define partitions One test case per partition
	Black box, boundary	Define partitions One test case per boundary
	Black box, random	Generate randomly test cases (few % of all possible inputs)

Coverage

Object tested	Coverage	
Unit (class)	White box	Consider a method Statement coverage Decision coverage Consider a loop in a method Loop coverage

Coverage

Object tested	Coverage	
Integration (some classes)	dependencies	At least one test case per dependency

Coverage

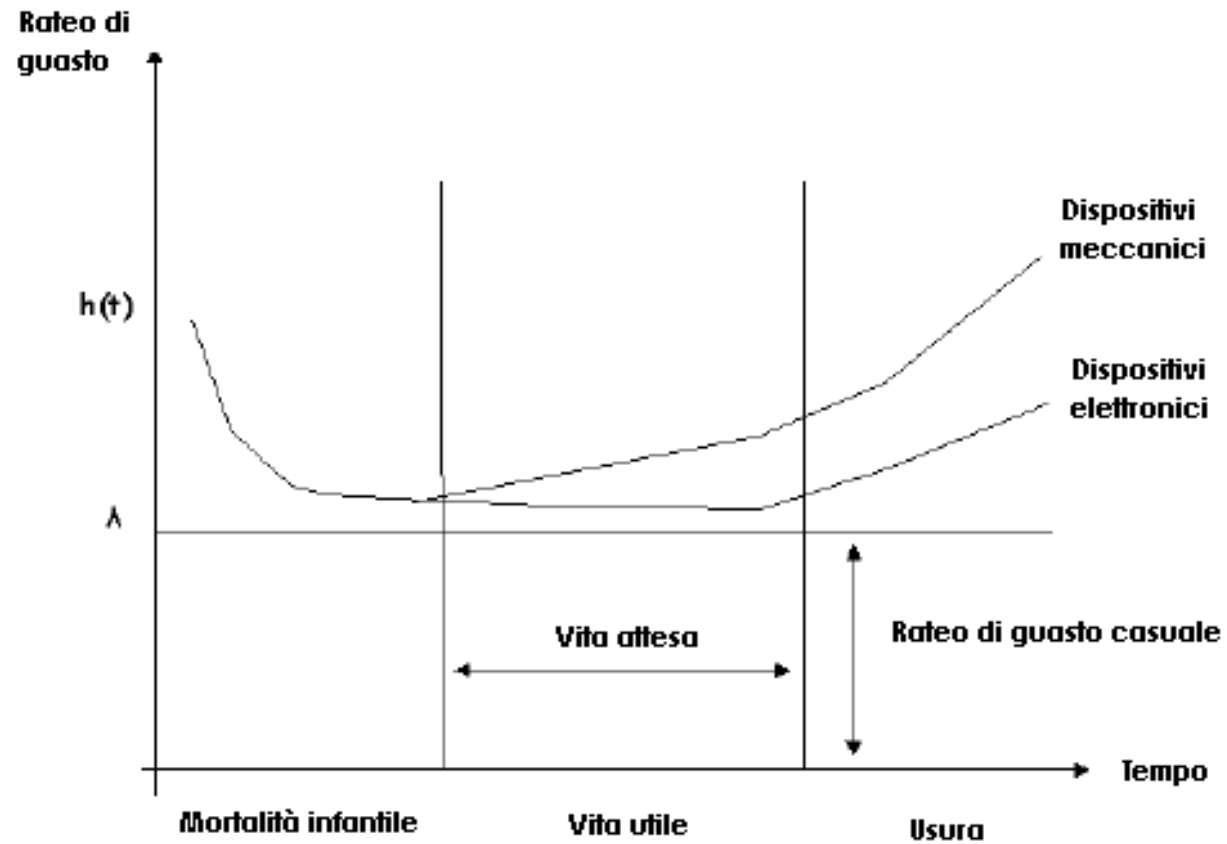
Object tested	Coverage	
System (all classes)	Functional requirements	At least one test case per requirement (may correspond to testing methods of 'main' class at end of integration testing)
	Scenarios	At least one test case per scenario
	Non functional requirements	At least one test case (JUnit) per non functional requirement (efficiency)
		Meaningful tests for usability, portability, etc

Reliability testing

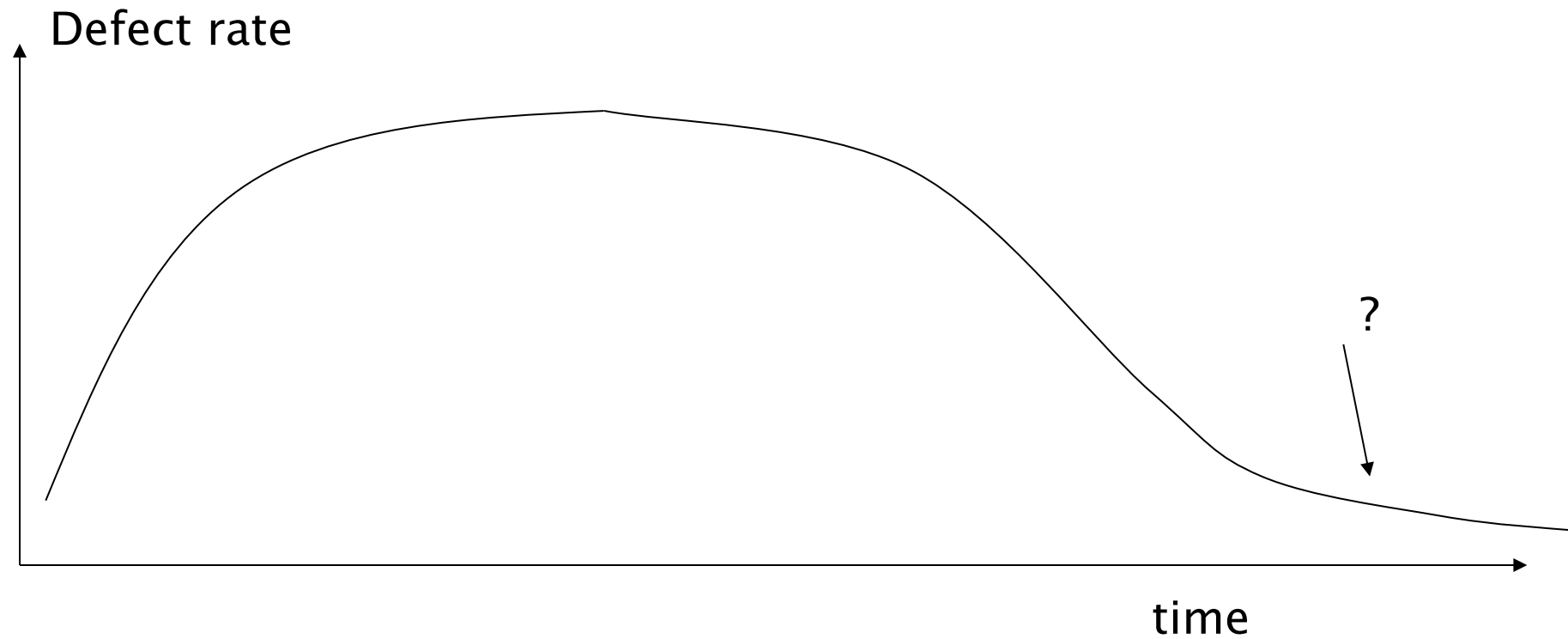
- A variant of system testing
- Aims at providing estimate of reliability
= $P(\text{failure over period of time})$
- Other measures of reliability:
 - defect rate = defect/time
 - MTBF = mean time between defects

- Constraints
 - Large number of test cases
 - Independent
 - Defect fix does not introduce other defect

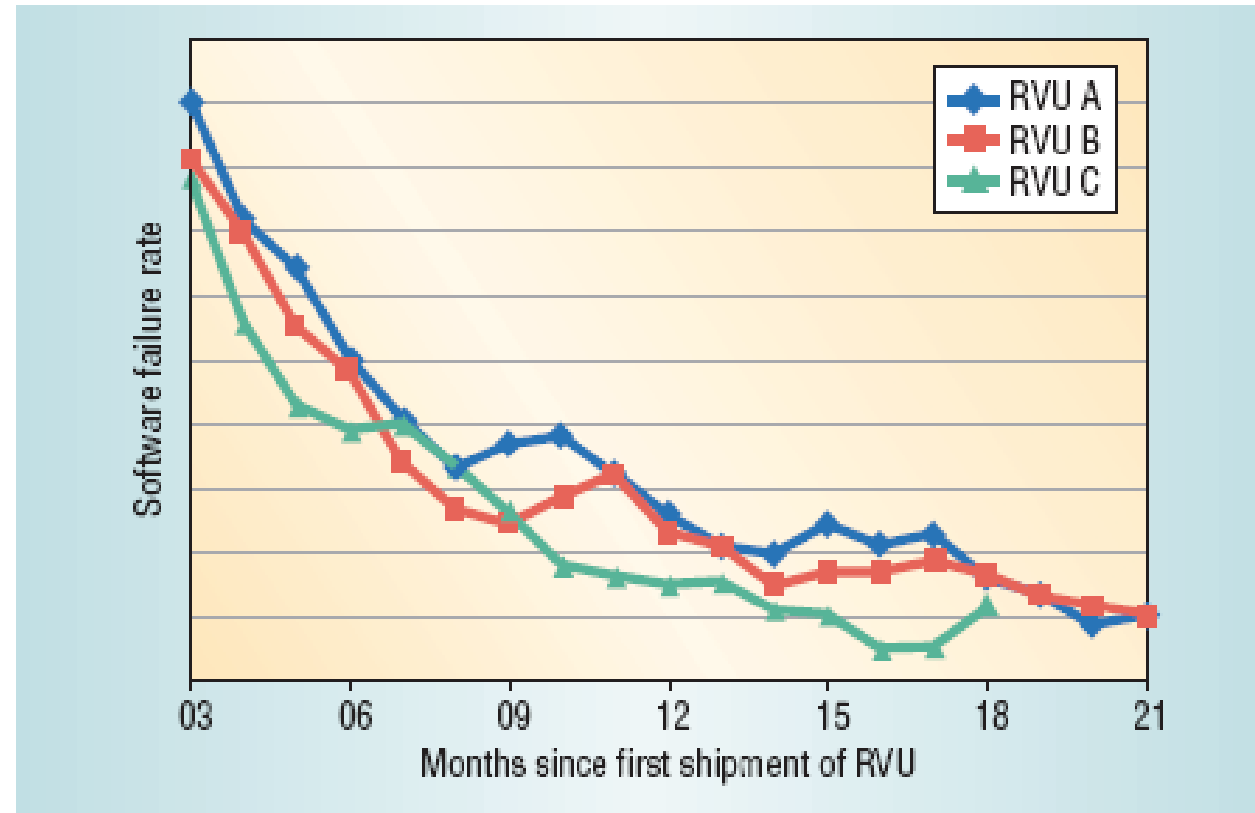
Hw reliability



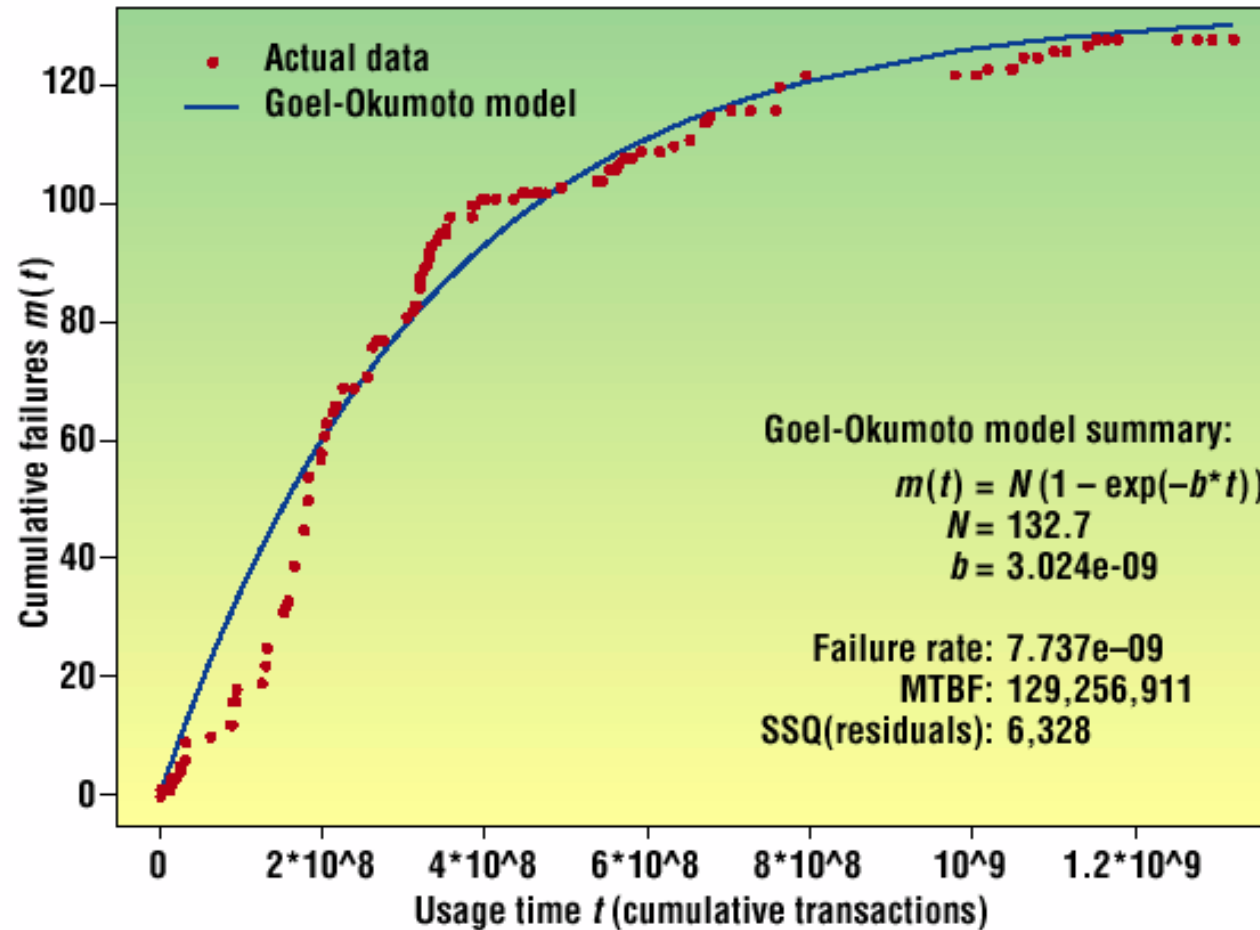
Sw reliability



HP



IBM application



Risk based testing / safety

- A variant of system testing
- Identify risks
- Characterize risks: probability, effect
- Rank risks
- Handle risks

Ex. ABS (Anti Lock Brake)

- Risks:
 - No brake when push pedal
 - Brake when no push on pedal
 - Push pedal and wheels block
- Rank risks
- Develop tests to cover more risky situations

User profiles based testing

- Variant of risk based testing
- Identify user profiles, and usage profiles
- Rank them by frequency of usage
- Test more used profiles

Ex. MS Word

- 5% of functions are used by 90% of users
- Focus test effort on those functions first
 - Ex allocate 90% of test effort on 5% of functions

Regression testing

- Regression testing
 - Tests previously defined are repeated after a change
 - To assure that the change has not introduced defects
 - Time0
 - Element (unit, system) in v0, test set t0 is defined and applied, all tests pass
 - Time1
 - Element is changed to v1
 - Test set t0 is re-applied, do all tests still pass?

Test, documentation and automation

The problem

- Test cases should be not only documented
 - So that they are not lost, and can be reapplied
 - Cfr. Test cases are just invented and applied
- But also automated
 - So that application of test cases is fast and error free
 - Cfr manual application of test cases

- Informal testing
 - Test cases are not documented
 - Invent inputs on the fly, apply them, check output manually
- Formal testing
 - Non operational (Word, excel..)
 - Operational (JUnit CUnit Jest Mocha etc)

Representing test cases

- Not operational
 - i.e. Word document
- Operational
 - Web page + translator to programming language
 - Fitnesse, FIT or SLIM
 - Scripting or Programming language
 - JS, Jest
 - Java, Eclipse + Junit
 - (similar for C, C#, http, perl,..)

Economics for test automation

- Costs
 - E_w : Effort to invent test case
 - E_a : Effort to document and automate test case
 - E_r : Effort to run test case

Automation

- Iteration 1: $E_w + E_a + E_r$
- Iteration 2: E_r
- Iteration 3: E_r
- ...
- Iteration n: E_r

No automation

- Iteration 1: $E_w + E_r$
- Iteration 2: $E_w + E_r$
- Iteration 3: $E_w + E_r$
- ...
- Iteration n: $E_w + E_r$

Break even point:

$$E_a / E_w < n - 1$$

Economics of test automation

- Automating a test case is worth if it is executed (without changes) many times
 - Rule of thumb: $n > 2, 3$ times

Goodness of test cases

Good test case

- A good test case satisfies the following criteria:
 - Reasonable probability of catching an error
 - Does interesting things
 - Does not do unnecessary things
 - Neither too simple nor too complex
 - Makes failures obvious
 - Mutually Exclusive, Collectively Exhaustive

Mutation testing

Mutation Testing

- Are our test cases 'good'?
- Idea:
 1. write test cases
 2. inject errors in program (single small change)
 3. verify if test cases catch the errors injected

Mutation Testing

- Mutation Testing (a.k.a., Mutation analysis, Program mutation)
 - Introduced in early 1970s
 - Used in software/hardware

Terms

- *Mutant*: program with one change
- *Killable mutant*: non functionally equivalent. A test case can kill it
- *Equivalent mutant*: functionally equivalent to program. No test case can kill it.
- *Mutation score*:
 - property of a test suite (goal: 100%)
 - Killed non-equivalent mutants / all non-equivalent mutants

Mutations

- Common mutations
 - Delete a statement
 - Swap two statements
 - Replace arithmetic operation
 - Replace boolean relation
 - Replace a variable
 - Replace boolean subexpression with constant value

Mutation examples

```
function sum(a: number, b: number): number {  
    return a + b;  
}
```

```
function sum(b: number, a: number): number {  
    b--;  
    return a + b;  
}
```

```
function sumIncrement(a: number, b: number): number  
{  
    a++;  
    return a + b;  
}
```

```
function sum(a: number, b: number): number  
{  
    return a * b;  
}
```

...

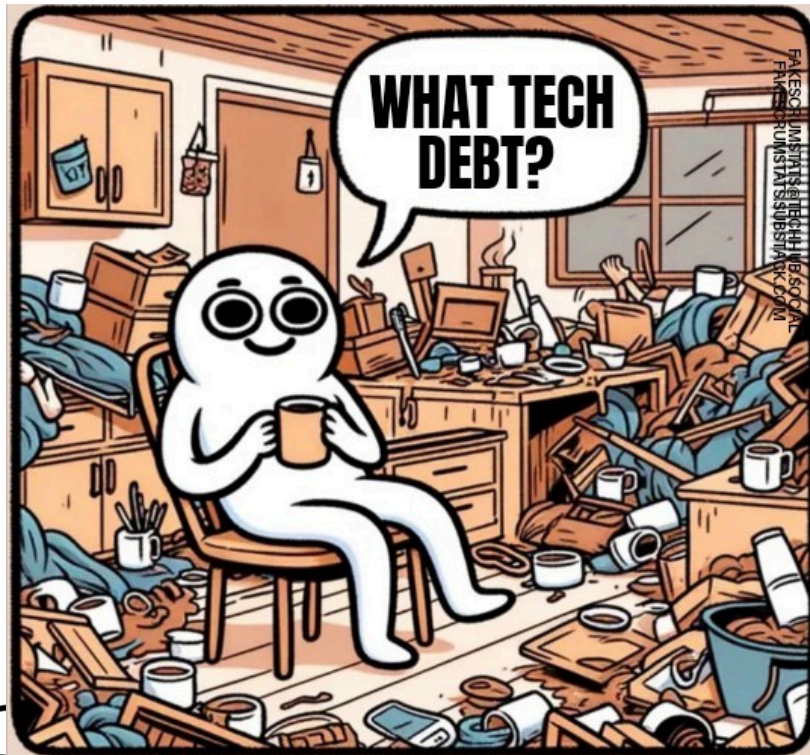
Mutations in Typescript

- Stryker
 - `npm install --save-dev @stryker-mutator/core`
 - `npm install --save-dev @stryker-mutator/typescript`
 - `npm install --save-dev @stryker-mutator/jest-runner`
- `npx stryker init`
- `npx stryker run`

Code and Test Smells

Bad Smells (Fowler)

- Fowler et al., Refactoring, Improving quality of existing code
- <https://refactoring.guru/refactoring/smells>



Bad code smells

- A *bad code smell* is an indication of a potential problem in the source code
- bad code smell may not definitively identify an error, it does suggest a potential trouble spot, that is, a place where there is an increased risk of a future failure.
- The presence of bad code smells is symptomatic of developers failing to follow good design principles, increasing the technical debt

Bad smells

- Duplicated code
- Long method
- Large class
- Long parameter list
- Divergent change
- Shotgun surgery
- Feature envy
- Data clumps
- Primitive obsession
- Switch statements
- Parallel inheritance hierarchies
- Lazy class
- Speculative generality
- Temporary field
- Message chain
- Middle-man
- Inappropriate intimacy
- Alternative classes with different interfaces
- Incomplete Library class
- Data class
- Refused bequest
- Comments

Test Smells (Van Deursen)

- Van Deursen et al., Refactoring Test Code
- <https://test-smell-catalog.readthedocs.io/en/latest/>

Test smells

- **Mystery Guest**
using external resources, e.g. file and test is not self-contained anymore
- **Test Run War**
arise when the tests run fine as long as you are the only one testing but fail when more programmers run them
- **Lazy Test**
occurs when several test methods check the samemethod using the same fixture
- **Resource Optimism**
Test code that makes optimistic assumptions about the existence of external resources
- **General Fixture**
Happens when the setUp method is too general and different tests only access part of the fixture
- **Eager Test**
test method checks several methods of the object to be tested

Test smells

- **Assertion Roulette**
This smell comes from having a number of assertions in a test method that have no explanation
- **Sensitive Equality**
It is a smarter way to write equality checks using the toString method. A typical way is computing the actual result, map it to a string, compared to the expected value
- **Test Code Duplication**
Test code may contain undesirable duplication
- **Indirect Testing**
when a test class contains methods that actually perform tests on other objects
- **For Testers Only**
When a production class contains methods that are only used by test method
- **Magic Number**
When using a pure number as an oracle for an assertion, instead of defining a constant in the code

Testing tools for Typescript

- Unit & Integration Testing

- Jest: Comprehensive testing solution for JavaScript and TypeScript.
- Mocha + Chai: Flexible testing with powerful assertion libraries.
- AVA: Modern test runner with parallel execution capabilities.

- Mocking Tools

- ts-mockito: TypeScript mocking library inspired by Mockito, providing type-safe mocking.
- Sinon.js: Versatile mocking library that works with any testing framework.

- Code Coverage Tools

- Istanbul (nyc): Popular tool for tracking code coverage, supports TypeScript out-of-the-box.
- Jest: Offers built-in support for code coverage analysis.

Testing tools for Typescript

- GUI Testing Frameworks

- Cypress & Selenium: End-to-end testing for anything that runs in a browser.
- Puppeteer: Node library to control headless Chrome, ideal for SPA testing.
- TestCafe: Browser testing without plugins, supports real user simulation.

- Profiling Tools

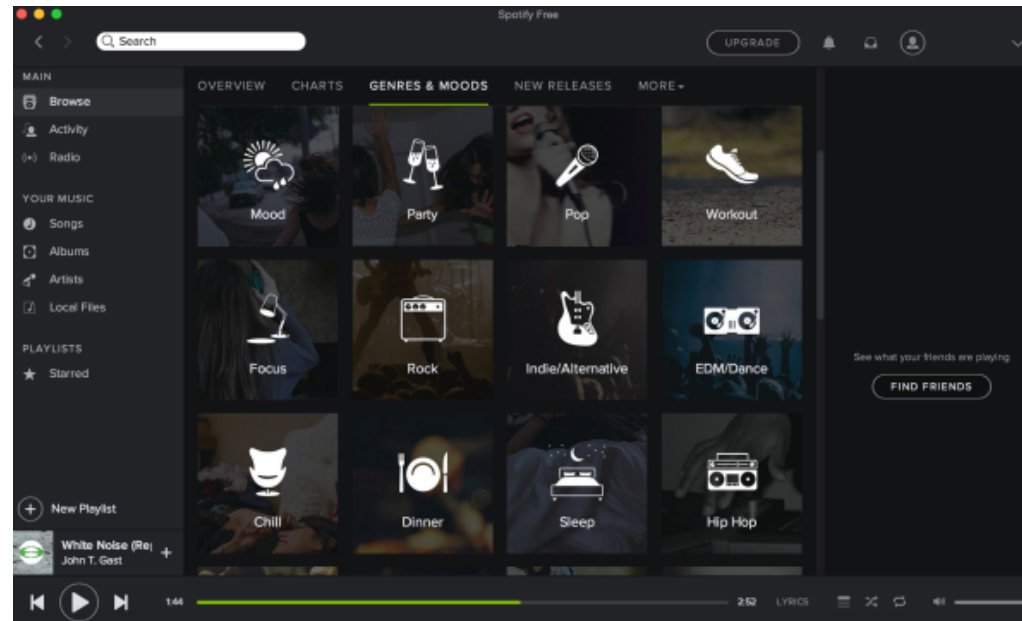
- Node.js Built-in Profiler: Analyze performance and resource usage within Node.js environments.
- Visual Studio Code Extensions: Tools like "Debugger for Chrome" and "JavaScript Profiler" for in-IDE profiling.
- Chrome DevTools: Excellent for frontend application profiling with proper source map configuration

GUI Test tools

- GUI testing
 - A variant of system testing

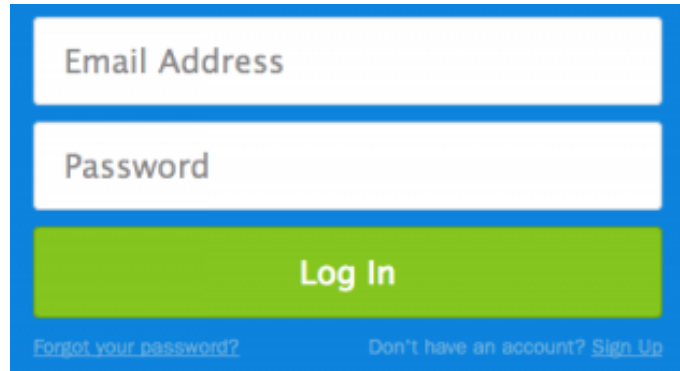
What is GUI Testing

- **GUI Software:** any software provided with a GUI (Graphical User Interface).
- **GUI Testing** is the practice of testing a software application through its user interface.



What is GUI Testing for

- **Functional tests:** black box tests exercising the basic functionalities of an application through interaction with the GUI, without knowing the source code.



Email Address

Password

Log In

[Forgot your password?](#) [Don't have an account? Sign Up](#)

Example:
Test that the log-in
functionalities behave
properly.

What is GUI Testing for

- **Look & Feel verification:** GUI should appear as defined in the requirements document, or in the mockups provided by the stakeholders.
- **Compatibility testing:** testing that the application is deployed and behaving properly on different devices, screens and layouts (device diversity).

Approaches to GUI Testing

- Manual
- Scripted
- Capture & Replay
- Model-based
- Visual / Image Recognition

Manual GUI Testing

Manual execution of test scenarios (informal testing).

- Easy to setup, does not require tools;
- Error prone, hardly reproducible, expensive.

Scripted GUI Testing

Development of test scripts using dedicated scripting languages.

- Scripts can be automatically executed and used for regression testing;
- Test scripts can be difficult to write, and hard to maintain during the evolution of the software.

Scripted GUI Testing

Example:

Cypress



```
describe('Login Test', () => {  
  it('Visits the login page and logs in', () => {  
    cy.visit('http://localhost:8080/login')  
    cy.get('#username').type('user1')  
    cy.get('#password').type('password1')  
  
    cy.get('#loginButton').click()  
  
    cy.url().should('include', '/home')  
  });  
});
```

Capture & Replay

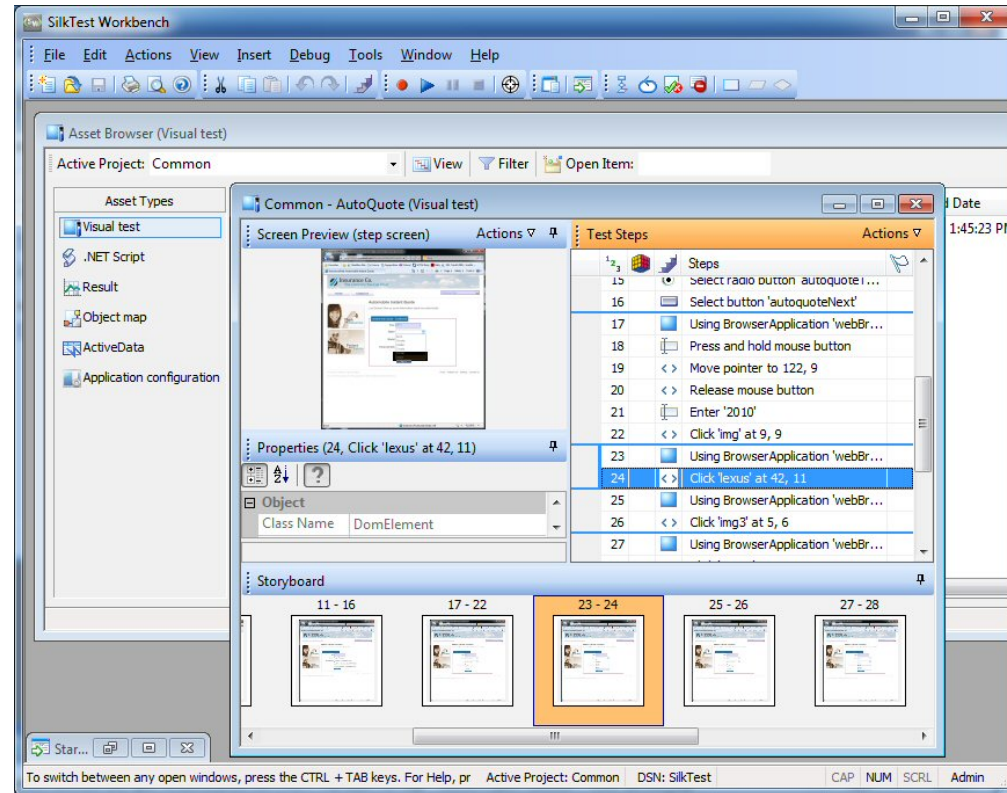
User inputs are given once to the user interface (CAPTURE), and then codified into a repeatable script (REPLAY).

- Faster and easier to obtain test scripts with respect to pure scripted techniques;
- Very fragile to the evolution of the user interface;
- Scripts must be enriched manually to perform complex operations.

Capture & Replay

Examples:

SilkTest, HP UFT
(desktop).



Model-Based GUI Testing

Test cases are obtained based on models of the user interface (e.g., oriented graphs or finite state machines).

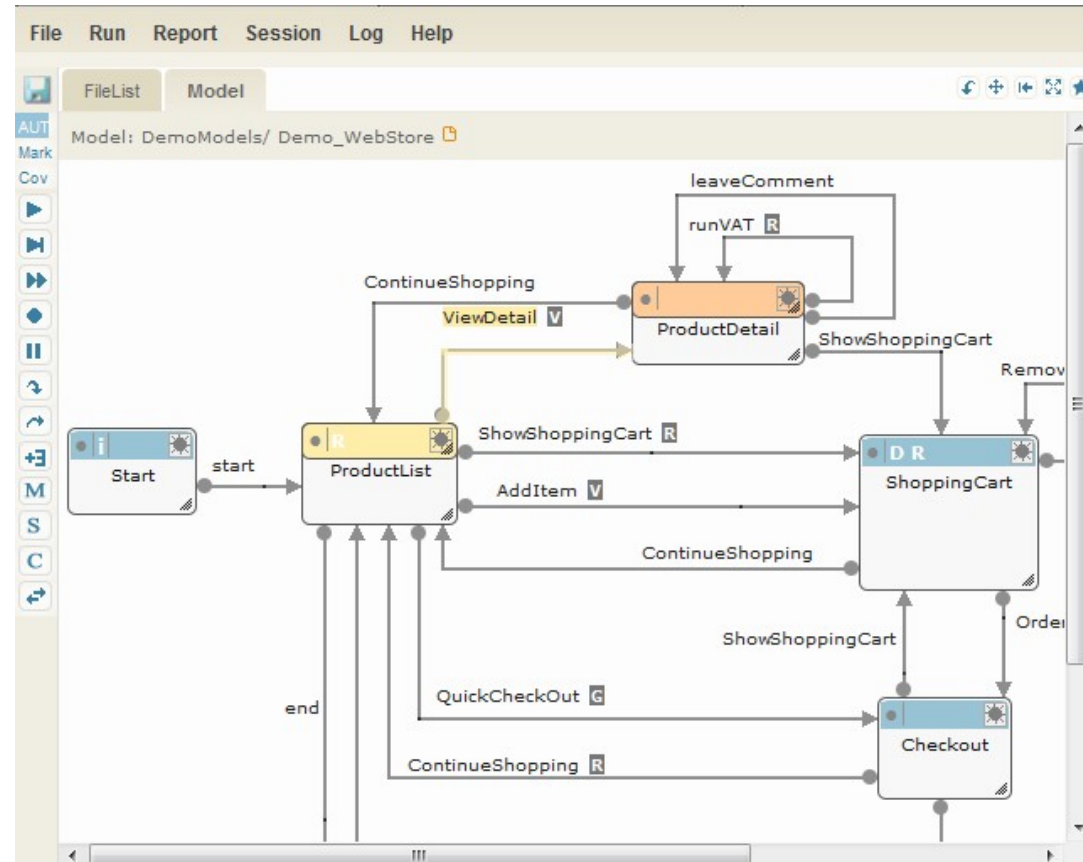
- Allows automated execution and generation of use cases;
- Very high coverage of use cases and functionalities can be obtainable once a model is available;
- Manual effort required in defining and tailoring the GUI model.

Model-Based Testing

Tools:

TestOptimal (web),

MobiGUITAR (mobile).



Visual Testing

Image recognition techniques are used to identify elements of the user interface to interact with.

- Easy definition of test cases with no need of technical knowledge – only screen captures are needed;
- Can be applied seamlessly to any kind of software provided with an (emulated) user interface;
- Very high fragility to even minor changes in the GUI;
- Difficult in-depth testing of application functionalities.

Visual Testing

Examples:

Sikuli, EyeAutomate

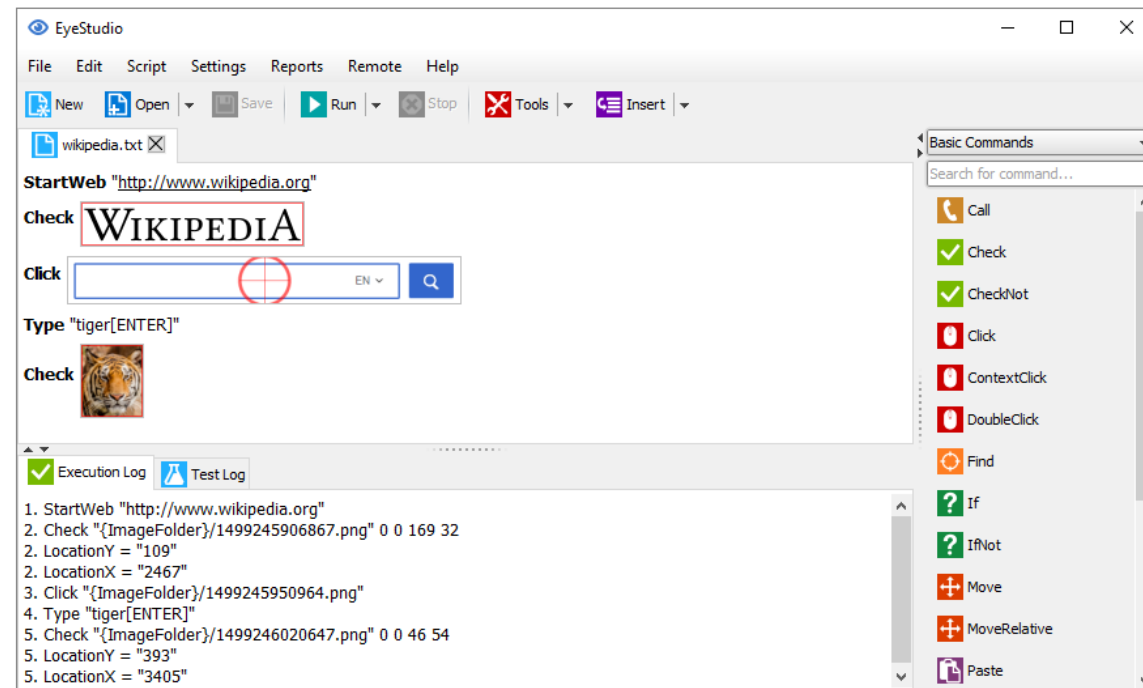


Table based

Table based testing

- Test cases are written as tables on a wiki, and linked to the application to be tested
- Pro:
 - Allows end users to write tests (especially acceptance tests, black box tests)
 - Independent of GUI
 - Allows automation
- Cons
 - Requires fixtures

- Function receives a set of words, should count number of words and occurrences of word 'black'

input	expected		actual	
	word count	'black' count	word count	'black' count
red blue	2	0		
blue black green	3	1		
white red brown black	4	1		

Coverage

- Show graphical and numerical coverage (statements, branches, conditions) on source code
- Ex., Istanbul, Jest

Profilers

- Trace time spent per function, given specific test execution
 - Performance test

Testing - certifications

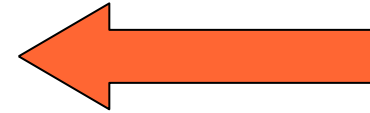
- ISTQB
 - www.istqb.org
 - Int software testing qualifications board
 - Delegations in most countries
 - In italy ita-istqb

ISTQB

- Publishes Syllabus
 - Available free
 - Foundation, advanced, expert levels
 - Core, agile, specialist tracks
- Provides Certifications, via exam
 - See foundation level

Static analysis

- Static
 - inspections
 - source code analysis
- Dynamic
 - testing



Static analysis techniques

- Compilation static analysis
- Control flow analysis
- Data flow analysis
- Symbolic execution
- Reverse documentation of design

Compilation analysis

- Compilers analyze the code checking for
 - Syntax correctness
 - Types correctness
 - Semantic correctness
- The errors detected by a compiler strongly depend on the language
 - Loose vs. strongly typed languages
 - Static vs. dynamic visibility

MISRA-C

- MISRA: Motor Industry Software Reliability Association
- Issues Misra-C, guidelines for C programs
 - Issue1, 1998
 - 127 rules, 93 compulsory
 - Issue2, 2004
 - 141 rules, 121 compulsory

Rules, examples

- 5 Use only characters in the source character set. This excludes the characters \$ and @, among others.
- 22 Declarations of identifiers denoting objects should have the narrowest block scope unless a wider scope is necessary.
- 34 The operands of the && and || operators shall be enclosed in parenthesis unless they are single identifiers.
- 67 Identifiers modified within the increment expression of a loop header shall not be modified inside the block controlled by that loop header.
- 103 Relational operators shall not be applied to objects of pointer type except where both operands are of the same type and both point into the same object.

Rule 5

signed char dollar = '\$';

- not accepted

signed char esc_m = '\m';

- not accepted (what would be the associated behaviour to this escape sequence?)

Rule 34

```
if ((var++) || (num == 11)) { ... } /* OK */
```

```
if (var++ || num == 11) { ... } /* NOT OK */
```

```
if ((vect[num]==4) && (num == 11)) { ... } /* OK */
```

```
if (vect[num] == 4 && (num == 11)) { ... } /* NOT  
OK */
```

```
if ((structure.field != 0) && (num < 11)) { ... }  
/* OK */
```


Rule 67

```
for (int i = 0; i < max; i++) {  
    i=i+1;    // NO  
}
```

Misra static analyzers

- Parse source code and check if rules are violated
 - QA-C by Programming Research, is a full featured MISRA C1 and C2 validator.
 - Testbed by LDRA, offers a static and dynamic analysis.
 - PC-Lint by Gimpel, is one of the fastest and least expensive validtors.
 - DAC by Ristan-CASE, provides a reverse engineering, documentation and code analyzer.

Code Analyzers

- ESLint
- SonarQube
- CodeClimate

CodeClimate Output

Breakdown

17 FILES



Codebase summary

MAINTAINABILITY

D 2 wks

TEST COVERAGE



Repository stats

CODE SMELLS

61

DUPLICATION

17

OTHER ISSUES

0

File `WifiWizard2.java` has 1207 lines of code (exceeds 250 allowed).

Consider refactoring. [OPEN](#)

```
1  /*
2   * Copyright 2018 Myles McNamara
3   *
4   * Licensed under the Apache License, Version 2.0 (the "License");
5   * you may not use this file except in compliance with the License.
```

Found in `src/android/wifiwizard2/WifiWizard2.java` - About 3 days to fix

`WifiWizard2` has 52 methods (exceeds 20 allowed). Consider refactoring. [OPEN](#)

```
64  public class WifiWizard2 extends CordovaPlugin {
65
66      private static final String TAG = "WifiWizard2";
67      private static final int API_VERSION = VERSION.SDK_INT;
68
69
```

Found in `src/android/wifiwizard2/WifiWizard2.java` - About 7 hrs to fix

Method `execute` has a Cognitive Complexity of 40 (exceeds 5 allowed).

Consider refactoring. [OPEN](#)

```
160  @Override
161  public boolean execute(String action, JSONArray data, CallbackContext callbackContext)
162      throws JSONException {
163
164      this.callbackContext = callbackContext;
```

Found in `src/android/wifiwizard2/WifiWizard2.java` - About 6 hrs to fix

Example with ESLint

```
21 export function badFunction(input: any): any {
22   let data = input;
23   let result = 0;
24   let a = 10, b = 20;
25   data.forEach((d: any) => {
26     if (d.someProp == undefined) {
27       console.log("Errore: proprietà mancante");
28       result += a * b;
29     } else {
30       result += d.someProp;
31     }
32   });
33   globalVar = result;
34   console.log("Il risultato è: " + result);
35   return result;
36 }
37
38 var globalVar = 0;
39
```

```
hardo@yanez: jest % npx eslint src/
```

```
/Users/hardo/git/jest/src/mathFunctions.ts
```

```
21:37 error Unexpected any. Specify a different type @typescript-eslint/no-explicit-any
21:43 error Unexpected any. Specify a different type @typescript-eslint/no-explicit-any
22:7 error 'data' is never reassigned. Use 'const' instead prefer-const
24:7 error 'a' is never reassigned. Use 'const' instead prefer-const
24:15 error 'b' is never reassigned. Use 'const' instead prefer-const
25:20 error Unexpected any. Specify a different type @typescript-eslint/no-explicit-any
38:1 error Unexpected var, use let or const instead no-var
38:5 error 'globalVar' is assigned a value but never used @typescript-eslint/no-unused-vars
```

```
* 8 problems (8 errors, 0 warnings)
2 errors and 0 warnings potentially fixable with the `--fix` option.
```

Data flow analysis

- Analyzes the values of variables during execution to find out anomalies
- Looks like dynamic but some information can be collected statically
- Three operations on variables
 - Definition: - write- a new value is assigned
 - Use: - read- the value of the variable is read
 - Nullification: the variable has no significant value

Data flow analysis

- Correct sequences
- D U
 - The use of a variable must be always preceded by a definition of the same variable
- Suspect (forbidden) sequences
- D D
- N U
 - A use of a variable not preceded by a definition corresponds to the use of an undefined value

Data flow analysis

- Tools recover the sequence and recognize suspect ones

	x1	x2	x
<code>void swap(float*x1, float* x2) {</code>	D	D	-
<code>int float x;</code>	-	-	N
<code>*x2 = x;</code>	-	D	U
<code>*x2 = *x1;</code>	U	D	-
<code>*x1 = x;</code>	D	-	U
<code>}</code>			

Note: A yellow callout box points to the code line `*x2 = x;` with the text `x = *x2;`. Blue circles highlight the 'D' and 'U' values in the x2 and x columns for the first three rows of the table.

Symbolic execution

- The program is executed with symbolic values instead of actual values
- Output variables are expressed as symbolic formulas of input variables
- Symbolic execution may be extremely complex even for simple programs

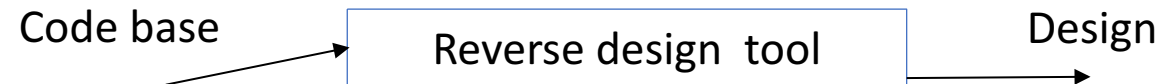
Symbolic execution

```
1 function product(x: number, y: number, z: number): number {  
2   let tmp1: number, tmp2: number;  
3   tmp1 = x * y;  
4   tmp2 = y * z;  
5   return tmp1 * tmp2 / y;  
6 }
```

<i>Stmt</i>	<i>x</i>	<i>y</i>	<i>z</i>	<i>tmp1</i>	<i>tmp2</i>	<i>product</i>
2	X	Y	Z	?	?	?
3	X	Y	Z	X*Y	?	?
4	X	Y	Z	X*Y	Y*Z	?
5	X	Y	Z	X*Y	Y*Z	X*Y*Y*Z / Y

Reverse documentation of design / code

- Over time the code changes and may or may not correspond to the initial design
- How to reconstruct the actual, current design?



Testing and quality

- Analysis of defects found in a software project is a key tool to assess the quality of a software product
 - See before 'reliability testing'
 - See chapter on project management

Summary

- VandV is about preventing, finding and fixing defects
- Many techniques available: testing, inspection, static analysis
- Cost and effort of VandV should be compared with cost of defects left to the end user
 - Cost of quality vs. cost of non-quality